

REPUBLIQUE DU SENEGAL

Un Peuple – Un But – Une Foi



École 221

ECOLE 221

Spécialité : Développement Web

MEMOIRE TECHNIQUE DE SOUTENANCE

Module JavaScript

GesPharmacie / PHARMA 221

Mémoire technique du backend, API de gestion de pharmacie développée avec Express, TypeScript et Prisma

Présenté par :	Papa Mamlick Teuw
Encadré par :	M. DIOUF
Année académique :	2025–2026
Spécialité :	Développement Web
Module :	JavaScript
Projet :	GesPharmacie / PHARMA 221
Perimetre :	API backend Express, TypeScript, Prisma et PostgreSQL
Nature du document :	Memoire technique academique
Etablissement :	Ecole 221
Date de soutenance :	2025-2026

Table des matières

Résumé	6
1 Introduction générale	7
1.1 Objet du mémoire	7
1.2 Référentiel d'analyse	7
2 Objectif du projet	9
2.1 Ce que le projet doit résoudre	9
2.2 Ce que le backend doit produire	9
2.3 Pourquoi le mémoire se concentre sur le backend	9
3 Partie 1. Initialisation du projet	10
3.1 Préparer l'environnement avant d'écrire le premier module	10
3.2 Organiser le travail dès le début	11
3.3 Créer le socle HTTP du projet	13
3.4 Mettre en place les fondations techniques avant les modules	14
3.4.1 Validation de l'environnement	14
3.4.2 Journalisation	15
3.4.3 Accès à la base de données	15
3.4.4 Outils transverses	16
3.5 Préparer les portes d'entrée avant les fonctionnalités	18
4 Partie 2. Étude détaillée de l'architecture choisie	20
4.1 Architecture retenue	20
4.2 Pourquoi cette architecture est adaptée	21
4.3 Capture directe de l'architecture source	21
4.4 Comment une requête traverse l'application	22
4.5 Ce que fait chaque dossier principal	24
4.5.1 <code>src/config</code>	24
4.5.2 <code>src/common</code>	24
4.5.3 <code>src/middlewares</code>	25
4.5.4 <code>src/infrastructure</code>	25
4.5.5 <code>src/modules</code>	26
4.6 Ordre de reconstruction à respecter	26
5 Partie 3. Étude détaillée de chaque module	27
5.1 Logique générale de construction d'un module	27
5.1.1 Ordre conseillé pour construire un module	27
5.1.2 Ordre conseillé pour lire un module	27

5.2	Module d'authentification	27
5.2.1	Ce que ce module doit faire	27
5.2.2	Dépendances principalement utilisées	28
5.2.3	Étape 1. Valider les entrées du module	28
5.2.4	Étape 2. Déclarer les routes du module	28
5.2.5	Étape 3. Construire le contrôleur	29
5.2.6	Étape 4. Écrire le service	30
5.2.7	Étape 5. Écrire le repository	30
5.2.8	Étape 6. Assembler le module	31
5.2.9	Étape 7. Vérifier le module auth dans Swagger	32
5.2.10	Ce qu'il faut retenir	33
5.3	Module clients	33
5.3.1	Ce que ce module doit faire	33
5.3.2	Dépendances principalement utilisées	33
5.3.3	Étape 1. Définir les données d'entrée	33
5.3.4	Étape 2. Déclarer les routes	34
5.3.5	Étape 3. Construire le contrôleur	35
5.3.6	Étape 4. Écrire le service	35
5.3.7	Étape 5. Écrire le repository	37
5.3.8	Étape 6. Assembler le module puis le raccorder au backend	38
5.3.9	Étape 7. Vérifier le module clients dans Swagger	39
5.4	Module fournisseurs	40
5.4.1	Ce que ce module doit faire	40
5.4.2	Dépendances principalement utilisées	40
5.4.3	Étape 1. Définir les données d'entrée	40
5.4.4	Étape 2. Déclarer les routes	41
5.4.5	Étape 3. Construire le contrôleur	42
5.4.6	Étape 4. Écrire le service	42
5.4.7	Étape 5. Écrire le repository	44
5.4.8	Étape 6. Assembler le module puis le raccorder au backend	44
5.4.9	Étape 7. Vérifier le module fournisseurs dans Swagger	45
5.5	Module médicaments	46
5.5.1	Ce que ce module doit faire	46
5.5.2	Dépendances principalement utilisées	46
5.5.3	Étape 1. Définir les données d'entrée	46
5.5.4	Étape 2. Déclarer les routes	47
5.5.5	Étape 3. Construire le contrôleur	48
5.5.6	Étape 4. Écrire le service	48
5.5.7	Étape 5. Écrire le repository	49
5.5.8	Étape 6. Assembler le module puis le raccorder au backend	50
5.5.9	Étape 7. Vérifier le module médicaments dans Swagger	51
5.6	Module ventes	51
5.6.1	Ce que ce module doit faire	51
5.6.2	Dépendances principalement utilisées	51
5.6.3	Étape 1. Définir les données d'entrée	52
5.6.4	Étape 2. Déclarer les routes	53
5.6.5	Étape 3. Construire le contrôleur	54
5.6.6	Étape 4. Écrire le service	54

5.6.7	Étape 5. Écrire le repository	55
5.6.8	Étape 6. Assembler le module puis le raccorder au backend	56
5.6.9	Étape 7. Vérifier le module ventes dans Swagger	57
5.7	Synthèse des responsabilités métier	58
6	Partie 4. Construction du modèle de données	59
6.1	Ce que nous voulons construire	59
6.2	Dépendances et commandes à connaître	59
6.3	Ordre conseillé pour reconstruire la base	59
6.4	Étape 1. Définir les tables, les relations et les contraintes	60
6.5	Étape 2. Générer le client Prisma et les migrations	61
6.6	Étape 3. Préparer un jeu de données de démonstration	61
6.7	Ce que fait chaque fichier de cette partie	62
6.8	Ce qu'il faut vérifier à la fin	63
7	Partie 5. Sécurité, validation et robustesse	64
7.1	Ce que nous voulons protéger	64
7.2	Dépendances et commandes à connaître	64
7.3	Ordre conseillé pour reconstruire la sécurité	64
7.4	Étape 1. Vérifier les variables d'environnement au démarrage	65
7.5	Étape 2. Contrôler les entrées HTTP	65
7.6	Étape 3. Vérifier les jetons JWT	66
7.7	Étape 4. Gérer les erreurs proprement	68
7.8	Étape 5. Journaliser ce qui se passe	69
7.9	Ce qu'il faut retenir	69
8	Partie 6. Documentation d'API et observabilité	70
8.1	Ce que nous voulons rendre visible	70
8.2	Dépendances et commandes à connaître	70
8.3	Ordre conseillé pour reconstruire cette partie	70
8.4	Étape 1. Décrire l'API dans OpenAPI	70
8.5	Étape 2. Publier Swagger UI	71
8.6	Étape 3. Publier le mémoire en ligne avec /pdf	72
8.7	Étape 4. Vérifier les points d'accès publics	73
8.8	Ce qu'il faut retenir	76
9	Partie 7. Qualité logicielle, tests et intégration continue	77
9.1	Ce que nous voulons sécuriser	77
9.2	Dépendances et commandes à connaître	77
9.3	Ordre conseillé pour construire la qualité logicielle	77
9.4	Étape 1. Écrire les tests autour des règles métier	77
9.5	Étape 2. Exécuter la suite localement	78
9.6	Étape 3. Automatiser la vérification dans GitHub Actions	79
9.7	Étape 4. Comprendre la logique du pipeline	80
9.8	Ce qu'il faut retenir	80
10	Partie 8. Déploiement et exploitation	81
10.1	Ce que nous voulons obtenir	81
10.2	Dépendances et commandes à connaître	81

10.3	Ordre conseillé pour passer du local à la production	81
10.4	Étape 1. Vérifier les scripts et le démarrage local	81
10.5	Étape 2. Préparer l'exécution locale avec Docker	83
10.6	Étape 3. Décrire la production dans <code>render.yaml</code>	83
10.7	Ce que fait chaque fichier de déploiement	84
10.8	Ce qu'il faut vérifier à la fin	84
11	Partie 9. Démonstration technique backend	85
11.1	Comment démontrer le projet dans le bon ordre	85
11.2	Les points d'accès utiles pour la soutenance	85
11.3	Étape 1. Montrer la documentation interactive	85
11.4	Étape 2. Enchaîner avec les médicaments	86
11.5	Étape 3. Terminer par la vente	87
11.6	Ce que prouve réellement cette démonstration	87
12	Limites actuelles et perspectives	88
12.1	Limites identifiées	88
12.2	Perspectives d'évolution	88
13	Conclusion générale	89
14	Annexes techniques	90
14.1	Annexe A. Variables et configuration	90
14.2	Annexe B. Extraits de preuves d'exécution	90



Résumé

Le présent document décrit le backend du projet **GesPharmacie / PHARMA 221**, une API web conçue pour gérer les opérations essentielles d'une pharmacie: authentification, gestion des clients, gestion des fournisseurs, gestion du stock de médicaments et enregistrement des ventes. Le périmètre documentaire retenu est volontairement limité au backend, car c'est la partie effectivement disponible et vérifiable dans le dépôt de travail au moment de la rédaction.

L'analyse du code montre une base technique cohérente reposant sur **Express 4**, **TypeScript**, **Prisma ORM** et **PostgreSQL**. L'architecture est modulaire. Chaque domaine fonctionnel suit une chaîne explicite allant de la route HTTP jusqu'au dépôt de persistance, avec un passage par des contrôleurs, des services et des validateurs. Cette organisation facilite la lisibilité du code, l'isolation des règles métier et la mise en place de tests unitaires ciblés.

Le projet adopte également des pratiques de qualité logicielle déjà significatives: tests unitaires exécutables, génération de documentation OpenAPI, pipeline GitHub Actions, préparation au déploiement Render et exécution locale via Docker Compose. Le mémoire met cependant en évidence certaines limites actuelles, notamment une authentification encore incomplète du point de vue métier, l'absence d'une table d'utilisateurs persistante et une protection JWT qui ne couvre pas encore l'ensemble des routes fonctionnelles.

Ce mémoire poursuit donc un double objectif. D'une part, il explicite de manière méthodique ce qui est déjà mis en œuvre dans le backend. D'autre part, il fournit une lecture critique de la solution afin d'identifier les consolidations nécessaires pour un passage vers un niveau de maturité plus élevé.

1. Introduction générale

Le besoin de numérisation des activités pharmaceutiques répond à plusieurs contraintes concrètes: fiabilité des données de stock, rapidité de traitement des opérations de vente, traçabilité des mouvements et sécurisation des informations métier. Dans une pharmacie, l'erreur n'a pas seulement un coût organisationnel; elle peut également produire des conséquences directes sur la qualité du service rendu au patient ou au client.

Dans ce contexte, le backend GesPharmacie a pour vocation de fournir une base d'API claire, testable et extensible. L'objectif n'est pas uniquement de stocker des informations. Il s'agit surtout de centraliser des règles métier essentielles, comme l'interdiction de vendre un médicament expiré, l'impossibilité de supprimer une entité encore référencée par des opérations, ou encore le recalcul systématique du montant total d'une vente.

L'approche adoptée dans ce mémoire est démonstrative. Chaque affirmation repose sur un élément vérifiable du dépôt: structure des dossiers, configuration d'environnement, schéma Prisma, code des services, tests unitaires et pipeline d'intégration continue. La rédaction cherche donc moins à produire un discours théorique qu'à exposer une lecture rigoureuse de la solution effectivement implémentée.

1.1. Objet du mémoire

Ce mémoire porte exclusivement sur le backend du projet. Il couvre:

- la logique fonctionnelle observable dans le code;
- l'architecture logicielle retenue;
- la modélisation des données;
- les mécanismes de validation, de sécurité et de robustesse;
- la documentation technique disponible;
- le déploiement et la qualité logicielle.

1.2. Référentiel d'analyse

L'analyse a été menée directement sur le dépôt local du projet. La base de travail observable au moment de la rédaction correspond au commit `d188487`, positionné sur la branche `feature/prisma-seed-data`.

Les éléments suivants ont été vérifiés localement pendant la rédaction:

- compilation TypeScript du backend;
- exécution de la suite de tests unitaires;
- lecture des fichiers de configuration, du schéma Prisma et du workflow CI;

- inspection des modules métier et des middlewares.

2. Objectif du projet

2.1. Ce que le projet doit résoudre

Le projet **GesPharmacie / PHARMA 221** a pour objectif de construire le backend d’une application de gestion de pharmacie. En langage simple, cela signifie que l’application doit être capable d’enregistrer les clients, les fournisseurs, les médicaments et les ventes, tout en empêchant les incohérences métier les plus dangereuses.

Le problème réel n’est donc pas seulement “stocker des données”. Le vrai besoin consiste à faire respecter des règles. Par exemple:

- un médicament doit être rattaché à un fournisseur valide;
- une vente ne peut pas être enregistrée si le client n’existe pas;
- un médicament expiré ne doit pas être vendu;
- un stock insuffisant doit bloquer l’opération;
- certaines suppressions doivent être refusées pour ne pas casser l’historique.

2.2. Ce que le backend doit produire

Le backend doit fournir un résultat concret, lisible et réutilisable:

- une API REST disponible sous `/api/v1`;
- un point de santé pour vérifier si le service fonctionne;
- un module d’authentification minimale par JWT;
- un module `clients`;
- un module `fournisseurs`;
- un module `medicaments`;
- un module `ventes`;
- une documentation Swagger/OpenAPI;
- une base PostgreSQL pilotée par Prisma;
- une structure de code assez claire pour qu’un autre développeur puisse refaire l’application.

2.3. Pourquoi le mémoire se concentre sur le backend

Le dépôt observé contient surtout la partie backend. C’est donc cette partie qui sert de base à l’analyse et à la reconstruction. Le mémoire ne cherche pas à raconter un projet imaginaire. Il montre comment le backend a été construit, pourquoi cette architecture a été choisie, et comment un lecteur peut refaire chaque module en suivant les étapes dans le bon ordre.

3. Partie 1. Initialisation du projet

3.1. Préparer l'environnement avant d'écrire le premier module

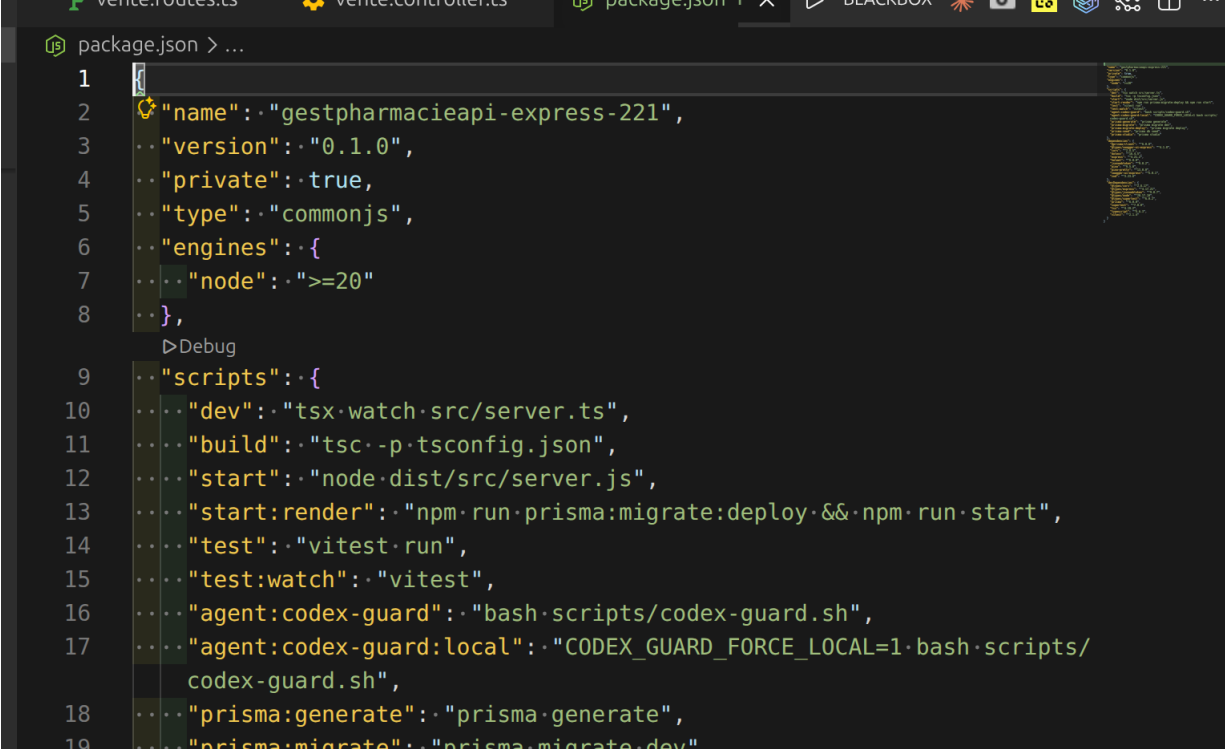
Avant de parler des modules, il faut d'abord préparer le terrain. Si cette étape est mal faite, tout le reste devient confus. Pour refaire proprement ce backend, il faut installer:

1. **Node.js** en version ≥ 20 , car c'est le runtime attendu par le projet.
2. **npm**, pour installer les dépendances et lancer les scripts.
3. **PostgreSQL**, car les données sont stockées dans une base relationnelle.
4. **VS Code** ou un éditeur équivalent, pour garder une lecture claire du projet.

Quand on repart de zéro, la logique d'installation la plus simple est la suivante:

1. créer le projet avec `npm init -y`;
2. installer les dépendances d'exécution avec `npm install express cors helmet dotenv zod jsonwebtoken pino pino-pretty swagger-ui-express @prisma/client`;
3. installer les dépendances de développement avec `npm install -D typescript tsx prisma vitest @types/node @types/express @types/jsonwebtoken @types/swagger-ui-express @types/cors @types/supertest`;
4. initialiser Prisma, créer le schéma et préparer les migrations;
5. seulement après cela, commencer l'écriture du code applicatif.

La capture suivante montre la base réelle de configuration du projet dans `package.json`. On y voit à la fois le runtime, les scripts d'exécution et les dépendances qui rendent possible toute l'application.



```
1  {
2    "name": "gestpharmacieapi-express-221",
3    "version": "0.1.0",
4    "private": true,
5    "type": "commonjs",
6    "engines": {
7      "node": ">=20"
8    },
9    "scripts": {
10     "dev": "tsx watch src/server.ts",
11     "build": "tsc -p tsconfig.json",
12     "start": "node dist/src/server.js",
13     "start:render": "npm run prisma:migrate:deploy && npm run start",
14     "test": "vitest run",
15     "test:watch": "vitest",
16     "agent:codex-guard": "bash scripts/codex-guard.sh",
17     "agent:codex-guard:local": "CODEX_GUARD_FORCE_LOCAL=1 bash scripts/codex-guard.sh",
18     "prisma:generate": "prisma generate",
19     "prisma:migrate": "prisma migrate dev",
```

Figure 3.1: Capture réelle du fichier `package.json` au démarrage du projet

La lecture de ce fichier permet déjà de comprendre plusieurs décisions:

- `express`, `cors` et `helmet` servent à construire l'API HTTP;
- `zod` sert à valider les entrées;
- `jsonwebtoken` sert au module d'authentification;
- `@prisma/client` et `prisma` servent à la couche base de données;
- `tsx` sert au développement local;
- `typescript` sert à typer le projet;
- `vitest` sert à vérifier le comportement.

3.2. Organiser le travail dès le début

Un bon backend ne se construit pas en ajoutant des fichiers au hasard. Le dépôt montre une progression par sprints. Cette organisation a un intérêt très pédagogique, car elle permet de construire une chose à la fois.

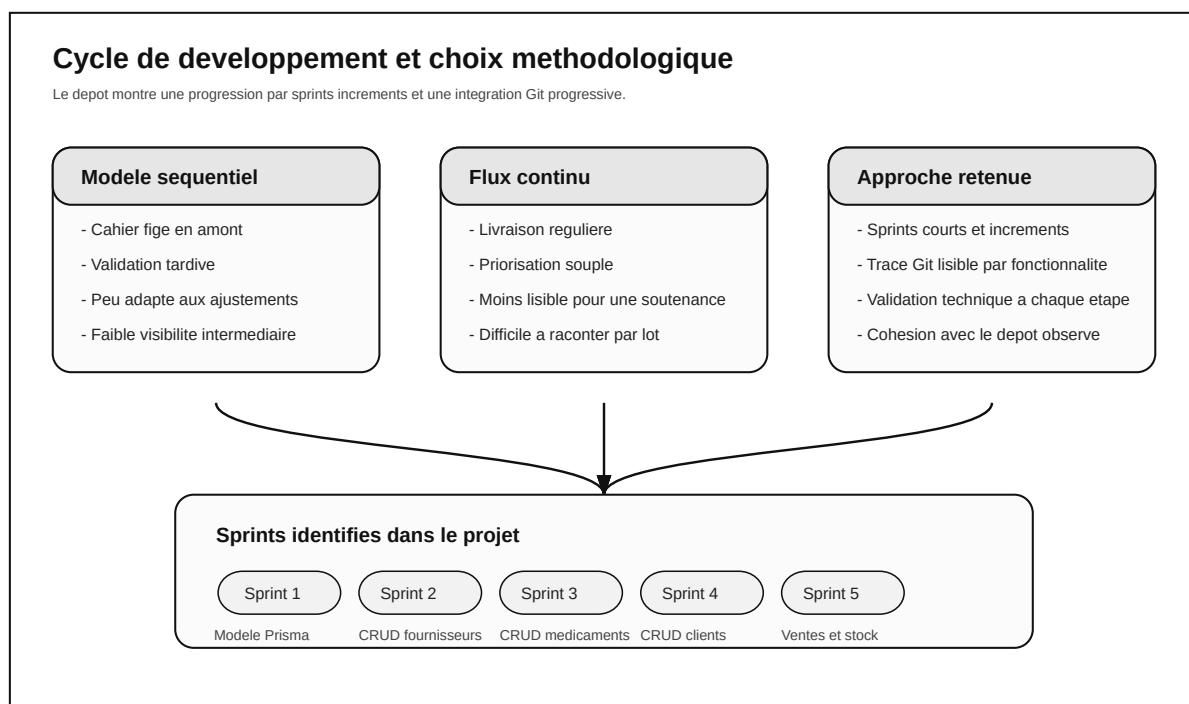


Figure 3.2: *Cycle de developpement et choix methodologique*

La progression observee est la suivante:

- **Sprint 1:** modelisation du domaine avec Prisma;
- **Sprint 2:** fournisseurs;
- **Sprint 3:** medicaments;
- **Sprint 4:** clients;
- **Sprint 5:** ventes.

En parallele, la strategie Git separe les branches de reference et les branches de travail. Cette discipline est utile, car elle evite de melanger l'experimentation et la version stable.

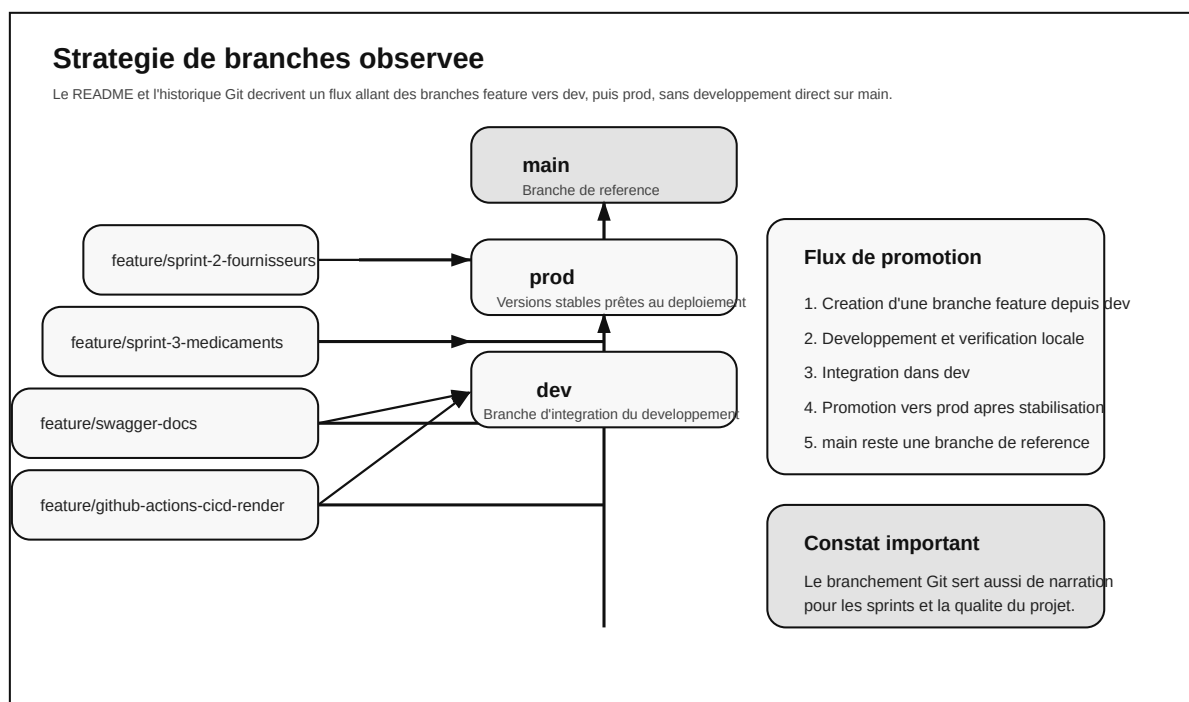


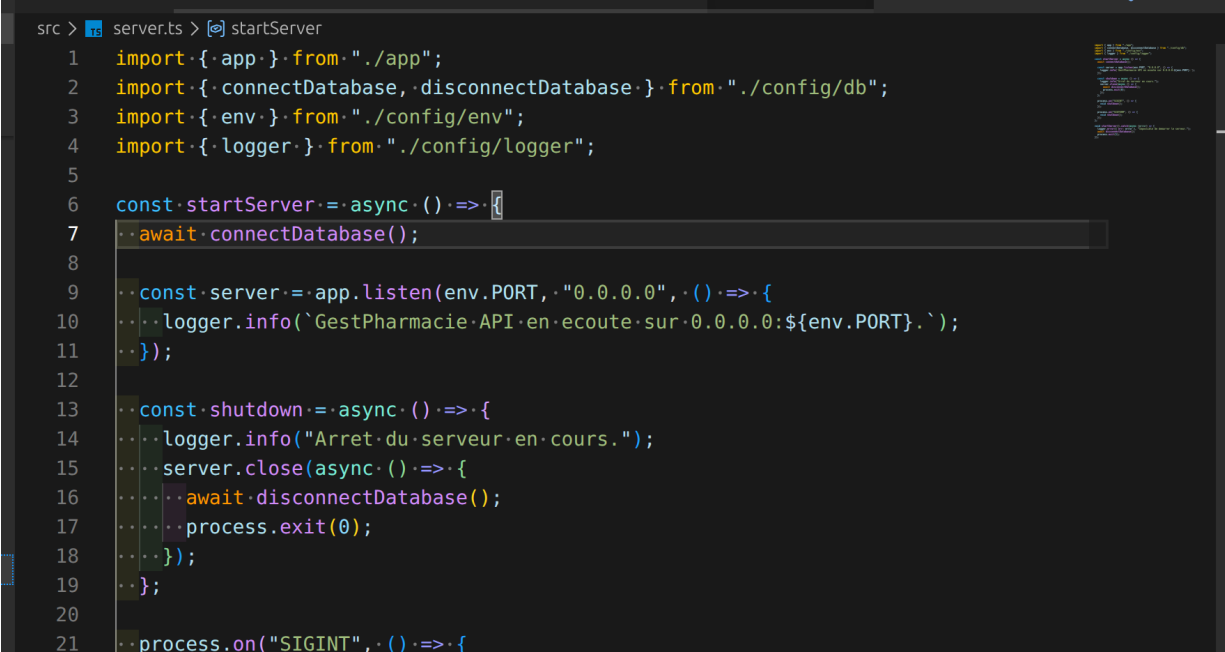
Figure 3.3: Stratégie Git observée

3.3. Créer le socle HTTP du projet

Une fois les dépendances installées, la première vraie étape de code consiste à monter le socle HTTP. Il faut un fichier qui construit l'application et un fichier qui lance réellement le serveur.

```
src > ts app.ts > ...
1  import express from "express";
2  import cors from "cors";
3  import helmet from "helmet";
4
5  import { buildDocsRouter } from "../docs";
6  import { errorHandler, notFoundHandler } from "../middlewares/error.middleware";
7  import { apiRouter } from "../routes";
8
9  export const app = express();
10
11  app.use(helmet());
12  app.use(cors());
13  app.use(express.json());
14
15  app.use("/docs", buildDocsRouter());
16  app.use("/api/v1", apiRouter);
17
18  app.use(notFoundHandler);
19  app.use(errorHandler);
20
```

Figure 3.4: Capture réelle de la composition de l'application Express dans `src/app.ts`



```
src > ts server.ts > startServer
1 import { app } from './app';
2 import { connectDatabase, disconnectDatabase } from './config/db';
3 import { env } from './config/env';
4 import { logger } from './config/logger';
5
6 const startServer = async () => {
7   await connectDatabase();
8
9   const server = app.listen(env.PORT, '0.0.0.0', () => {
10     logger.info(`GestPharmacie API en écoute sur 0.0.0.0:${env.PORT}`);
11   });
12
13   const shutdown = async () => {
14     logger.info("Arret du serveur en cours.");
15     server.close(async () => {
16       await disconnectDatabase();
17       process.exit(0);
18     });
19   };
20
21   process.on("SIGINT", () => {
```

Figure 3.5: Capture réelle du démarrage serveur dans `src/server.ts`

Le rôle des deux fichiers est très simple:

- `app.ts` assemble les middlewares, la documentation Swagger, la route `/pdf`, le routeur principal et la gestion des erreurs;
- `server.ts` se charge de démarrer le service, d'ouvrir le port et de connecter la base.

Ce découpage est important. Il permet de comprendre qu'un serveur backend n'est pas "un seul gros fichier". On sépare la définition de l'application de son lancement réel.

3.4. Mettre en place les fondations techniques avant les modules

Avant de coder `auth`, `clients` ou `ventes`, il faut mettre en place plusieurs éléments partagés.

3.4.1. Validation de l'environnement

Le projet valide les variables critiques dès le démarrage.

```
src > config > env.ts > ...
1  import "dotenv/config";
2
3  import { z } from "zod";
4
5  const envSchema = z.object({
6    .. NODE_ENV: z.enum(["development", "test", "production"]).default("development"),
7    .. PORT: z.coerce.number().int().positive().default(4000),
8    .. DATABASE_URL: z.string().min(1, "DATABASE_URL est obligatoire."),
9    .. JWT_SECRET: z.string().min(8, "JWT_SECRET doit contenir au moins 8 caracteres."),
10   .. LOG_LEVEL: z
11     .. .enum(["fatal", "error", "warn", "info", "debug", "trace", "silent"])
12     .. .default("info"),
13   });
14
15  export const env = envSchema.parse(process.env);
16
17
```

Figure 3.6: Capture réelle de la validation d'environnement dans `src/config/env.ts`

Cette étape évite qu'un backend démarre avec une base absente, un secret JWT manquant ou un port invalide.

3.4.2. Journalisation

Le logger est installé très tôt pour rendre les messages d'exécution lisibles.

```
src > config > logger.ts > ...
1  import pino from "pino";
2
3  import { env } from "../env";
4
5  export const logger = pino({
6    .. level: env.LOG_LEVEL,
7    .. transport:
8      .. env.NODE_ENV === "development"
9      .. ? {
10         .. target: "pino-pretty",
11         .. options: {
12           .. colorize: true,
13         },
14       }
15      .. : undefined,
16   });
17
18
```

Figure 3.7: Capture réelle de la configuration du logger dans `src/config/logger.ts`

3.4.3. Accès à la base de données

Prisma et la connexion base sont préparés avant les modules métier.

```
src > infrastructure > database > client.ts > ...
1  import { PrismaClient } from "@prisma/client";
2
3  import { env } from "../../config/env";
4
5  const globalForPrisma = globalThis as unknown as {
6    prisma?: PrismaClient;
7  };
8
9  export const prismaClient =
10    globalForPrisma.prisma ??
11    new PrismaClient({
12      log: env.NODE_ENV === "development" ? ["query", "warn", "error"] : ["error"],
13    });
14
15  if (env.NODE_ENV !== "production") {
16    globalForPrisma.prisma = prismaClient;
17  }
18
```

Figure 3.8: Capture réelle du client Prisma dans `src/infrastructure/database/client.ts`

```
src > infrastructure > database > prisma.ts > ...
1  import { prismaClient } from "./client";
2
3  export const db = prismaClient;
4
5  export const connectDatabase = async () => {
6    await db.$connect();
7  };
8
9  export const disconnectDatabase = async () => {
10    await db.$disconnect();
11  };
12
```

Figure 3.9: Capture réelle de la connexion base de données dans `src/infrastructure/database/prisma.ts`

3.4.4. Outils transverses

Le backend prépare aussi dès le début les éléments qui seront réutilisés partout:

```

src > middlewares > validate.middleware.ts > ...
1  import { NextFunction, Request, Response } from "express";
2  import { ZodTypeAny } from "zod";
3
4  import { AppError } from "../common/errors/app-error";
5  import { APP_MESSAGES } from "../common/messages";
6  Q
7  export const validate = (schema: ZodTypeAny) => {
8    return (req: Request, _res: Response, next: NextFunction) => {
9      const parsed = schema.safeParse({
10        body: req.body,
11        params: req.params,
12        query: req.query,
13      });
14
15      if (!parsed.success) {
16        next(new AppError(422, APP_MESSAGES.VALIDATION_FAILED, parsed.error.flatten()));
17      }
18      return;
19    }
20    if (parsed.data.body) {

```

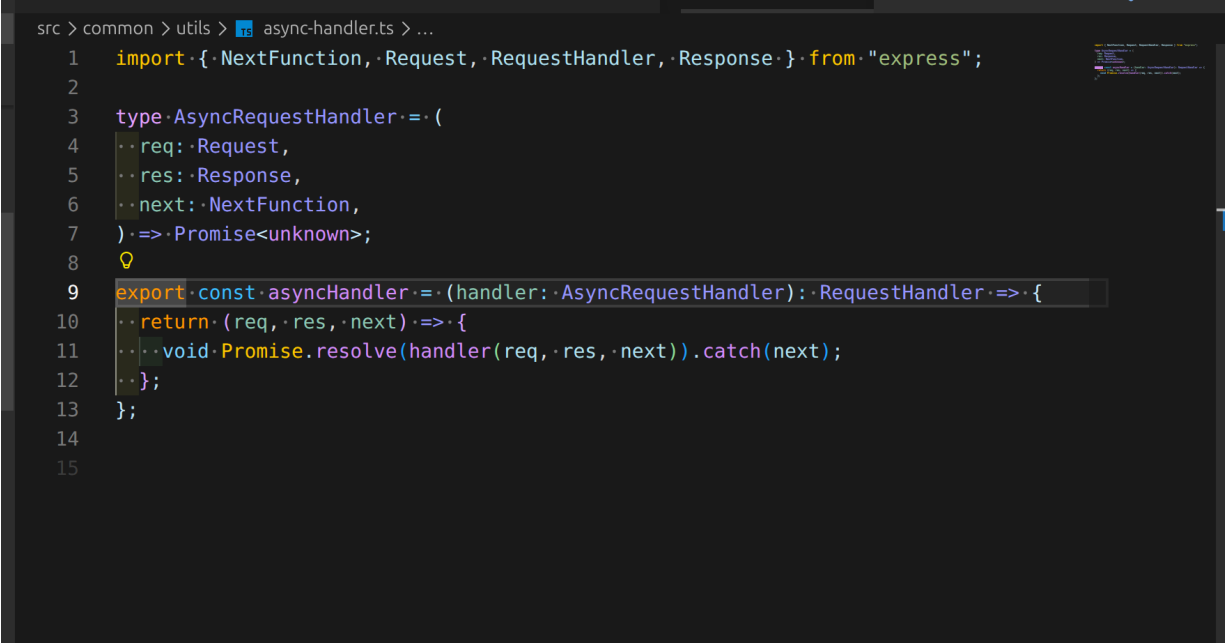
Figure 3.10: Capture réelle du middleware de validation dans `src/middlewares/validate.middleware.ts`

```

src > middlewares > error.middleware.ts
3
4  import { AppError } from "../common/errors/app-error";
5  import { APP_MESSAGES } from "../common/messages";
6  import { env } from "../config/env";
7  import { logger } from "../config/logger";
8
9  export const notFoundHandler: RequestHandler = (req, _res, next) => {
10    next(
11      new AppError(404, APP_MESSAGES.NOT_FOUND, {
12        path: req.originalUrl,
13      }),
14    );
15  };
16
17  export const errorHandler: ErrorRequestHandler = (error, req, res, _next) => {
18    let appError: AppError;
19
20    if (error instanceof AppError) {
21      appError = error;
22    } else if (error instanceof Prisma.PrismaClientKnownRequestError) {
23      if (error.code === "P2002") {

```

Figure 3.11: Capture réelle du middleware d'erreur dans `src/middlewares/error.middleware.ts`



```

src > common > utils > async-handler.ts > ...
1  import { NextFunction, Request, RequestHandler, Response } from "express";
2
3  type AsyncRequestHandler = (
4    req: Request,
5    res: Response,
6    next: NextFunction,
7  ) => Promise<unknown>;
8
9  export const asyncHandler = (handler: AsyncRequestHandler): RequestHandler => {
10    return (req, res, next) => {
11      void Promise.resolve(handler(req, res, next)).catch(next);
12    };
13  };
14
15

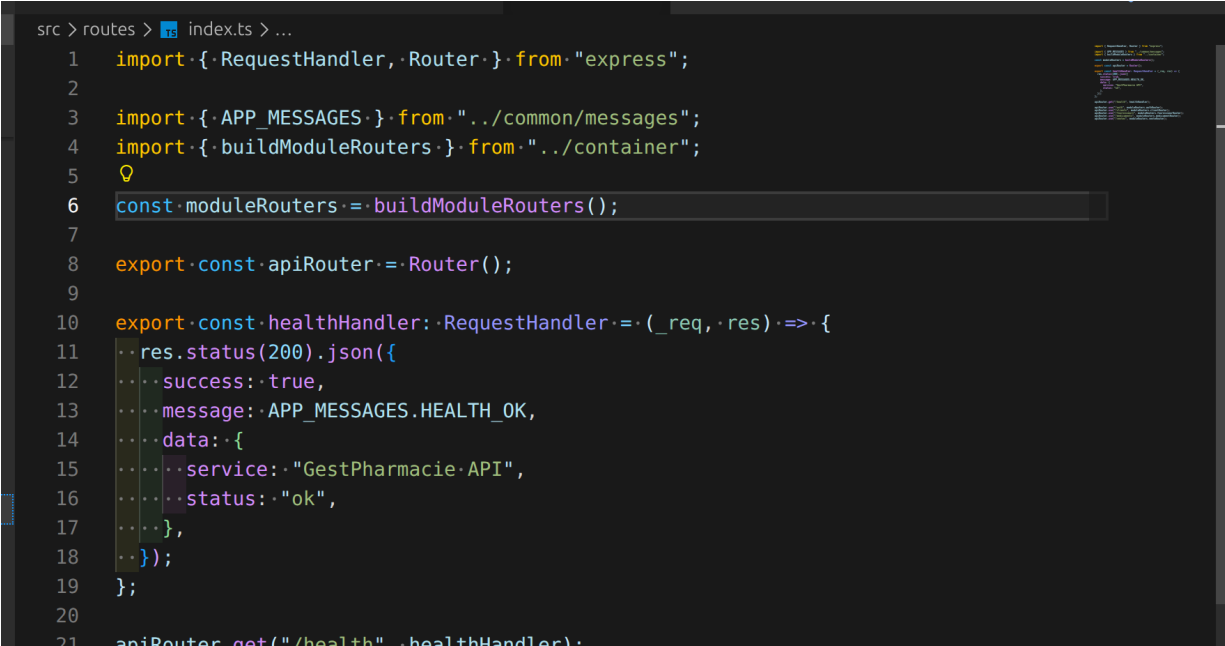
```

Figure 3.12: Capture réelle de l'utilitaire `asyncHandler`

L'idée est simple: ces briques ne sont pas des modules métier, mais elles rendent possible une implémentation propre des modules métier.

3.5. Préparer les portes d'entrée avant les fonctionnalités

Avant de créer un vrai module, il faut préparer l'entrée du backend: le routage principal et la documentation.



```

src > routes > index.ts > ...
1  import { RequestHandler, Router } from "express";
2
3  import { APP_MESSAGES } from "../common/messages";
4  import { buildModuleRouters } from "../container";
5
6  const moduleRouters = buildModuleRouters();
7
8  export const apiRouter = Router();
9
10 export const healthHandler: RequestHandler = (_req, res) => {
11   res.status(200).json({
12     success: true,
13     message: APP_MESSAGES.HEALTH_OK,
14     data: {
15       service: "GestPharmacie-API",
16       status: "ok",
17     },
18   });
19 };
20
21 apiRouter.get("/health", healthHandler);

```

Figure 3.13: Capture réelle du routage principal dans `src/routes/index.ts`

```
src > container > index.ts > ...
1  import { buildAuthModule } from './auth.container';
2  import { buildClientModule } from './client.container';
3  import { buildFournisseurModule } from './fournisseur.container';
4  import { buildMedicamentModule } from './medicament.container';
5  import { buildVenteModule } from './vente.container';
6  Q
7  export const buildModuleRouters = () => ({
8    ..authRouter: buildAuthModule(),
9    ..clientRouter: buildClientModule(),
10   ..fournisseurRouter: buildFournisseurModule(),
11   ..medicamentRouter: buildMedicamentModule(),
12   ..venteRouter: buildVenteModule(),
13 });
14
```

Figure 3.14: Capture réelle de la fabrique des routeurs de modules dans `src/container/index.ts`

```
src > docs > swagger.ts > ...
1  import { RequestHandler, Router } from "express";
2  import swaggerUi from "swagger-ui-express";
3
4  import { openApiDocument } from './openapi';
5
6  export const relaxSwaggerHeaders: RequestHandler = (_req, res, next) => {
7    ..res.removeHeader("Content-Security-Policy");
8    ..res.removeHeader("Cross-Origin-Opener-Policy");
9    ..res.removeHeader("Cross-Origin-Resource-Policy");
10   ..res.removeHeader("Origin-Agent-Cluster");
11   ..next();
12 };
13 Q
14 export const openApiHandler: RequestHandler = (_req, res) => {
15   ..res.status(200).json(openApiDocument);
16 };
17
18 export const docsUiOptions = {
19   ..explorer: true,
20   ..customSiteTitle: "PHARMA 221 API Docs",
21   ..swaggerOptions: {
```

Figure 3.15: Capture réelle de la documentation Swagger dans `src/docs/swagger.ts`

Cette étape sert à répondre à trois questions:

- où les requêtes arrivent-elles?
- comment les modules sont-ils branchés?
- comment l'API sera-t-elle testée et lue ensuite?

Tant que ce socle n'existe pas, il est inutile de détailler un module métier. Il faut donc toujours initialiser le projet avant d'entrer dans `auth`, `clients`, `fournisseurs`, `medicaments` ou `ventes`.

4. Partie 2. Étude détaillée de l'architecture choisie

4.1. Architecture retenue

Le backend de GesPharmacie adopte une architecture **monolithique modulaire en couches**. En langage simple, cela veut dire:

- un seul backend est déployé;
- ce backend est divisé en modules métier;
- chaque module suit les mêmes couches internes;
- la base relationnelle est centralisée dans PostgreSQL via Prisma.

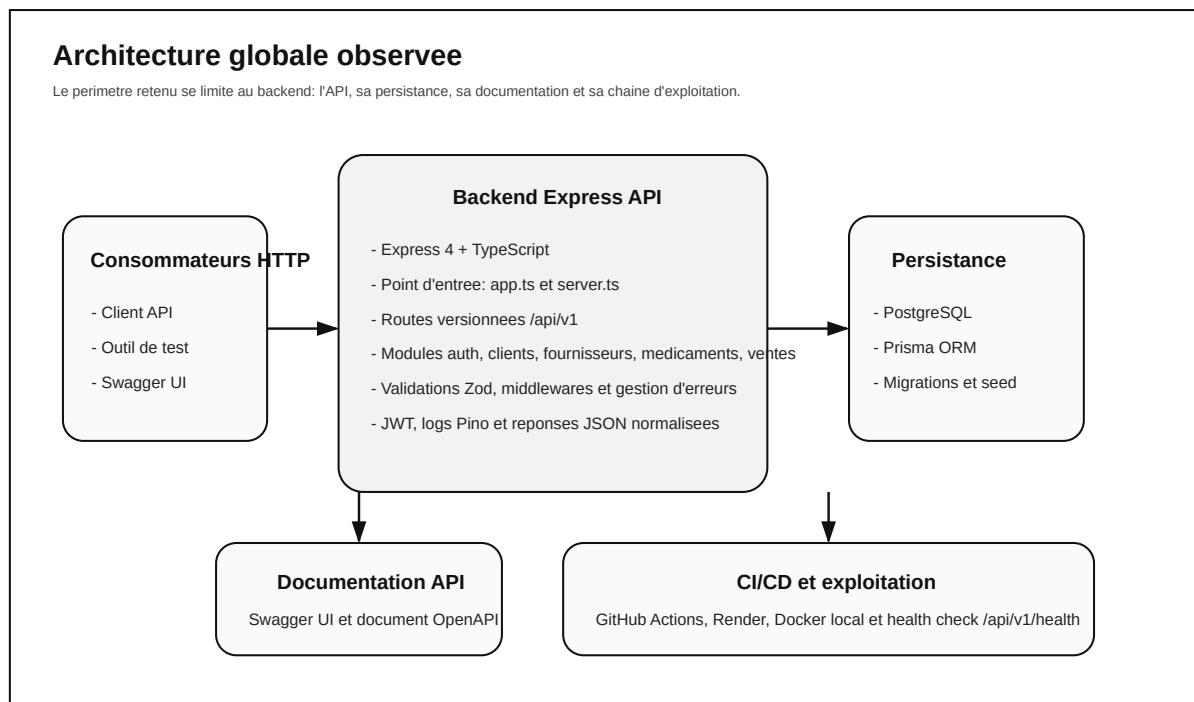


Figure 4.1: *Architecture globale du backend*

Ce choix est pertinent pour ce projet pour plusieurs raisons:

- le périmètre fonctionnel est suffisamment concentré pour ne pas imposer des microservices;
- le code doit rester lisible pour une soutenance;
- les règles métier doivent être séparées des détails HTTP;

- le déploiement doit rester simple sur Render.

4.2. Pourquoi cette architecture est adaptée

L'architecture retenue répond à deux besoins en même temps:

1. **faire fonctionner l'application;**
2. **permettre à quelqu'un d'autre de la refaire.**

Si tout le code était mélangé dans les routes, le projet pourrait peut-être fonctionner, mais il serait très difficile à expliquer et à maintenir. Ici, chaque couche porte une responsabilité claire:

Couche	Ce qu'elle fait
Route	reçoit l'URL et branche les middlewares
Controller	construit la réponse HTTP
Service	décide de la règle métier
Repository	parle à Prisma et à la base
Base de données	stocke les données et applique les relations

4.3. Capture directe de l'architecture source

L'arborescence réelle du dossier `src` confirme que cette architecture n'est pas seulement théorique.

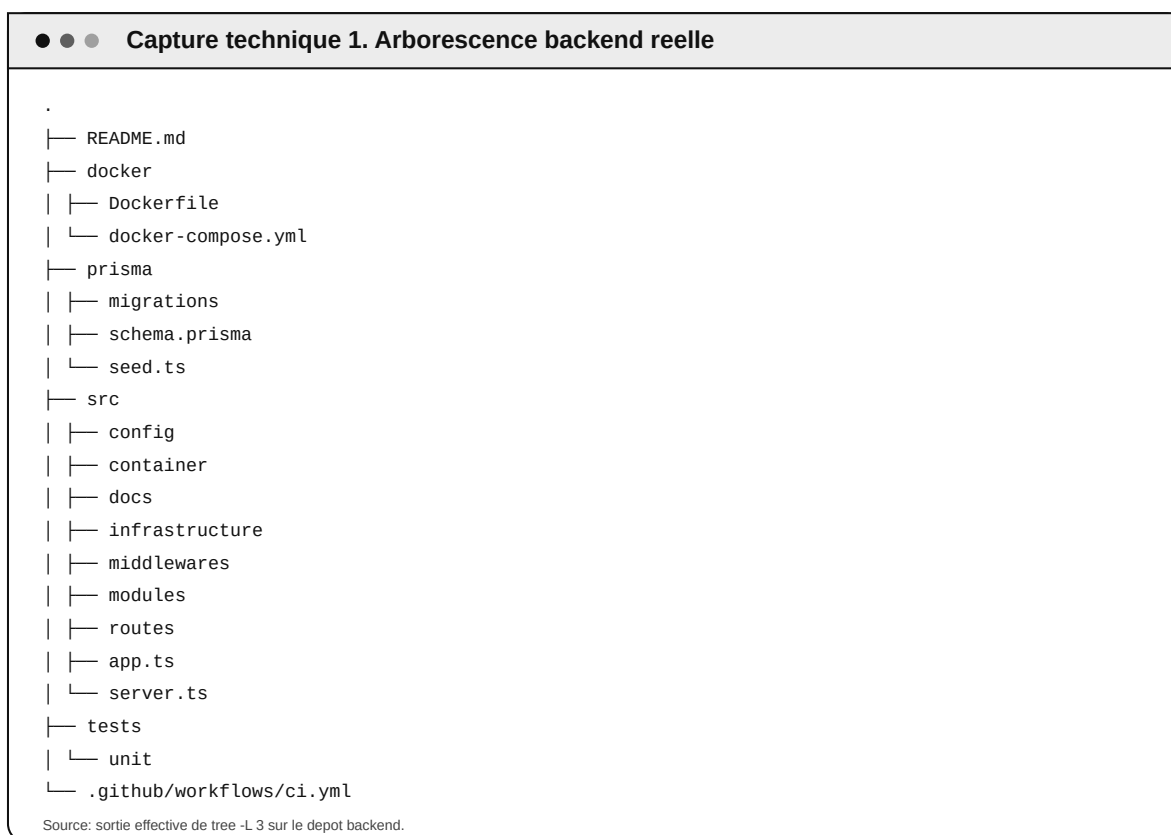


Figure 4.2: Capture technique de l'arborescence backend

L'ordre recommandé de lecture est le suivant:

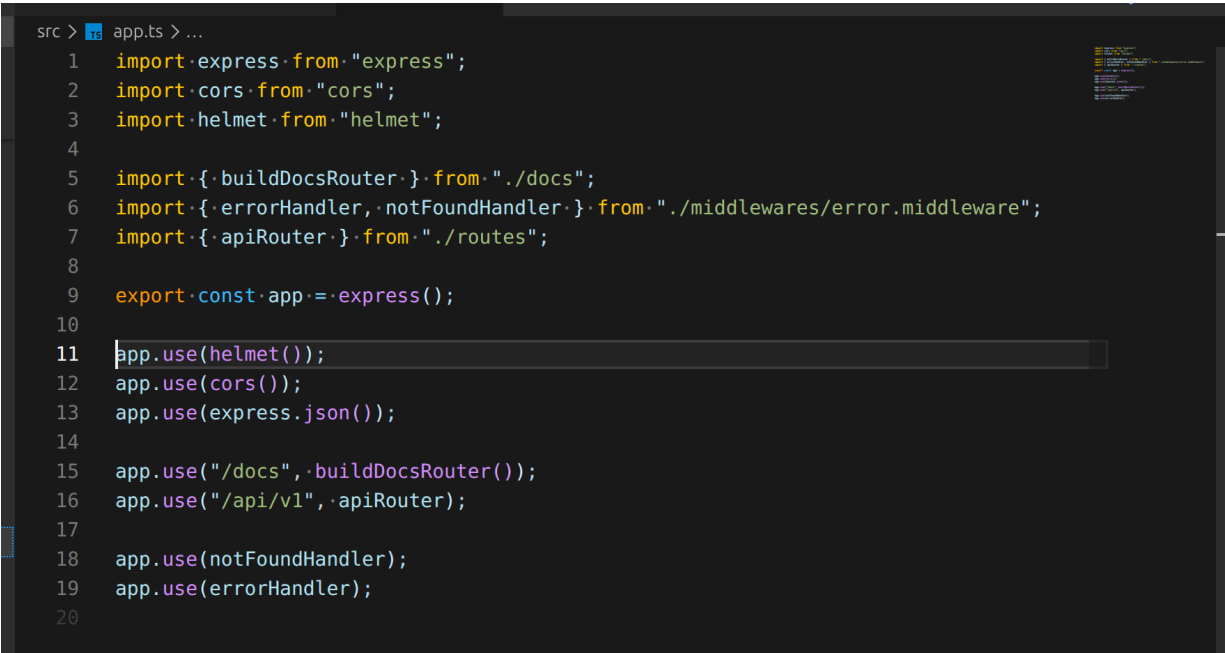
1. `src/config`
2. `src/common`
3. `src/middlewares`
4. `src/routes`
5. `src/container`
6. `src/modules`

Ce chemin de lecture permet d'abord de comprendre le cadre général, puis seulement ensuite les détails des modules.

4.4. Comment une requête traverse l'application

Le flux complet peut être résumé ainsi: **Requête HTTP -> route -> controller -> service -> repository -> Prisma -> PostgreSQL.**

Ce parcours est visible directement dans le code réel:



```
src > app.ts > ...
1  import express from "express";
2  import cors from "cors";
3  import helmet from "helmet";
4
5  import { buildDocsRouter } from "../docs";
6  import { errorHandler, notFoundHandler } from "../middlewares/error.middleware";
7  import { apiRouter } from "../routes";
8
9  export const app = express();
10
11 app.use(helmet());
12 app.use(cors());
13 app.use(express.json());
14
15 app.use("/docs", buildDocsRouter());
16 app.use("/api/v1", apiRouter);
17
18 app.use(notFoundHandler);
19 app.use(errorHandler);
20
```

Figure 4.3: Capture réelle du branchement principal dans `src/app.ts`

```
src > routes > index.ts > ...
1  import { RequestHandler, Router } from "express";
2
3  import { APP_MESSAGES } from "../common/messages";
4  import { buildModuleRouters } from "../container";
5  Q
6  const moduleRouters = buildModuleRouters();
7
8  export const apiRouter = Router();
9
10 export const healthHandler: RequestHandler = (_req, res) => {
11   res.status(200).json({
12     success: true,
13     message: APP_MESSAGES.HEALTH_OK,
14     data: {
15       service: "GestPharmacie-API",
16       status: "ok",
17     },
18   });
19 };
20
21 apiRouter.get("/health", healthHandler);
```

Figure 4.4: Capture réelle du routage principal dans *src/routes/index.ts*

```
src > container > client.container.ts > ...
1  import { ClientController } from "../modules/client/client.controller";
2  import { ClientRepository } from "../modules/client/client.repository";
3  import { buildClientRouter } from "../modules/client/client.routes";
4  import { ClientService } from "../modules/client/client.service";
5  Q
6  export const buildClientModule = () => {
7    const repository = new ClientRepository();
8    const service = new ClientService(repository);
9    const controller = new ClientController(service);
10
11    return buildClientRouter(controller);
12  };
13
```

Figure 4.5: Capture réelle de l'assemblage du module client dans *src/container/client.container.ts*

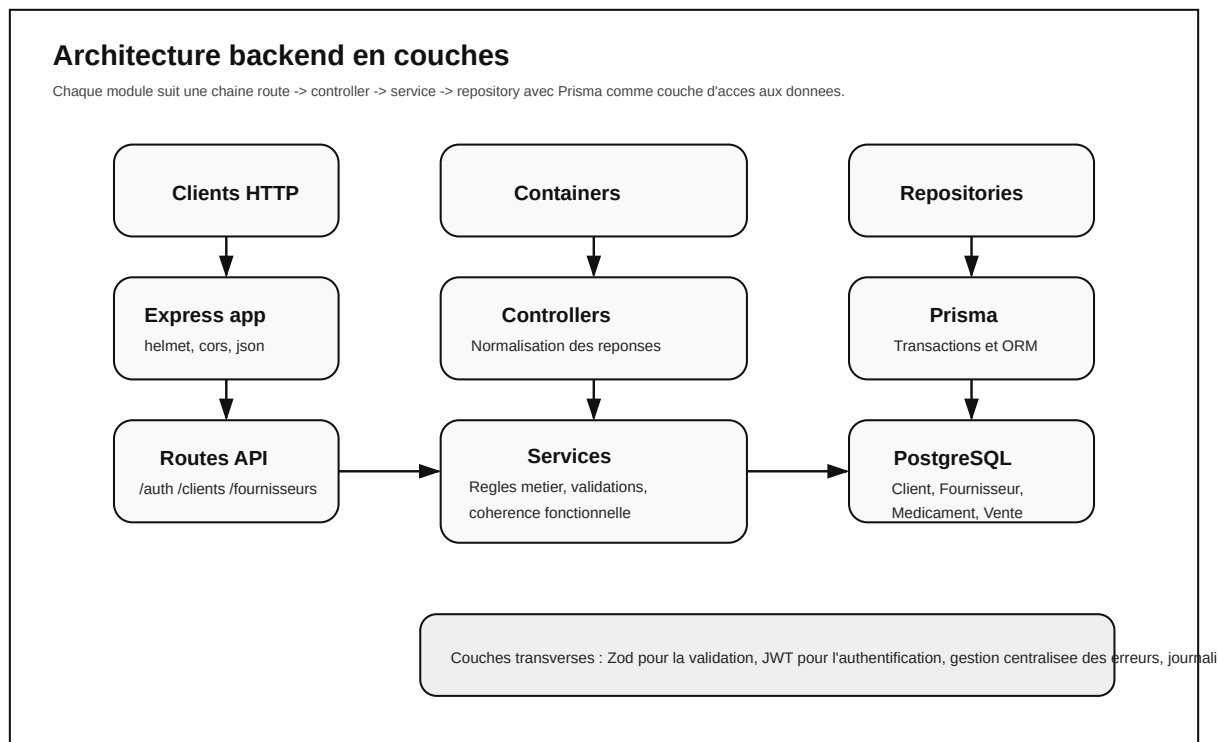


Figure 4.6: *Architecture backend en couches*

Pour un lecteur débutant, l'idée centrale est la suivante:

- la route ne décide pas de la logique métier;
- le service ne construit pas la réponse HTTP;
- le repository n'invente pas la règle fonctionnelle;
- chaque couche a son rôle, et c'est ce qui rend le projet compréhensible.

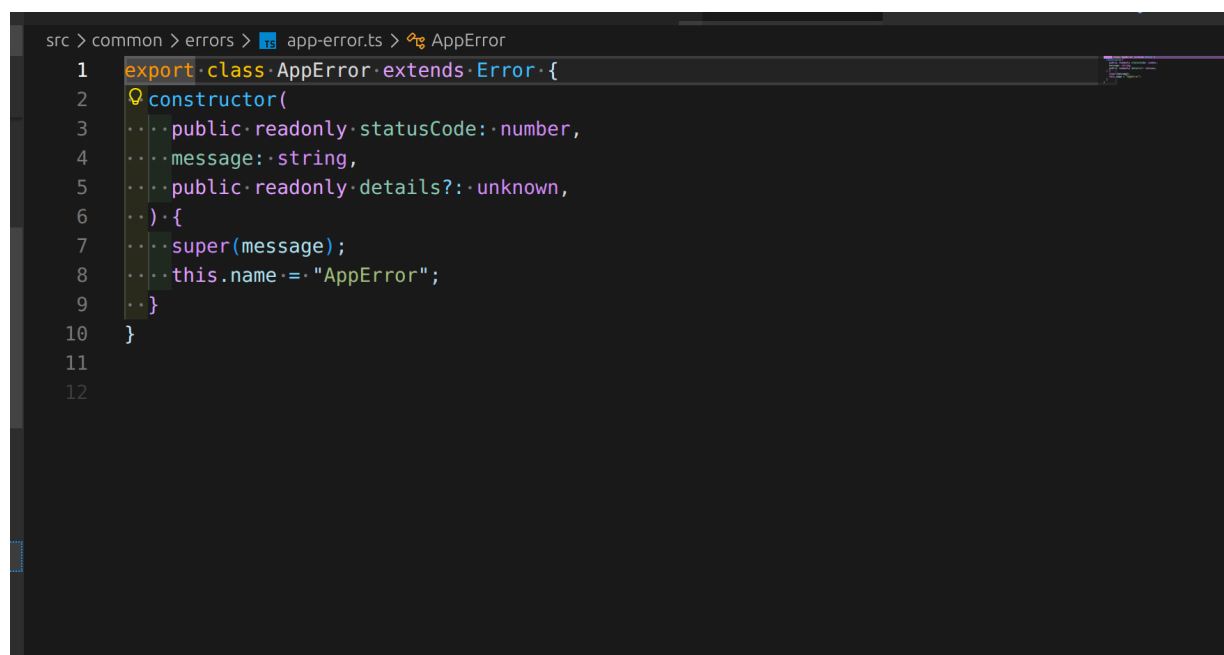
4.5. Ce que fait chaque dossier principal

4.5.1. `src/config`

Ce dossier centralise l'environnement, la base de données et la journalisation. Il sert à sécuriser le démarrage.

4.5.2. `src/common`

Ce dossier porte les interfaces, messages, erreurs et utilitaires réutilisés dans plusieurs modules.

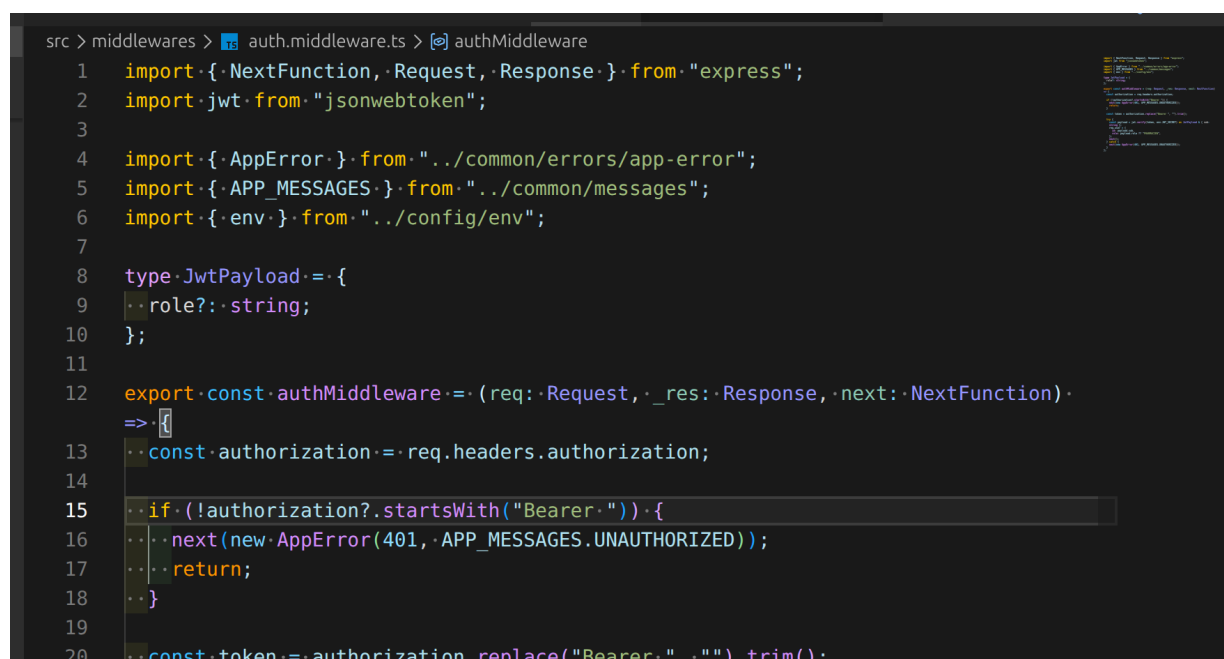


```
src > common > errors > app-error.ts > AppError
1 export class AppError extends Error {
2   constructor(
3     public readonly statusCode: number,
4     message: string,
5     public readonly details?: unknown,
6   ) {
7     super(message);
8     this.name = "AppError";
9   }
10 }
11
12
```

Figure 4.7: Capture réelle de la classe *AppError*

4.5.3. src/middlewares

Ce dossier contient la validation, la sécurité et le traitement centralisé des erreurs.

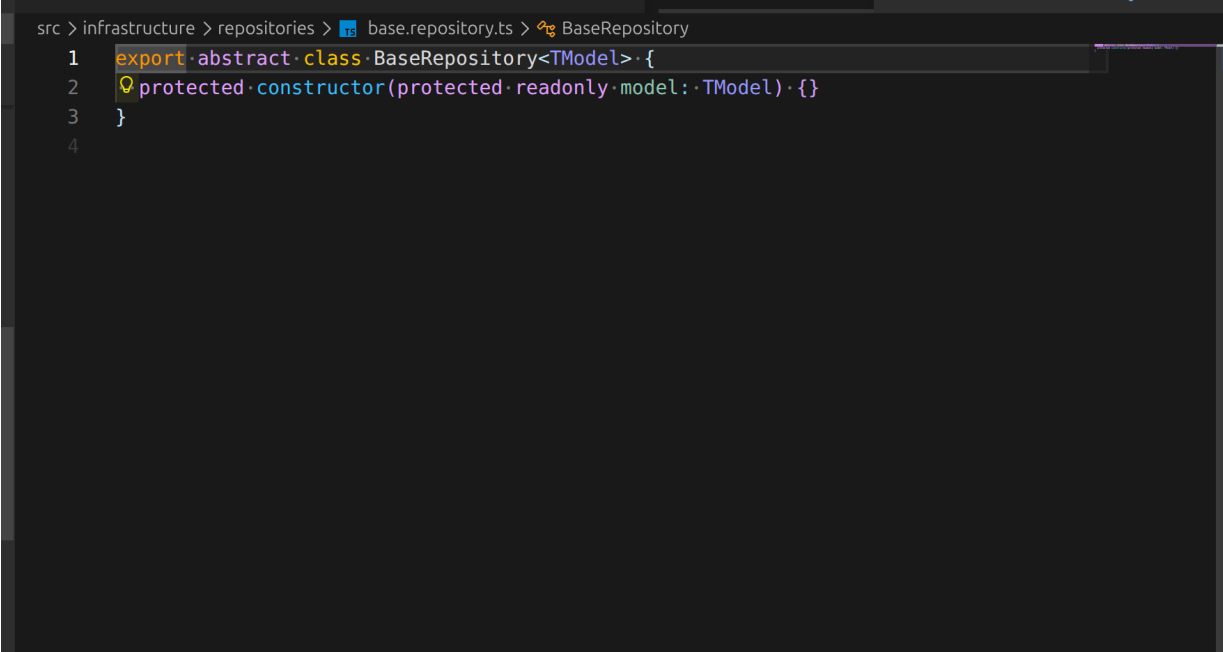


```
src > middlewares > auth.middleware.ts > authMiddleware
1 import { NextFunction, Request, Response } from "express";
2 import jwt from "jsonwebtoken";
3
4 import { AppError } from "../common/errors/app-error";
5 import { APP_MESSAGES } from "../common/messages";
6 import { env } from "../config/env";
7
8 type JwtPayload = {
9   role?: string;
10 };
11
12 export const authMiddleware = (req: Request, _res: Response, next: NextFunction) => {
13   const authorization = req.headers.authorization;
14
15   if (!authorization?.startsWith("Bearer ")) {
16     next(new AppError(401, APP_MESSAGES.UNAUTHORIZED));
17     return;
18   }
19
20   const token = authorization.replace("Bearer ", "").trim();
21 }
```

Figure 4.8: Capture réelle du middleware d'authentification dans *src/middlewares/auth.middleware.ts*

4.5.4. src/infrastructure

Ce dossier contient les composants techniques bas niveau, notamment le client Prisma et la classe de base des repositories.



```
src > infrastructure > repositories > base.repository.ts > BaseRepository
1 export abstract class BaseRepository<TModel> {
2   protected constructor(protected readonly model: TModel) {}
3 }
4
```

Figure 4.9: Capture réelle de la classe de base des repositories

4.5.5. src/modules

C'est ici que se trouve le vrai métier. Chaque module possède son validateur, ses routes, son contrôleur, son service et son dépôt.

4.6. Ordre de reconstruction à respecter

Pour refaire correctement le projet, il faut respecter cet ordre:

1. définir le besoin métier;
2. préparer l'environnement;
3. monter le socle HTTP;
4. préparer Prisma et la base;
5. installer les middlewares communs;
6. créer les modules un par un;
7. raccorder les modules au routeur principal;
8. documenter, tester et déployer.

Le point pédagogique le plus important est le suivant: **on ne commence pas par le module ventes**. On commence par les fondations, puis par les modules les plus simples, et seulement ensuite par les modules les plus sensibles.

5. Partie 3. Étude détaillée de chaque module

5.1. Logique générale de construction d'un module

Chaque module du projet suit volontairement la même mécanique. Cette régularité n'est pas un détail de style: elle permet à un lecteur débutant de refaire le projet sans se perdre. Pour bien comprendre cette mécanique, il faut distinguer deux ordres.

5.1.1. Ordre conseillé pour construire un module

1. définir clairement ce que le module doit accomplir;
2. installer les dépendances utiles au module;
3. écrire le **validateur**;
4. écrire les **routes**;
5. écrire le **controller**;
6. écrire le **service**;
7. écrire le **repository**;
8. assembler le tout dans le **container** puis raccorder le module;
9. vérifier le comportement dans **Swagger**.

5.1.2. Ordre conseillé pour lire un module

1. commencer par l'objectif du module;
2. regarder les **routes** pour voir les entrées;
3. regarder le **controller** pour voir la forme des réponses;
4. regarder le **service** pour voir les règles métier;
5. regarder le **repository** pour voir l'accès aux données;
6. revenir au **validateur** pour comprendre la forme précise des entrées;
7. terminer par le **container** puis le test dans **Swagger**.

Cette distinction est importante: on ne construit pas toujours dans le même ordre qu'on lit, mais dans ce mémoire, l'explication suit un ordre pédagogique stable pour qu'un autre développeur puisse refaire chaque module pas à pas.

5.2. Module d'authentification

5.2.1. Ce que ce module doit faire

Le module **auth** doit permettre:

- de recevoir un email et un mot de passe;
- de vérifier que la demande de connexion est bien formée;

- de fabriquer un jeton JWT;
- de relire l'utilisateur courant via `/me`.

5.2.2. Dépendances principalement utilisées

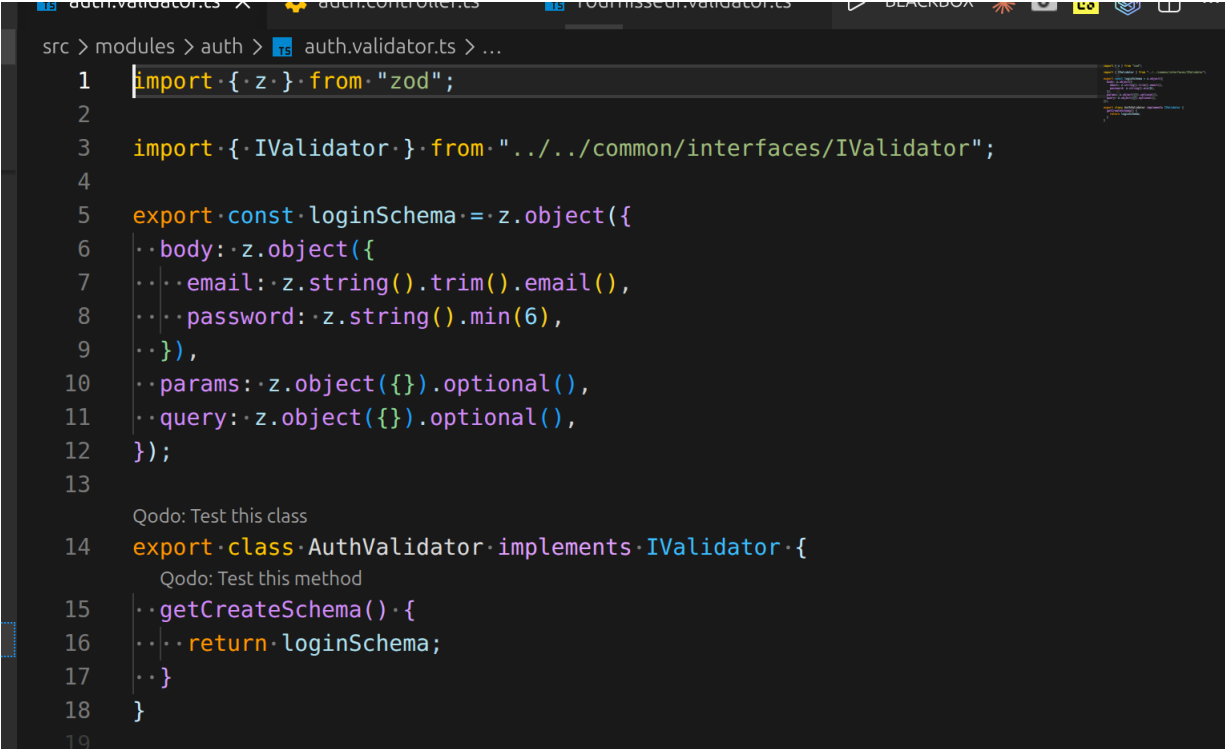
Pour reconstruire ce module, les dépendances essentielles sont:

- `express` pour les routes et le controller;
- `zod` pour valider la demande de connexion;
- `jsonwebtoken` pour fabriquer le JWT.

Commande d'installation minimale: `npm install express zod jsonwebtoken`.

5.2.3. Étape 1. Valider les entrées du module

Le validateur impose la forme minimale de la connexion: email correct et mot de passe assez long.



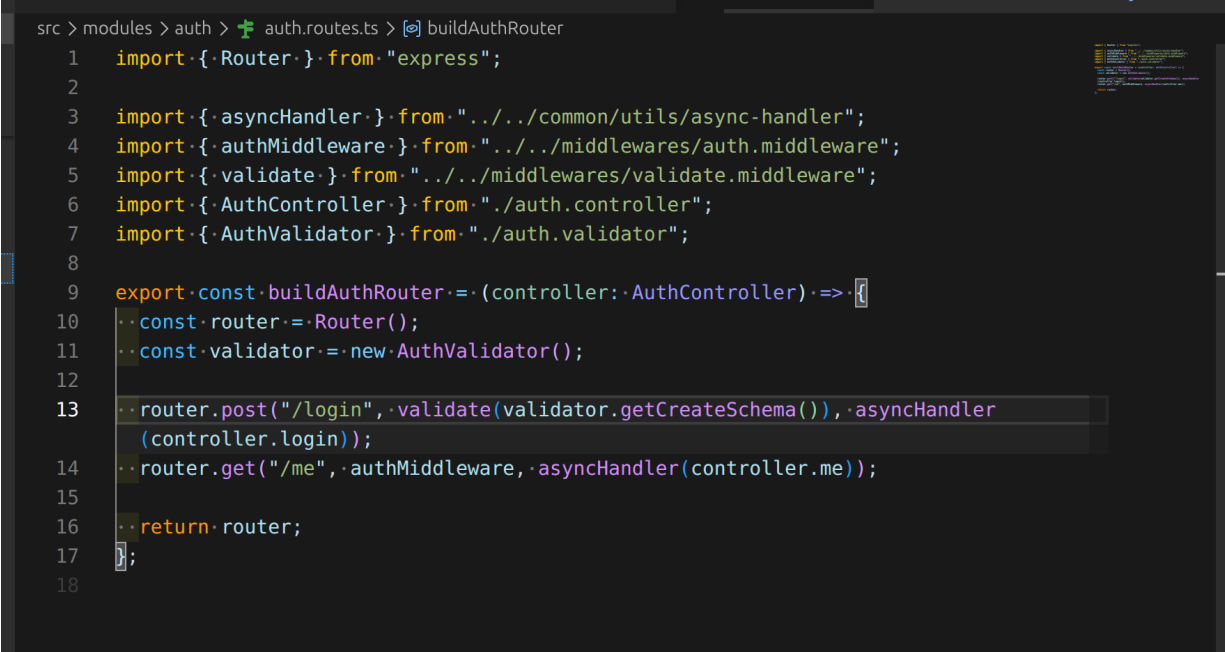
```
src > modules > auth > auth.validator.ts > ...
1  import { z } from "zod";
2
3  import { IValidator } from "../../common/interfaces/IValidator";
4
5  export const loginSchema = z.object({
6    body: z.object({
7      email: z.string().trim().email(),
8      password: z.string().min(6),
9    }),
10   params: z.object({}).optional(),
11   query: z.object({}).optional(),
12 });
13
14  Qodo: Test this class
15  export class AuthValidator implements IValidator {
16    Qodo: Test this method
17    getCreateSchema() {
18      return loginSchema;
19    }
20  }
```

Figure 5.1: Capture réelle du validateur du module auth

Cette étape est importante, car elle évite de transmettre au service des données vides ou mal structurées.

5.2.4. Étape 2. Déclarer les routes du module

Les routes exposent les deux entrées du module: connexion et lecture de l'utilisateur courant.



```

src > modules > auth > auth.routes.ts > buildAuthRouter
1  import { Router } from "express";
2
3  import { asyncHandler } from "../../common/utils/async-handler";
4  import { authMiddleware } from "../../middlewares/auth.middleware";
5  import { validate } from "../../middlewares/validate.middleware";
6  import { AuthController } from "../auth.controller";
7  import { AuthValidator } from "../auth.validator";
8
9  export const buildAuthRouter = (controller: AuthController) => {
10   const router = Router();
11   const validator = new AuthValidator();
12
13   router.post("/login", validate(validator.getCreateSchema()), asyncHandler
     (controller.login));
14   router.get("/me", authMiddleware, asyncHandler(controller.me));
15
16   return router;
17 };
18

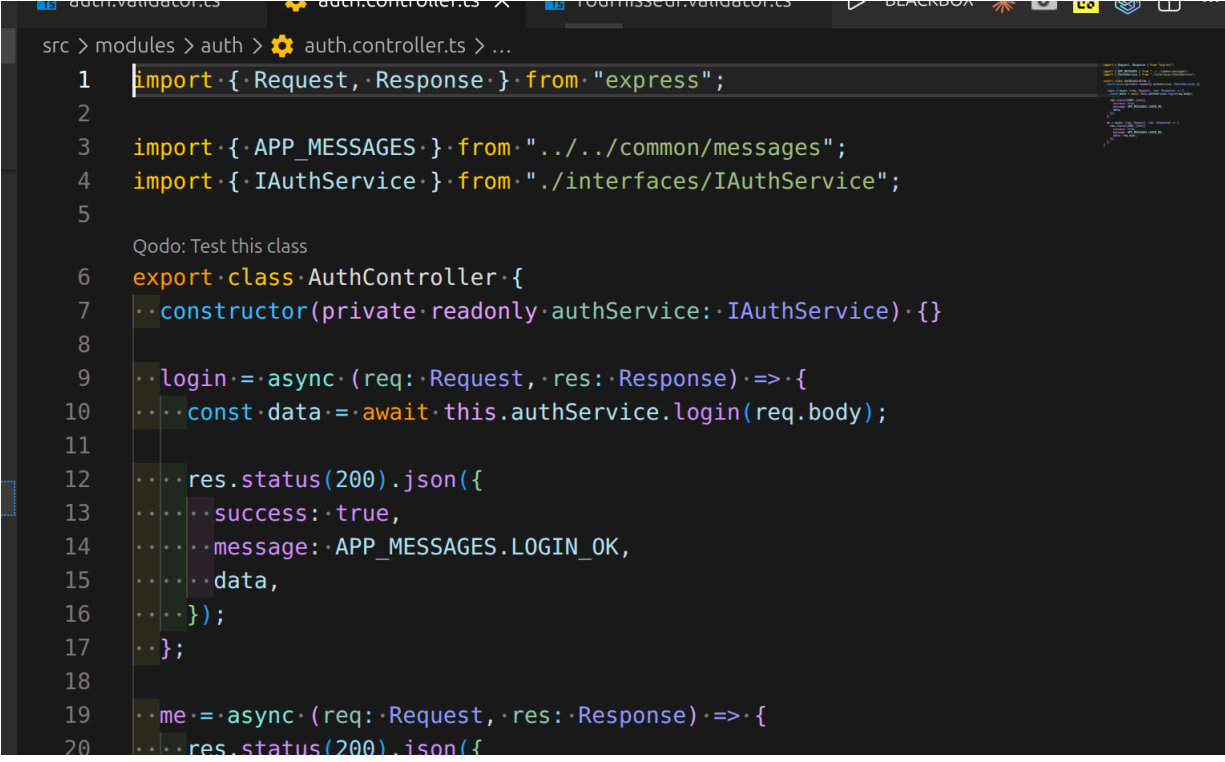
```

Figure 5.2: Capture réelle des routes d'authentification

La route ne fait pas la logique de connexion. Elle branche seulement le validateur, le contrôleur et le middleware d'authentification quand c'est nécessaire.

5.2.5. Étape 3. Construire le contrôleur

Le contrôleur transforme le résultat du service en réponse HTTP.



```

src > modules > auth > auth.controller.ts > ...
1  import { Request, Response } from "express";
2
3  import { APP_MESSAGES } from "../../common/messages";
4  import { IAuthService } from "../interfaces/IAuthService";
5
6  Qodo: Test this class
7  export class AuthController {
8   constructor(private readonly authService: IAuthService) {}
9
10   login = async (req: Request, res: Response) => {
11     const data = await this.authService.login(req.body);
12     res.status(200).json({
13       success: true,
14       message: APP_MESSAGES.LOGIN_OK,
15       data,
16     });
17   };
18
19   me = async (req: Request, res: Response) => {
20     res.status(200).json({

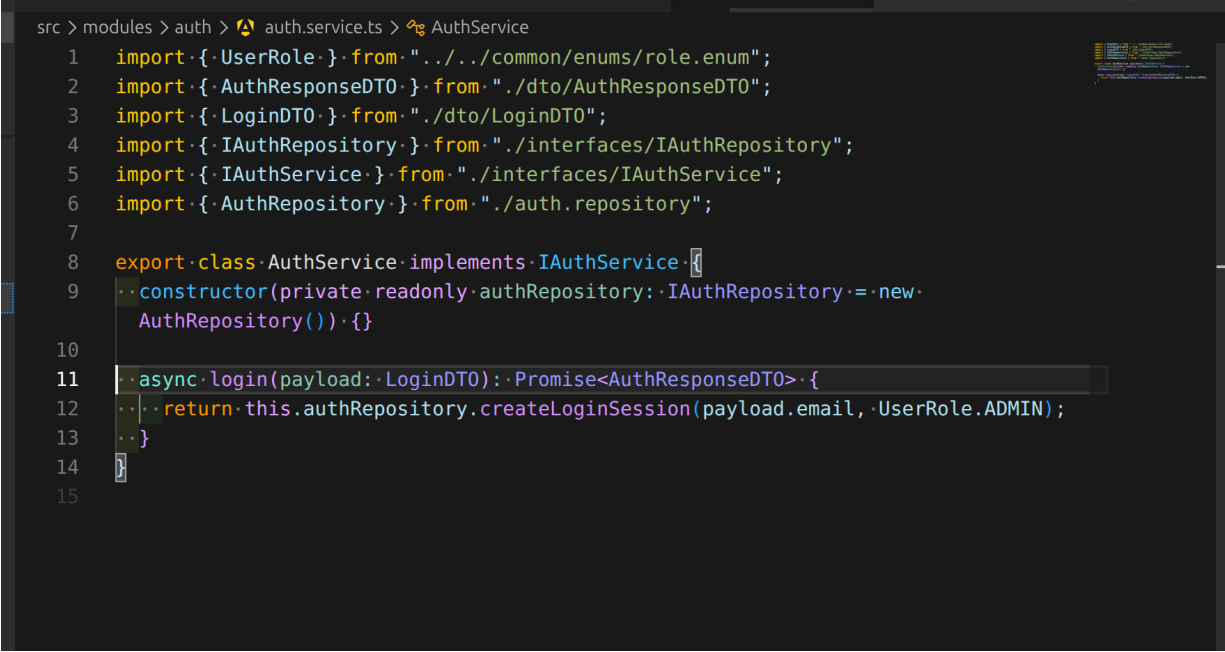
```

Figure 5.3: Capture réelle du controller du module auth

Cette couche permet d'obtenir une réponse homogène avec `success`, `message` et `data`.

5.2.6. Étape 4. Écrire le service

Le service porte la logique de connexion.



```
src > modules > auth > auth.service.ts > AuthService
1  import { UserRole } from "../../common/enums/role.enum";
2  import { AuthResponseDTO } from "../dto/AuthResponseDTO";
3  import { LoginDTO } from "../dto/LoginDTO";
4  import { IAuthRepository } from "../interfaces/IAuthRepository";
5  import { IAuthService } from "../interfaces/IAuthService";
6  import { AuthRepository } from "../auth.repository";
7
8  export class AuthService implements IAuthService {
9    constructor(private readonly authRepository: IAuthRepository = new
      AuthRepository()) {}
10
11    async login(payload: LoginDTO): Promise<AuthResponseDTO> {
12      return this.authRepository.createLoginSession(payload.email, UserRole.ADMIN);
13    }
14  }
15
```

Figure 5.4: Capture réelle du service de connexion

Le service reçoit la demande, décide quoi faire et demande ensuite au repository de produire le jeton.

5.2.7. Étape 5. Écrire le repository

Dans ce module, le repository a un rôle particulier: il fabrique le JWT.

```
src > modules > auth > auth.repository.ts > AuthRepository
1  import jwt from "jsonwebtoken";
2
3  import { UserRole } from "../../common/enums/role.enum";
4  import { env } from "../../config/env";
5  import { AuthResponseDTO } from "../dto/AuthResponseDTO";
6  import { IAuthRepository } from "../interfaces/IAuthRepository";
7
8  export class AuthRepository implements IAuthRepository {
9    .. async createLoginSession(email: string, role: UserRole):
10      .. Promise<AuthResponseDTO> {
11        .. const token = jwt.sign({ role }, env.JWT_SECRET, {
12        ..   .. subject: email,
13        ..   .. expiresIn: "1d",
14        ..   .. });
15        .. return {
16        ..   .. token,
17        ..   .. user: {
18        ..     .. email,
19        ..     .. role,
20        ..     .. },
```

Figure 5.5: Capture réelle de la génération du JWT

La ligne `jwt.sign(...)` signifie que le backend crée un jeton signé à partir d'informations minimales. Ce jeton pourra ensuite être présenté au backend pour prouver l'identité du client.

5.2.8. Étape 6. Assembler le module

Le container relie le repository, le service, le controller et les routes. Une fois cela fait, le module peut être branché au backend et devenir réellement utilisable.

```
src > container > auth.container.ts > ...
1  import { AuthController } from "../modules/auth/auth.controller";
2  import { AuthRepository } from "../modules/auth/auth.repository";
3  import { buildAuthRouter } from "../modules/auth/auth.routes";
4  import { AuthService } from "../modules/auth/auth.service";
5
6  export const buildAuthModule = () => {
7    .. const repository = new AuthRepository();
8    .. const service = new AuthService(repository);
9    .. const controller = new AuthController(service);
10
11    .. return buildAuthRouter(controller);
12  };
13
```

Figure 5.6: Capture réelle de l'assemblage du module `auth` dans `src/container/auth.container.ts`

```

src > modules > auth > auth.routes.ts > ...
1  import { Router } from "express";
2
3  import { asyncHandler } from "../../common/utils/async-handler";
4  import { authMiddleware } from "../../middlewares/auth.middleware";
5  import { validate } from "../../middlewares/validate.middleware";
6  import { AuthController } from "../auth.controller";
7  import { AuthValidator } from "../auth.validator";
8
9  export const buildAuthRouter = (controller: AuthController) => {
10   const router = Router();
11   const validator = new AuthValidator();
12
13   router.post("/login", validate(validator.getCreateSchema()), asyncHandler(
14     controller.login));
15   router.get("/me", authMiddleware, asyncHandler(controller.me));
16   return router;
17 };
18

```

Figure 5.7: Capture réelle du module d'authentification

À ce stade, le lecteur doit comprendre qu'un module n'est pas seulement une collection de fichiers: c'est un enchaînement complet, depuis la validation de l'entrée jusqu'au branchement dans l'application.

5.2.9. Étape 7. Vérifier le module auth dans Swagger

Une fois le module branché, il faut vérifier qu'il apparaît bien dans la documentation exposée par l'API et que ses routes sont testables.

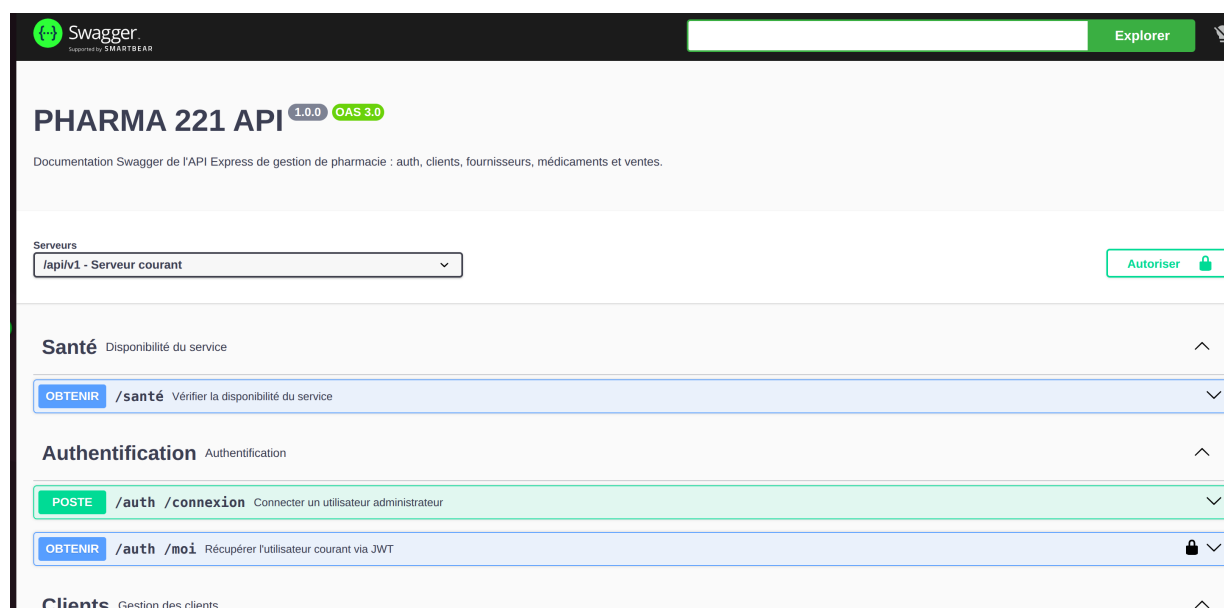


Figure 5.8: Vue générale de la documentation Swagger déployée

Le test attendu à ce stade est simple:

- vérifier que les routes **auth** sont visibles;

- envoyer une demande de connexion cohérente;
- contrôler que le backend renvoie bien un jeton ou une erreur claire.

5.2.10. Ce qu'il faut retenir

Le module `auth` est une porte d'entrée technique. Il est crédible, mais il reste limité car il ne repose pas encore sur une table utilisateur persistante.

5.3. Module clients

5.3.1. Ce que ce module doit faire

Le module `clients` doit permettre:

- de créer un client;
- de lister les clients;
- de consulter un client précis;
- de mettre à jour un client;
- de supprimer un client si cette suppression reste compatible avec l'historique des ventes.

5.3.2. Dépendances principalement utilisées

Les dépendances principales sont:

- `express` pour exposer les endpoints;
- `zod` pour valider les entrées;
- `@prisma/client` pour accéder aux données.

Commande d'installation minimale: `npm install express zod @prisma/client`.

Si le schéma Prisma vient d'être modifié, il faut aussi régénérer le client avec `npx prisma generate`.

5.3.3. Étape 1. Définir les données d'entrée

Le validateur décrit les champs attendus pour la création, la lecture par identifiant et la mise à jour.

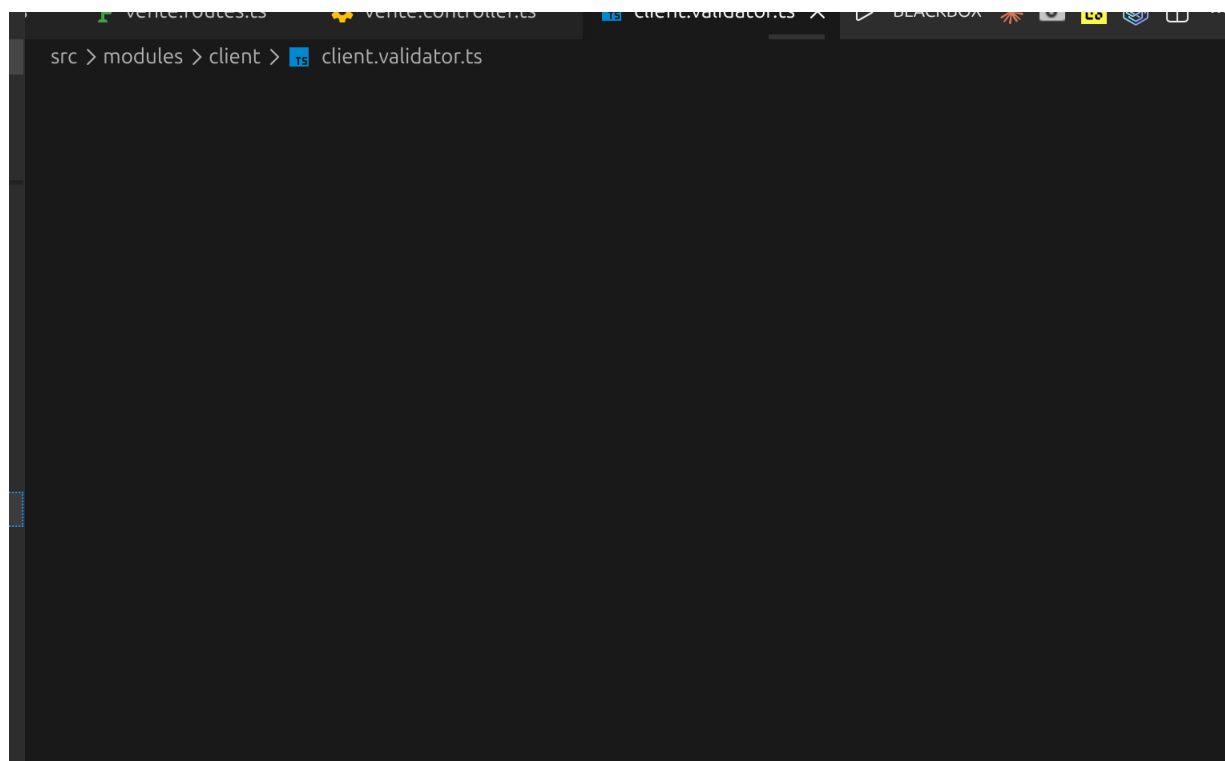


Figure 5.9: Capture réelle du validateur du module client

Le rôle de ce fichier est de dire clairement ce qu'un client doit contenir avant même d'atteindre la logique métier.

5.3.4. Étape 2. Déclarer les routes

Les routes disent quelles URLs déclenchent quelles actions.

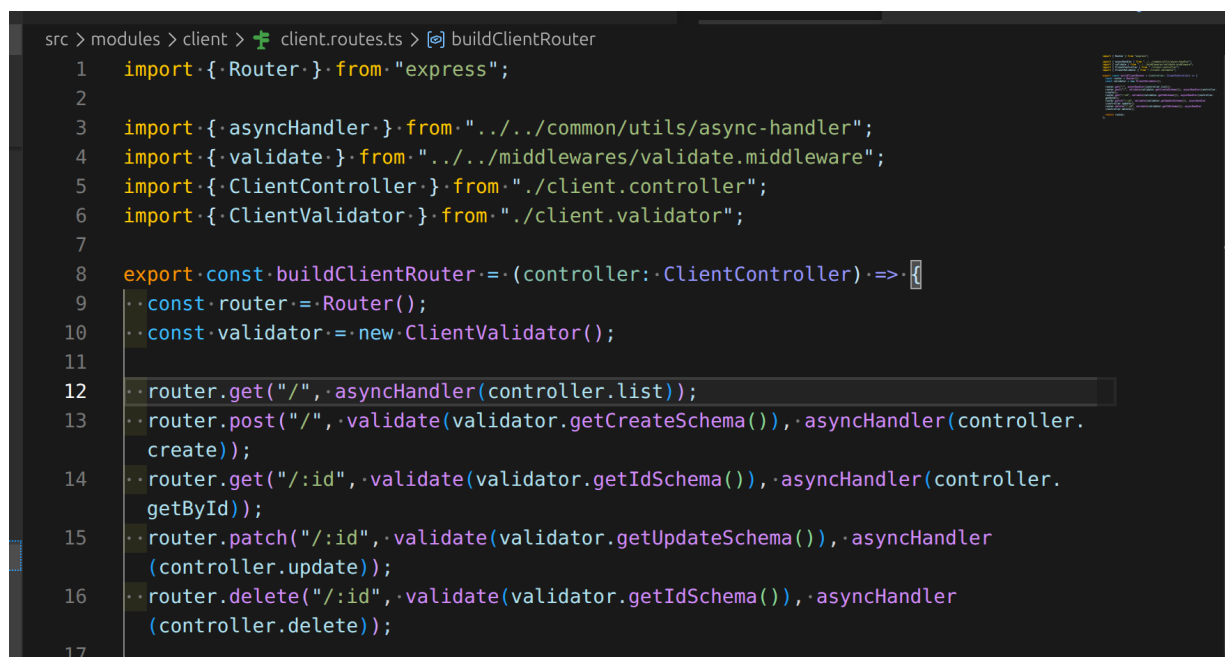
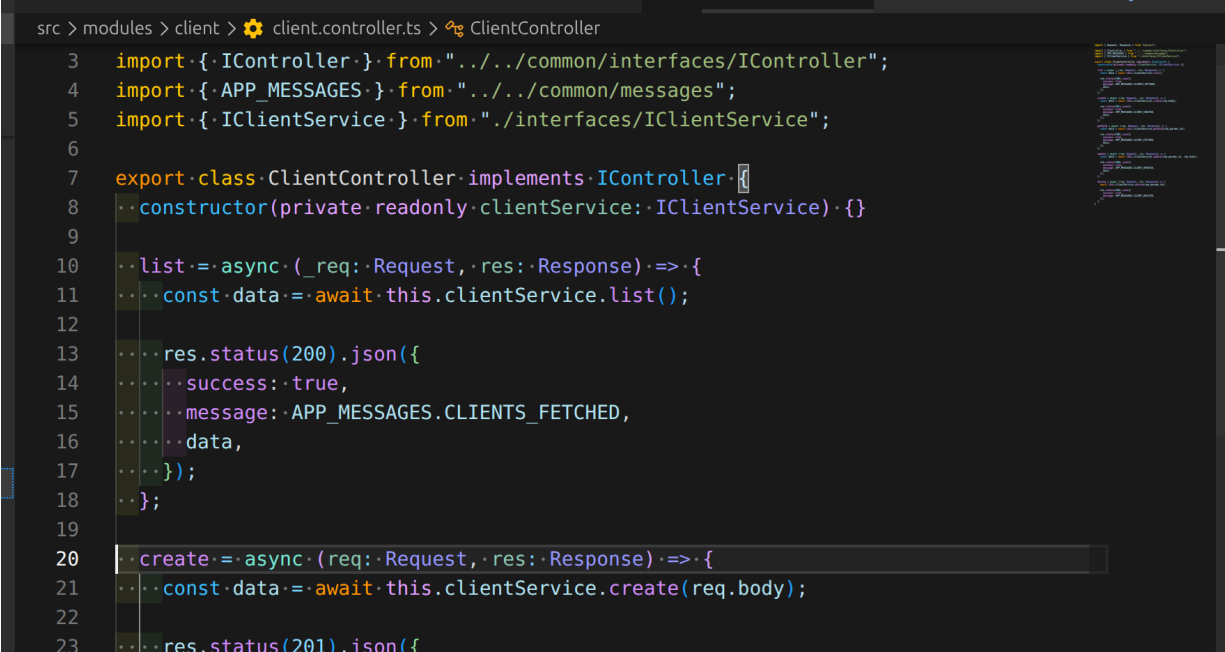


Figure 5.10: Capture réelle de la couche routes du module client

À ce niveau, la logique est simple: une URL arrive, la validation s'exécute, puis le contrôleur est appelé.

5.3.5. Étape 3. Construire le contrôleur

Le contrôleur formate la réponse.



```
src > modules > client > client.controller.ts > ClientController
3  import { IController } from "../../common/interfaces/IController";
4  import { APP_MESSAGES } from "../../common/messages";
5  import { IClientService } from "../interfaces/IClientService";
6
7  export class ClientController implements IController {
8    constructor(private readonly clientService: IClientService) {}
9
10   list = async (_req: Request, _res: Response) => {
11     const data = await this.clientService.list();
12
13     _res.status(200).json({
14       success: true,
15       message: APP_MESSAGES.CLIENTS_FETCHED,
16       data,
17     });
18   };
19
20   create = async (req: Request, _res: Response) => {
21     const data = await this.clientService.create(req.body);
22
23     _res.status(201).json({
```

Figure 5.11: Capture réelle de la couche controller du module client

Le contrôleur ne décide pas si un email existe déjà. Il ne fait qu'appeler le service et renvoyer un résultat HTTP cohérent.

5.3.6. Étape 4. Écrire le service

C'est ici que le module commence réellement à "réfléchir".

```

src > modules > client > client.service.ts > ClientService
1  import { IService } from "../../common/interfaces/IService";
2  import { AppError } from "../../common/errors/app-error";
3  import { CreateClientDTO } from "../dto/CreateClientDTO";
4  import { UpdateClientDTO } from "../dto/UpdateClientDTO";
5  import { ClientRecord, IClientRepository } from "../interfaces/IClientRepository";
6  import { IClientService } from "../interfaces/IClientService";
7  import { ClientRepository } from "../client.repository";
8
9  export class ClientService
10  .. implements IService<ClientRecord, CreateClientDTO, UpdateClientDTO, string>,
    IClientService
11
12  .. constructor(private readonly clientRepository: IClientRepository = new
    ClientRepository()) {}
13
14  .. private async ensureEmailAvailable(email: string, clientId?: string) {
15  ..   const existingClient = await this.clientRepository.findByEmail(email,
    clientId);
16
17  ..   if (existingClient) {
18  ..     throw new AppError(409, "L'email client existe déjà.", {

```

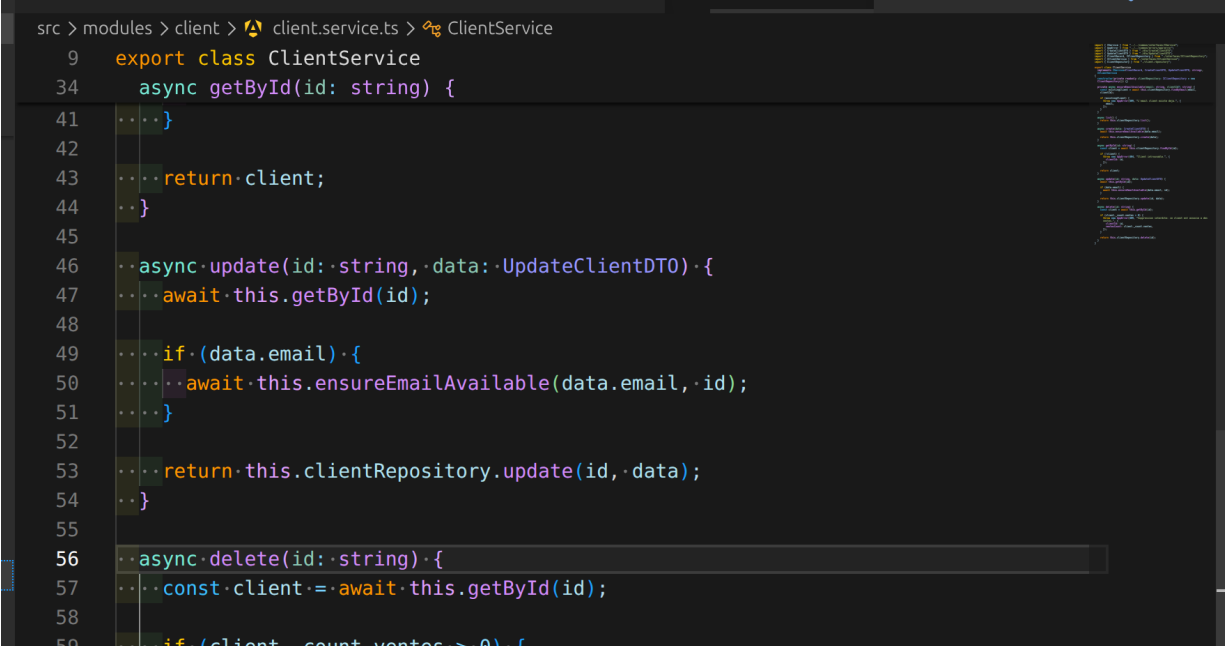
Figure 5.12: Capture réelle de la vérification d'unicité dans *client.service.ts*

```

src > modules > client > client.service.ts > ClientService
9  export class ClientService
13
14  .. private async ensureEmailAvailable(email: string, clientId?: string) {
15  ..   const existingClient = await this.clientRepository.findByEmail(email,
    clientId);
16
17  ..   if (existingClient) {
18  ..     throw new AppError(409, "L'email client existe déjà.", {
19  ..       email,
20  ..     });
21  ..   }
22  .. }
23
24  .. async list() {
25  ..   return this.clientRepository.list();
26  .. }
27
28  .. async create(data: CreateClientDTO) {
29  ..   await this.ensureEmailAvailable(data.email);
30
31  ..   return this.clientRepository.create(data);

```

Figure 5.13: Capture réelle de la création d'un client dans *client.service.ts*



```

src > modules > client > client.service.ts > ClientService
9   export class ClientService
34   async getById(id: string) {
41   ...}
42
43   ...return client;
44   ...}
45
46   async update(id: string, data: UpdateClientDTO) {
47   ...await this.getById(id);
48
49   ...if (data.email) {
50   ...await this.ensureEmailAvailable(data.email, id);
51   ...}
52
53   ...return this.clientRepository.update(id, data);
54   ...}
55
56   async delete(id: string) {
57   ...const client = await this.getById(id);
58
59   ...if (client.count.ventes > 0) {

```

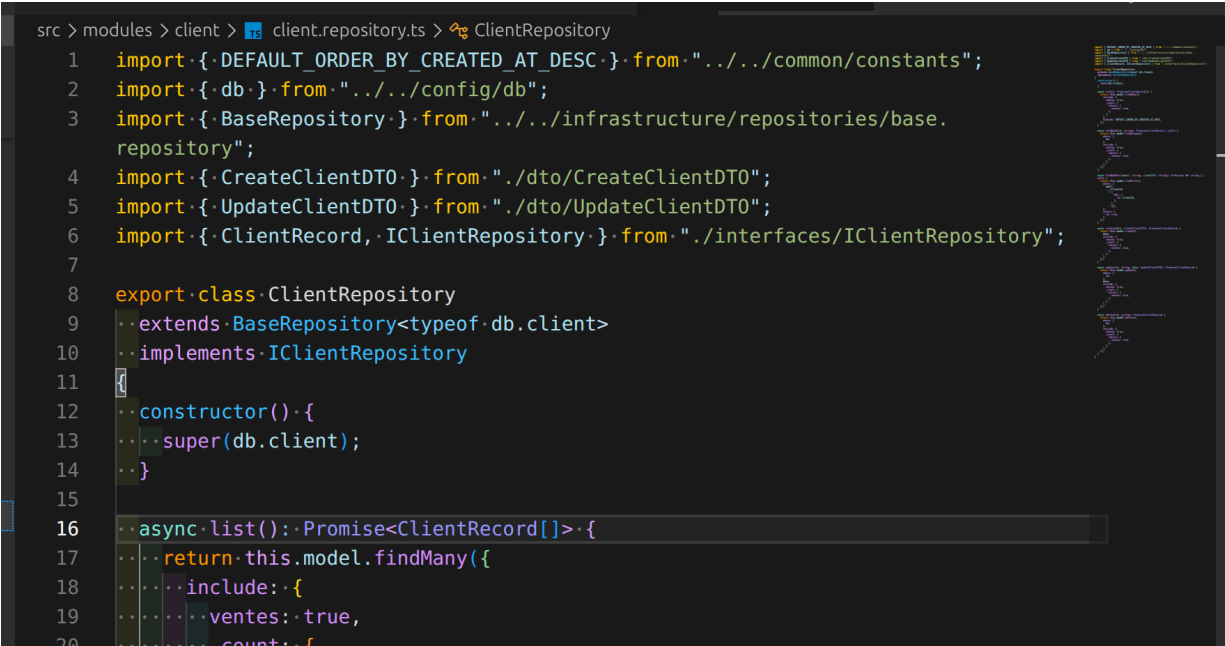
Figure 5.14: Capture réelle de la suppression protégée dans `client.service.ts`

Le service pose deux règles fortes:

- l'email doit rester unique;
- un client lié à des ventes ne doit pas être supprimé librement.

5.3.7. Étape 5. Écrire le repository

Le repository prépare les accès Prisma.



```

src > modules > client > client.repository.ts > ClientRepository
1   import { DEFAULT_ORDER_BY_CREATED_AT_DESC } from "../../common/constants";
2   import { db } from "../../config/db";
3   import { BaseRepository } from "../../infrastructure/repositories/base.
   repository";
4   import { CreateClientDTO } from "../dto/CreateClientDTO";
5   import { UpdateClientDTO } from "../dto/UpdateClientDTO";
6   import { ClientRecord, IClientRepository } from "../interfaces/IClientRepository";
7
8   export class ClientRepository
9   ...extends BaseRepository<typeof db.client>
10  ...implements IClientRepository
11  {
12  ...constructor() {
13  ...super(db.client);
14  ...}
15
16  ...async list(): Promise<ClientRecord[]> {
17  ...return this.model.findMany({
18  ...include: {
19  ...ventes: true,
20  ...count: {

```

Figure 5.15: Capture réelle de la lecture de liste dans `client.repository.ts`



Figure 5.16: Capture réelle de la recherche par email dans *client.repository.ts*

Le repository ne décide pas si la suppression est légitime. Il exécute seulement les accès à la base demandés par le service.

5.3.8. Étape 6. Assembler le module puis le raccorder au backend

Quand validator, routes, controller, service et repository sont prêts, il faut les relier dans le container du module.

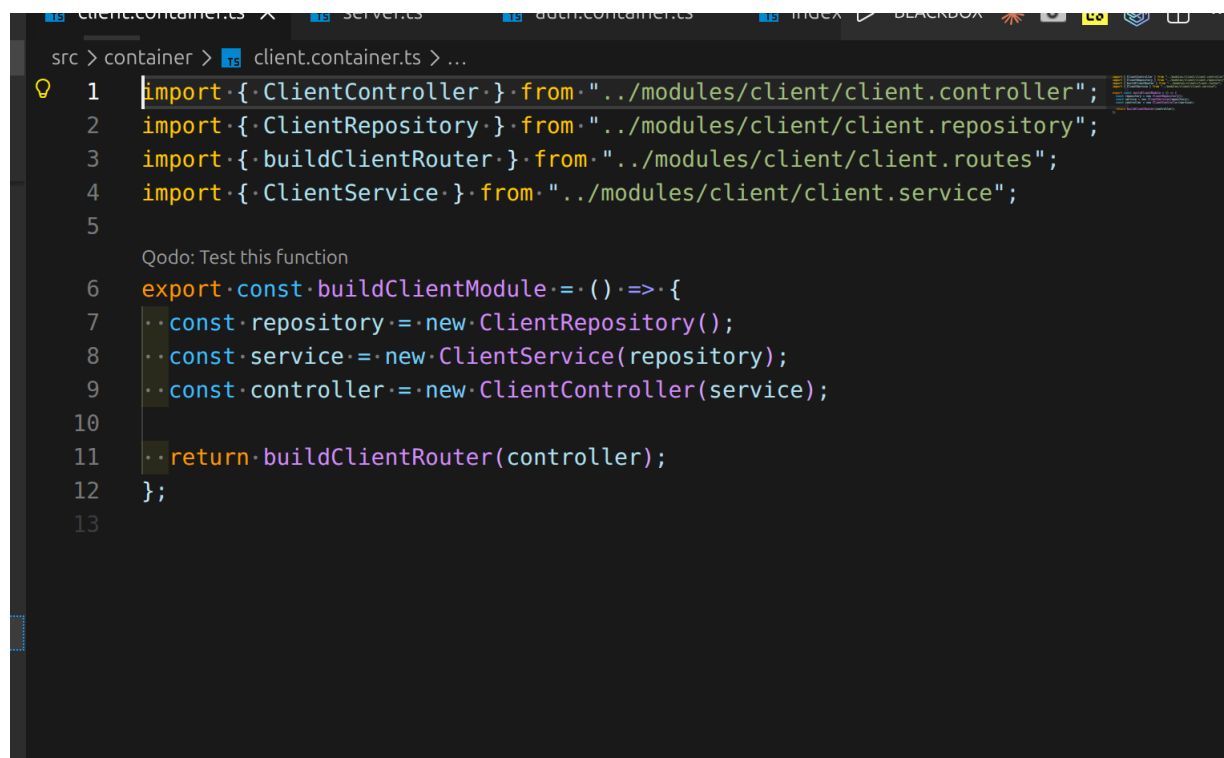


Figure 5.17: Capture réelle de l'assemblage du module client

```

src > modules > client > client.service.ts > ClientService
9   export class ClientService
34   async getById(id: string) {
41   ...}
42
43   ...return client;
44   ...}
45
46   ...async update(id: string, data: UpdateClientDTO) {
47   ...await this.getById(id);
48
49   ...if (data.email) {
50   ...await this.ensureEmailAvailable(data.email, id);
51   ...}
52
53   ...return this.clientRepository.update(id, data);
54   ...}
55
56   ...async delete(id: string) {
57   ...const client = await this.getById(id);
58
59   ...if (client.count.ventes > 0) {

```

Figure 5.18: Capture réelle du module clients

Cette étape est indispensable: sans container, le module existe dans les fichiers, mais il n'est pas encore branché dans l'application.

5.3.9. Étape 7. Vérifier le module clients dans Swagger

Après assemblage, il faut vérifier dans Swagger que les routes clients sont bien publiées et qu'un test de création ou de lecture peut être lancé.

Clients Gestion des clients			^
OBTENIR	/clients	Lister les clients	▼
POSTE	/clients	Créer un client	▼
OBTENIR	/clients/{id}	Récupérer un client	▼
CORRECTIF	/clients/{id}	Mettre à jour un client	▼
SUPPRIMER	/clients/{id}	Supprimer un client	▼
Fournisseurs Gestion des fournisseurs			^
OBTENIR	/fournisseurs	Lister les fournisseurs	▼
POSTE	/fournisseurs	Créer un fournisseur	▼
OBTENIR	/fournisseurs/{id}	Récupérer un fournisseur	▼
CORRECTIF	/fournisseurs/{id}	Mettre à jour un fournisseur	▼
SUPPRIMER	/fournisseurs/{id}	Supprimer un fournisseur	▼

Figure 5.19: Vue des routes clients et fournisseurs dans Swagger

Le contrôle attendu à ce niveau est le suivant:

- créer un client avec des données valides;

- relire la liste des clients;
- vérifier qu'un mauvais identifiant ou un email déjà utilisé produit une erreur claire.

Ce module montre bien la logique du projet: on ne fait pas seulement un CRUD. On protège aussi la cohérence du domaine.

5.4. Module fournisseurs

5.4.1. Ce que ce module doit faire

Le module **fournisseurs** sert à gérer les sources d'approvisionnement. Il doit permettre d'enregistrer un fournisseur, de le modifier, de le consulter et de le supprimer seulement si cette suppression ne casse pas la relation avec les médicaments.

5.4.2. Dépendances principalement utilisées

Les dépendances principales sont:

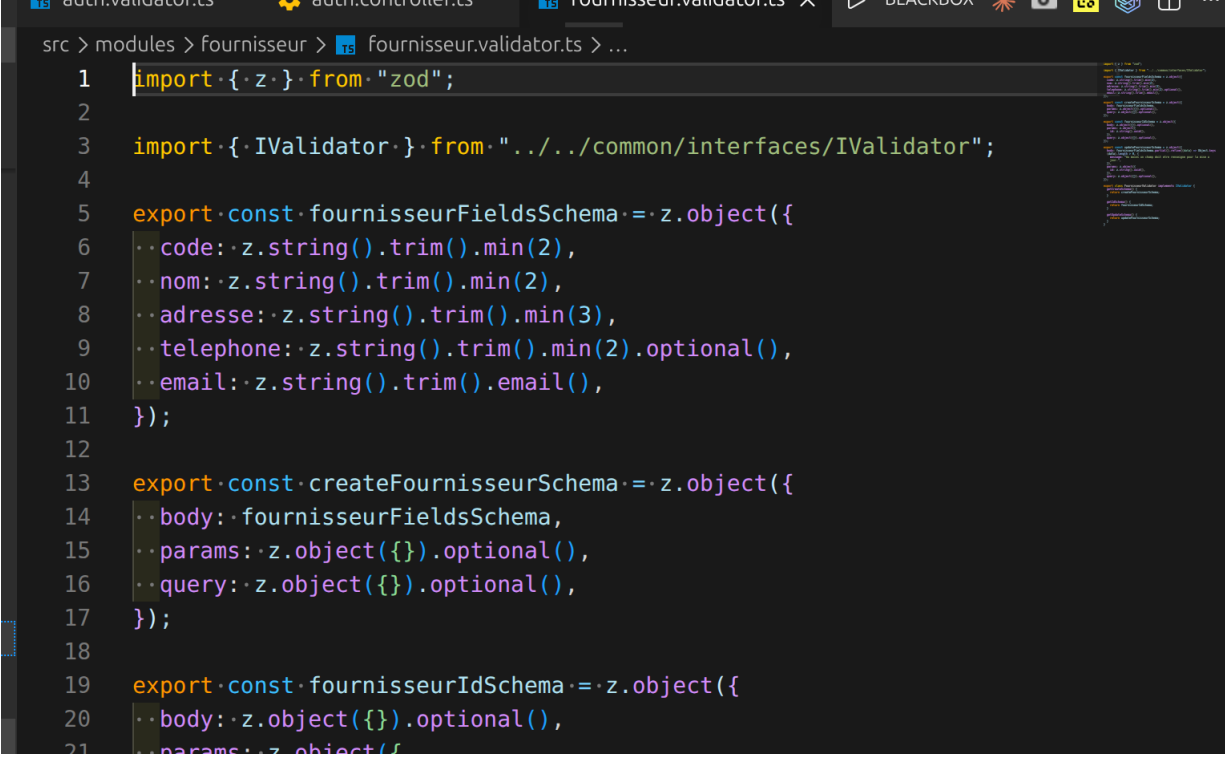
- `express`;
- `zod`;
- `@prisma/client`.

Commande d'installation minimale: `npm install express zod @prisma/client`.

Comme pour les autres modules connectés à la base, `npm run prisma generate` doit être relancé si le schéma évolue.

5.4.3. Étape 1. Définir les données d'entrée

Le validateur impose la forme des données fournisseur.



```

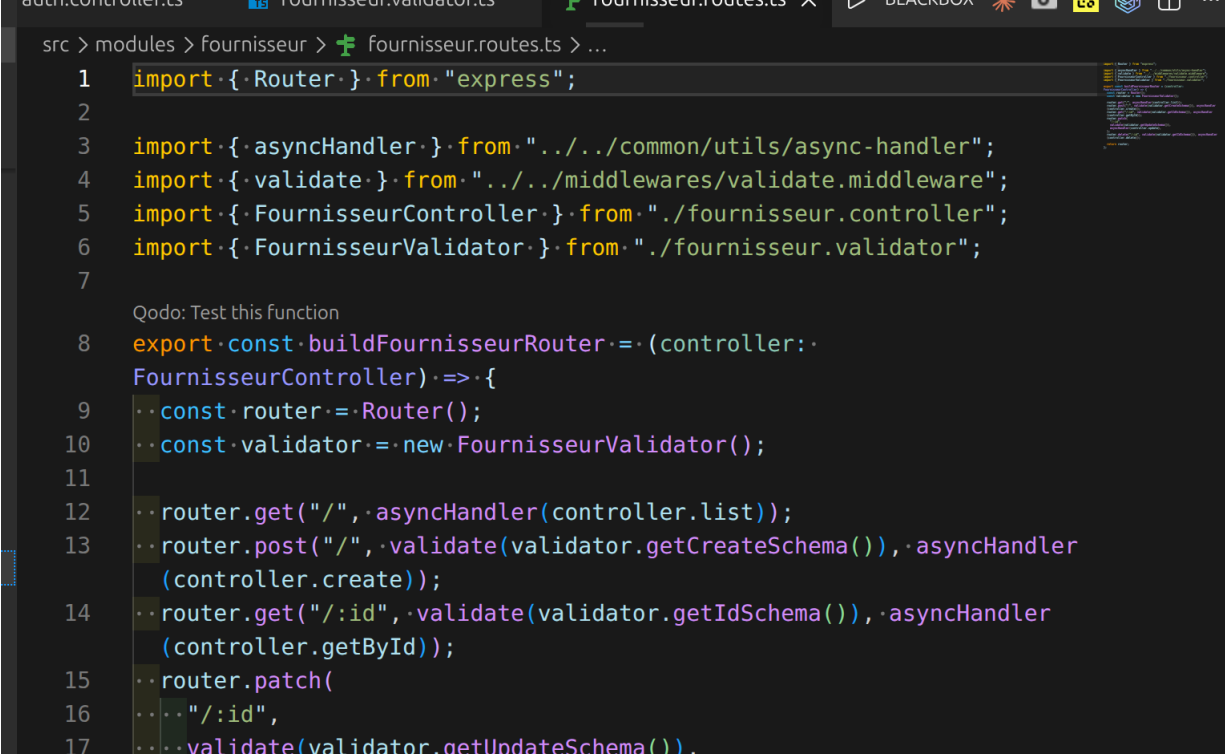
src > modules > fournisseur > fournisseur.validator.ts > ...
1  import { z } from "zod";
2
3  import { IValidator } from "../../common/interfaces/IValidator";
4
5  export const fournisseurFieldsSchema = z.object({
6    ..code: z.string().trim().min(2),
7    ..nom: z.string().trim().min(2),
8    ..adresse: z.string().trim().min(3),
9    ..telephone: z.string().trim().min(2).optional(),
10   ..email: z.string().trim().email(),
11  });
12
13  export const createFournisseurSchema = z.object({
14    ..body: fournisseurFieldsSchema,
15    ..params: z.object({}).optional(),
16    ..query: z.object({}).optional(),
17  });
18
19  export const fournisseurIdSchema = z.object({
20    ..body: z.object({}).optional(),
21    ..params: z.object({}).optional(),
22  });

```

Figure 5.20: Capture réelle du validateur du module fournisseur

Ce fichier est important, car il fixe les champs attendus très tôt: code, nom, adresse, téléphone et email.

5.4.4. Étape 2. Déclarer les routes



```

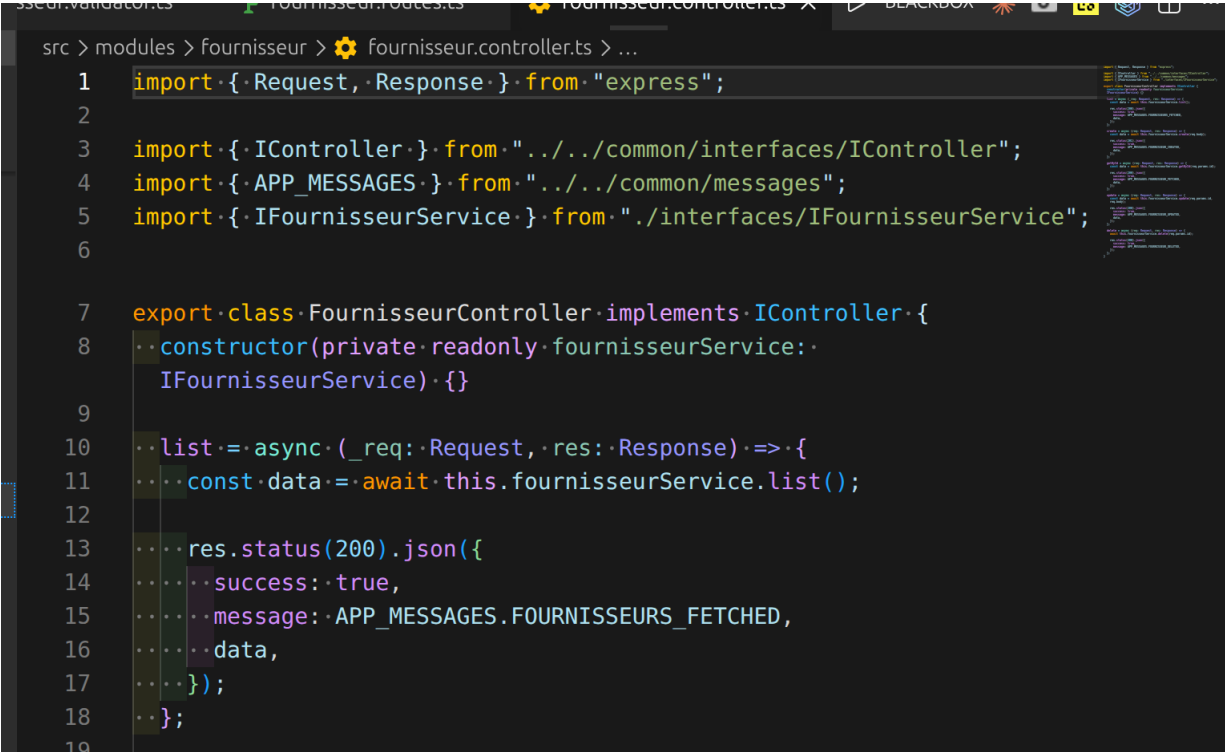
src > modules > fournisseur > fournisseur.routes.ts > ...
1  import { Router } from "express";
2
3  import { asyncHandler } from "../../common/utils/async-handler";
4  import { validate } from "../../middlewares/validate.middleware";
5  import { FournisseurController } from "../fournisseur.controller";
6  import { FournisseurValidator } from "../fournisseur.validator";
7
8  Qodo: Test this function
9  export const buildFournisseurRouter = (controller: FournisseurController) => {
10    ..const router = Router();
11    ..const validator = new FournisseurValidator();
12    ..router.get("/", asyncHandler(controller.list));
13    ..router.post("/", validate(validator.getCreateSchema()), asyncHandler(
14      controller.create));
15    ..router.get("/:id", validate(validator.getIdSchema()), asyncHandler(
16      controller.getById));
17    ..router.patch(
18      .."/:id",
19      ..validate(validator.getUpdateSchema()),
20    );
21  };

```

Figure 5.21: Capture réelle des routes du module fournisseur

On retrouve ici le même modèle que pour le module client: validation, puis passage au contrôleur.

5.4.5. Étape 3. Construire le contrôleur



```
src > modules > fournisseur > fournisseur.controller.ts > ...
1  import { Request, Response } from "express";
2
3  import { IController } from "../../common/interfaces/IController";
4  import { APP_MESSAGES } from "../../common/messages";
5  import { IFournisseurService } from "../interfaces/IFournisseurService";
6
7  export class FournisseurController implements IController {
8    constructor(private readonly fournisseurService: IFournisseurService) {}
9
10   list = async (_req: Request, res: Response) => {
11     const data = await this.fournisseurService.list();
12
13     res.status(200).json({
14       success: true,
15       message: APP_MESSAGES.FOURNISSEURS_FETCHED,
16       data,
17     });
18   };
19
```

Figure 5.22: Capture réelle du controller du module fournisseur

Le contrôleur porte le contrat HTTP du module: codes de réponse, messages et données renvoyées.

5.4.6. Étape 4. Écrire le service

Le service porte la vraie logique fournisseur.

```

src > modules > fournisseur > fournisseur.service.ts > FournisseurService
4  import { UpdateFournisseurDTO } from '../dto/UpdateFournisseurDTO';
5  import { IFournisseurRepository, FournisseurRecord } from '../interfaces/
   IFournisseurRepository';
6  import { IFournisseurService } from '../interfaces/IFournisseurService';
7  import { FournisseurRepository } from '../fournisseur.repository';
8
9  export class FournisseurService
10  .. implements
11  ... IService<FournisseurRecord, CreateFournisseurDTO, UpdateFournisseurDTO,
   string>,
12  ... IFournisseurService
13  {
14  .. constructor(
15  ... private readonly fournisseurRepository: IFournisseurRepository = new
   FournisseurRepository(),
16  .. ) {}
17
18  .. private async ensureCodeAvailable(code: string, fournisseurId?: string) {
19  ... const existingFournisseur = await this.fournisseurRepository.findByCode(code,
   fournisseurId);
20

```

Figure 5.23: Capture réelle du contrôle d'unicité fournisseur

```

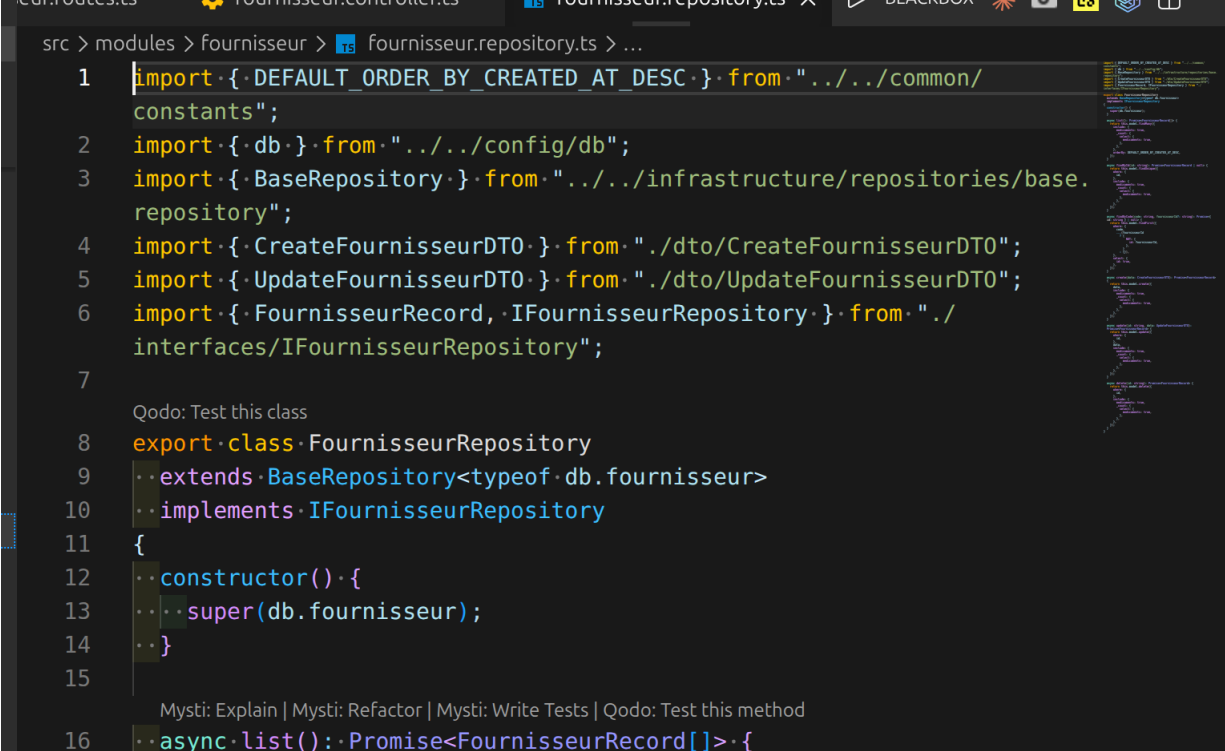
src > modules > fournisseur > fournisseur.service.ts > FournisseurService
9  export class FournisseurService
38  async getById(id: string) {
45  ... }
46
47  ... return fournisseur;
48  .. }
49
50  .. async update(id: string, data: UpdateFournisseurDTO) {
51  ... await this.getById(id);
52
53  ... if (data.code) {
54  ...   await this.ensureCodeAvailable(data.code, id);
55  ... }
56
57  ... return this.fournisseurRepository.update(id, data);
58  .. }
59
60  .. async delete(id: string) {
61  ... const fournisseur = await this.getById(id);
62
63  ... if (fournisseur.count.medicaments > 0) {

```

Figure 5.24: Capture réelle de la suppression protégée d'un fournisseur

Le service vérifie que le code fournisseur reste unique et qu'un fournisseur déjà relié à des médicaments n'est pas supprimé à la légère.

5.4.7. Étape 5. Écrire le repository



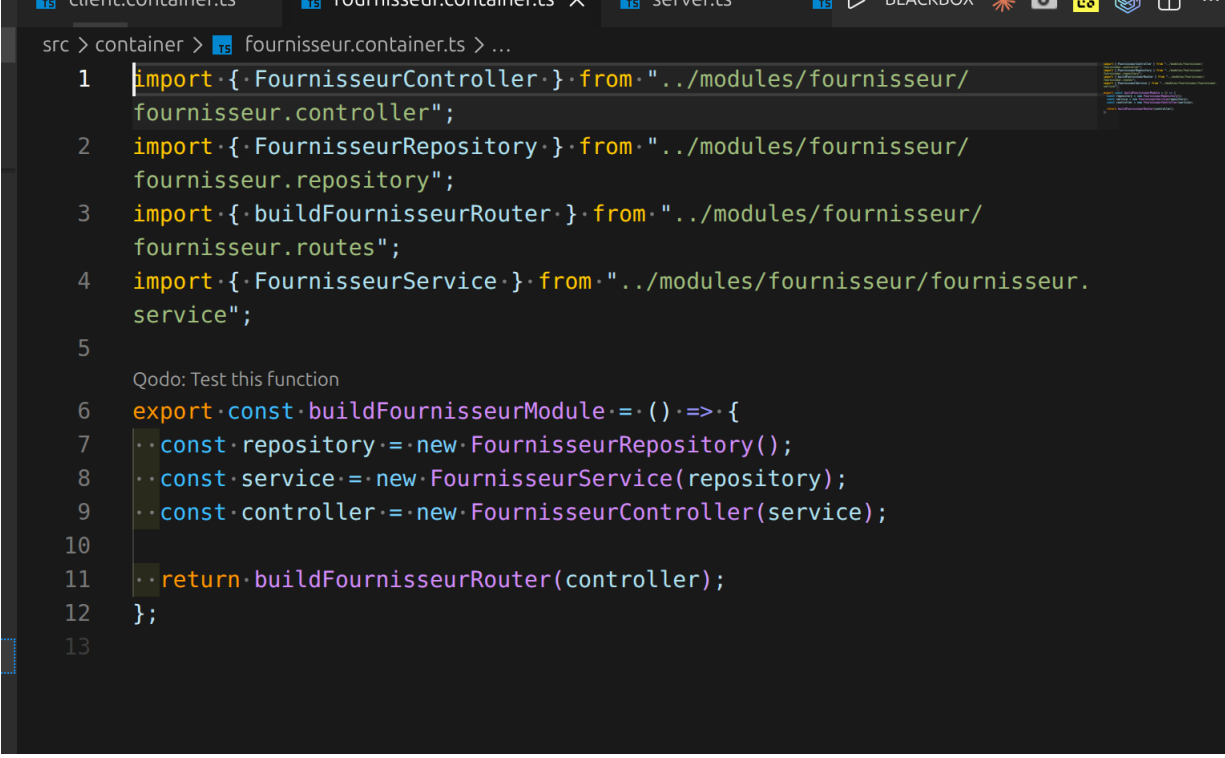
```
src > modules > fournisseur > fournisseur.repository.ts > ...
1  import { DEFAULT_ORDER_BY_CREATED_AT_DESC } from "../../common/
   constants";
2  import { db } from "../../config/db";
3  import { BaseRepository } from "../../infrastructure/repositories/base.
   repository";
4  import { CreateFournisseurDTO } from "../dto/CreateFournisseurDTO";
5  import { UpdateFournisseurDTO } from "../dto/UpdateFournisseurDTO";
6  import { FournisseurRecord, IFournisseurRepository } from "../
   interfaces/IFournisseurRepository";
7
   Qodo: Test this class
8  export class FournisseurRepository
9  .. extends BaseRepository<typeof db.fournisseur>
10 .. implements IFournisseurRepository
11 {
12 .. constructor() {
13 .. .. super(db.fournisseur);
14 .. }
15
   Mysti: Explain | Mysti: Refactor | Mysti: Write Tests | Qodo: Test this method
16 .. async list(): Promise<FournisseurRecord[]> {
```

Figure 5.25: Capture réelle du repository du module fournisseur

Le repository liste, recherche, crée, met à jour et supprime les fournisseurs avec Prisma, sans prendre de décision métier autonome.

5.4.8. Étape 6. Assembler le module puis le raccorder au backend

Le container fournisseur relie maintenant les briques déjà construites et fournit un routeur prêt à être branché dans l'application.

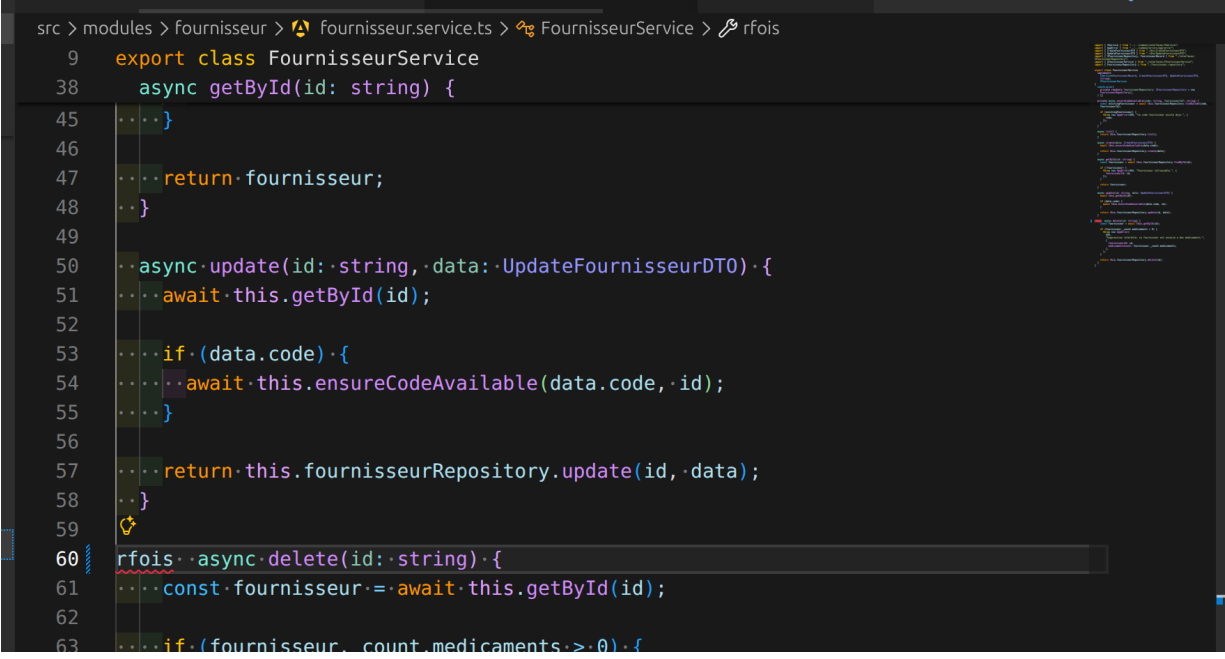


```

src > container > fournisseur.container.ts > ...
1  import { FournisseurController } from "../modules/fournisseur/
    fournisseur.controller";
2  import { FournisseurRepository } from "../modules/fournisseur/
    fournisseur.repository";
3  import { buildFournisseurRouter } from "../modules/fournisseur/
    fournisseur.routes";
4  import { FournisseurService } from "../modules/fournisseur/fournisseur.
    service";
5
    Qodo: Test this function
6  export const buildFournisseurModule = () => {
7    ..const repository = new FournisseurRepository();
8    ..const service = new FournisseurService(repository);
9    ..const controller = new FournisseurController(service);
10
11    ..return buildFournisseurRouter(controller);
12  };
13

```

Figure 5.26: Capture réelle de l'assemblage du module fournisseur



```

src > modules > fournisseur > fournisseur.service.ts > FournisseurService > rfois
9  export class FournisseurService
38  async getById(id: string) {
45  ...}
46
47  ..return fournisseur;
48  ..}
49
50  ..async update(id: string, data: UpdateFournisseurDTO) {
51  ..await this.getById(id);
52
53  ..if (data.code) {
54  ..await this.ensureCodeAvailable(data.code, id);
55  ..}
56
57  ..return this.fournisseurRepository.update(id, data);
58  ..}
59
60  rfois..async delete(id: string) {
61  ..const fournisseur = await this.getById(id);
62
63  ..if (fournisseur.count.medicaments > 0) {

```

Figure 5.27: Capture réelle du module fournisseurs

Cette étape matérialise la bascule entre “le code existe” et “la fonctionnalité est réellement disponible”.

5.4.9. Étape 7. Vérifier le module fournisseurs dans Swagger

Une fois le module raccordé, Swagger permet de tester l'entrée principale de création fournisseur avec une requête réelle.

The image shows a Swagger UI interface for the `/fournisseurs` endpoint. The header indicates the POST method and the purpose 'Croire un fournisseur'. The 'Paramètres' section is empty. The 'Corps de la requête' section is marked as 'required' and has a dropdown set to 'application/json'. An 'Exemple de valeur' section shows a JSON object:

```
{
  "code": "F0U-001",
  "nom": "Laborex",
  "adresse": "Takar",
  "telephone": "781112233",
  "email": "contact@laborex.sn"
}
```

. The 'Réponses' section shows a table with one response: status 201, description 'Fournisseur cree.', and a link 'Aucun lien'. Below the table, there is a 'Type de média' dropdown set to 'application/json' and a note 'En-tête des commandes Accept.'.

Figure 5.28: Capture Swagger de création d'un fournisseur

Le contrôle attendu est simple:

- créer un fournisseur valide;
- relire le fournisseur créé;
- vérifier qu'un code déjà utilisé ou une suppression interdite remonte un message explicite.

L'idée centrale de ce module est simple: un fournisseur n'est pas un enregistrement décoratif. Il fait partie de la chaîne réelle d'approvisionnement.

5.5. Module médicaments

5.5.1. Ce que ce module doit faire

Le module `medicaments` sert à gérer le stock vendable. Il doit enregistrer un produit, vérifier qu'il est correctement défini et s'assurer qu'il reste cohérent avec le fournisseur, le prix, le stock et la date d'expiration.

5.5.2. Dépendances principalement utilisées

Les dépendances principales sont:

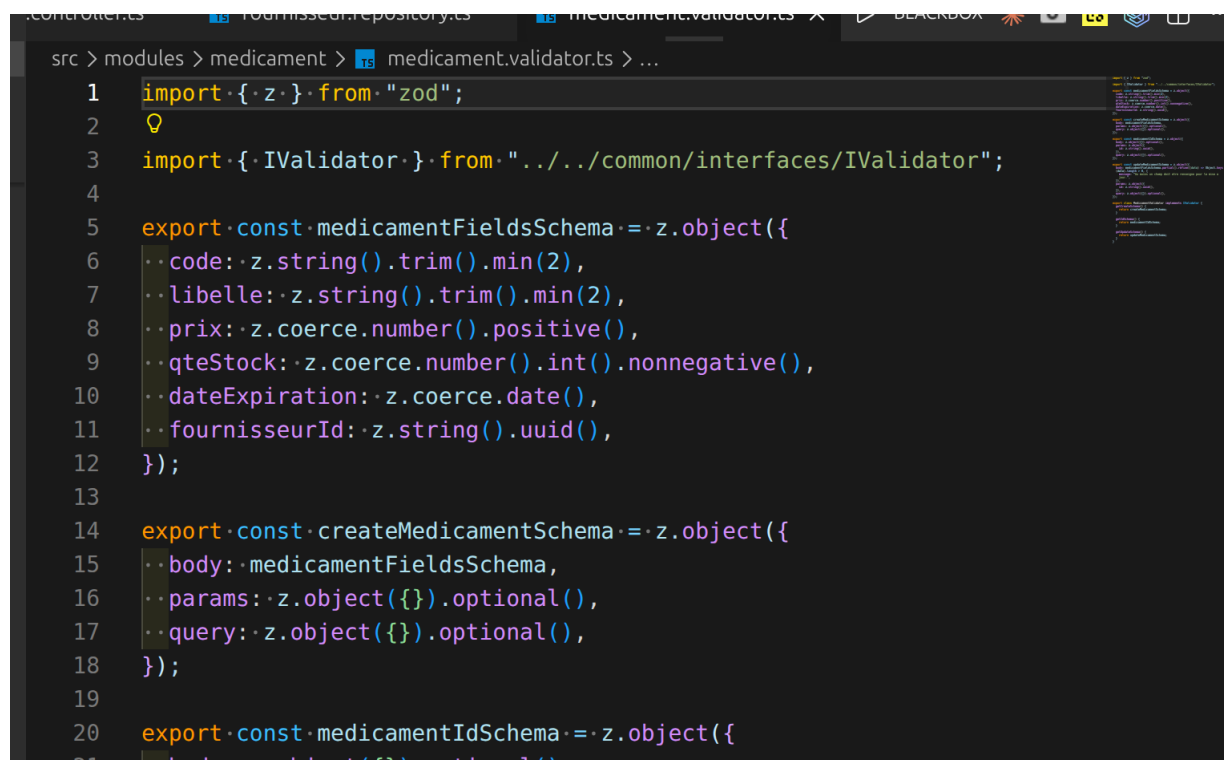
- `express`;
- `zod`;
- `@prisma/client`.

Commande d'installation minimale: `npm install express zod @prisma/client`.

Si les modèles Prisma changent, il faut aussi régénérer le client via `npx prisma generate`.

5.5.3. Étape 1. Définir les données d'entrée

Le validateur montre immédiatement la nature plus exigeante de ce module: prix, quantité, date d'expiration et identifiant fournisseur doivent tous être corrects.



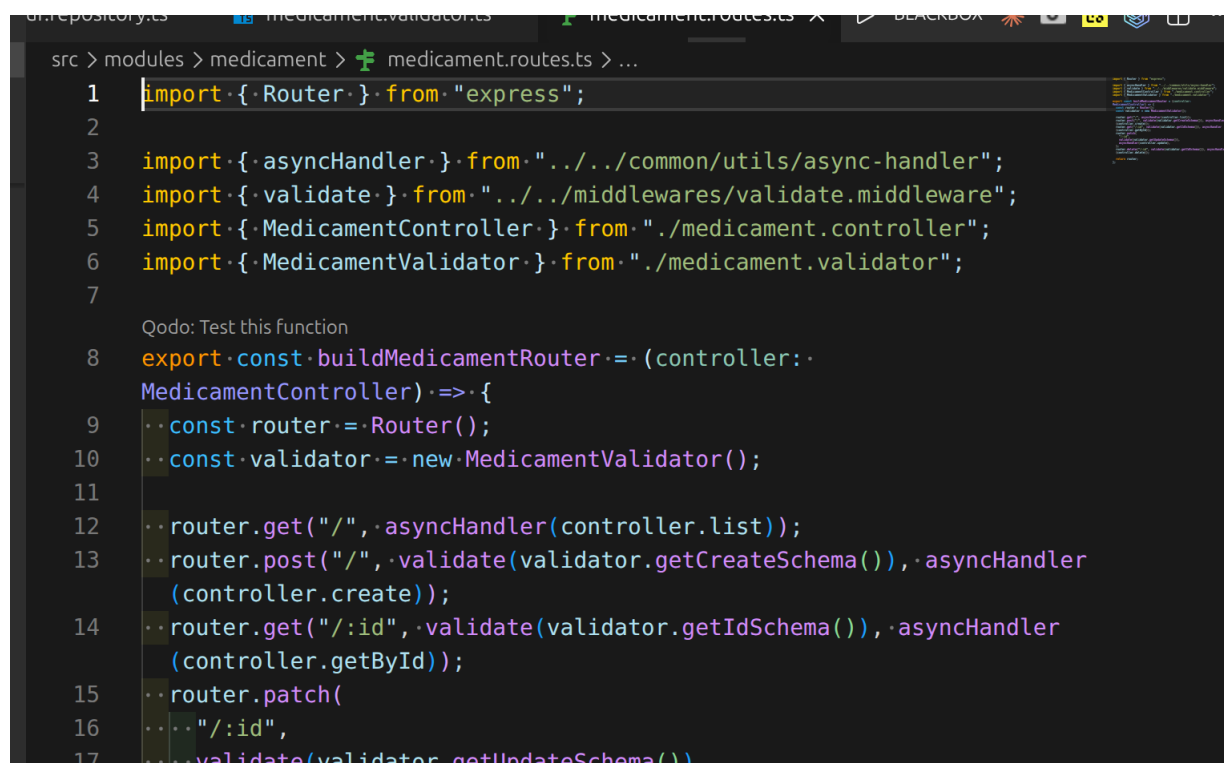
```

src > modules > medicament > medicament.validator.ts > ...
1  import { z } from "zod";
2
3  import { IValidator } from "../../common/interfaces/IValidator";
4
5  export const medicamentFieldsSchema = z.object({
6    ..code: z.string().trim().min(2),
7    ..libelle: z.string().trim().min(2),
8    ..prix: z.coerce.number().positive(),
9    ..qteStock: z.coerce.number().int().nonnegative(),
10   ..dateExpiration: z.coerce.date(),
11   ..fournisseurId: z.string().uuid(),
12 });
13
14 export const createMedicamentSchema = z.object({
15   ..body: medicamentFieldsSchema,
16   ..params: z.object({}).optional(),
17   ..query: z.object({}).optional(),
18 });
19
20 export const medicamentIdSchema = z.object({
21   ..body: z.object({}).optional(),

```

Figure 5.29: Capture réelle du validateur du module médicament

5.5.4. Étape 2. Déclarer les routes



```

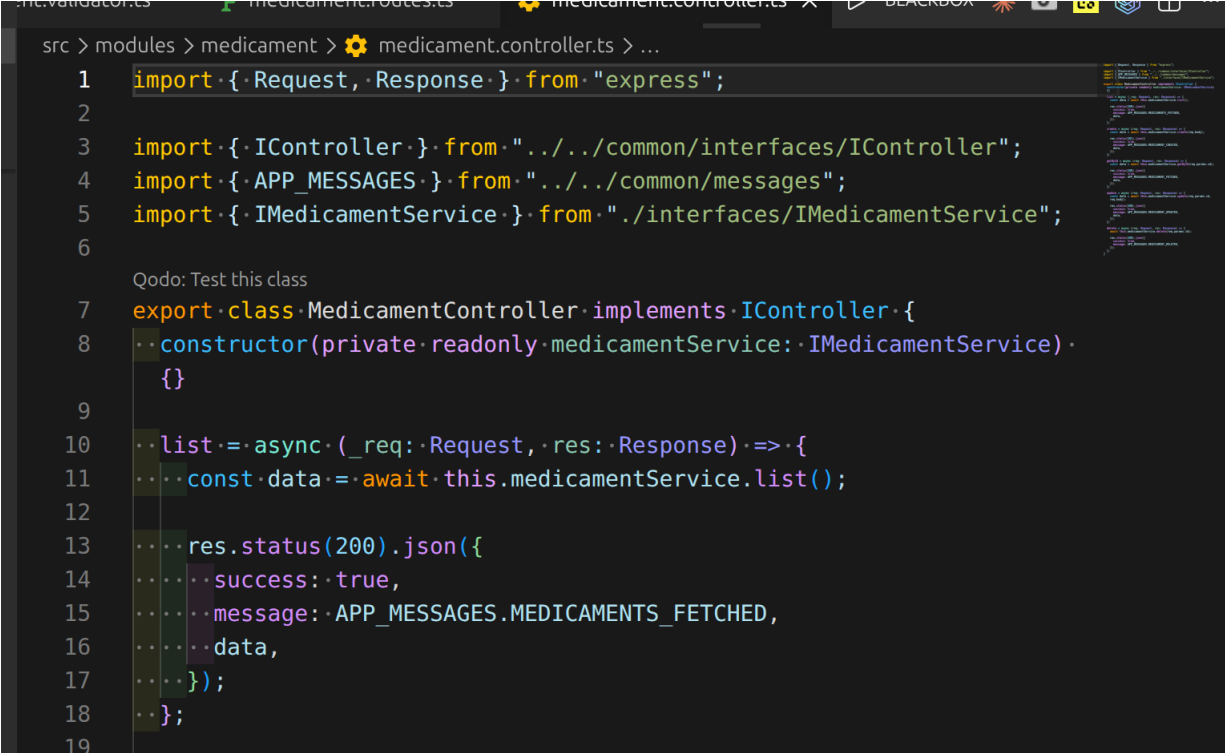
src > modules > medicament > medicament.routes.ts > ...
1  import { Router } from "express";
2
3  import { asyncHandler } from "../../common/utils/async-handler";
4  import { validate } from "../../middlewares/validate.middleware";
5  import { MedicamentController } from "../medicament.controller";
6  import { MedicamentValidator } from "../medicament.validator";
7
8  Qodo: Test this function
9  export const buildMedicamentRouter = (controller: MedicamentController) => {
10   ..const router = Router();
11   ..const validator = new MedicamentValidator();
12   ..router.get("/", asyncHandler(controller.list));
13   ..router.post("/", validate(validator.getCreateSchema()), asyncHandler
14     (controller.create));
15   ..router.get("/:id", validate(validator.getIdSchema()), asyncHandler
16     (controller.getById));
17   ..router.patch(
18     .."/:id",
19     ..validate(validator.getUpdateSchema()),

```

Figure 5.30: Capture réelle des routes du module médicament

Le module conserve la même logique d'entrée que les autres modules, ce qui maintient la cohérence globale du backend.

5.5.5. Étape 3. Construire le contrôleur



```

src > modules > medicament > medicament.controller.ts > ...
1  import { Request, Response } from "express";
2
3  import { IController } from "../../common/interfaces/IController";
4  import { APP_MESSAGES } from "../../common/messages";
5  import { IMedicamentService } from "./interfaces/IMedicamentService";
6
7  Qodo: Test this class
8  export class MedicamentController implements IController {
9      .. constructor(private readonly medicamentService: IMedicamentService) {
10         {}
11     }
12
13     .. list = async (_req: Request, _res: Response) => {
14         .. const data = await this.medicamentService.list();
15
16         .. res.status(200).json({
17             .. success: true,
18             .. message: APP_MESSAGES.MEDICAMENTS_FETCHED,
19             .. data,
20         });
21     };
22 }

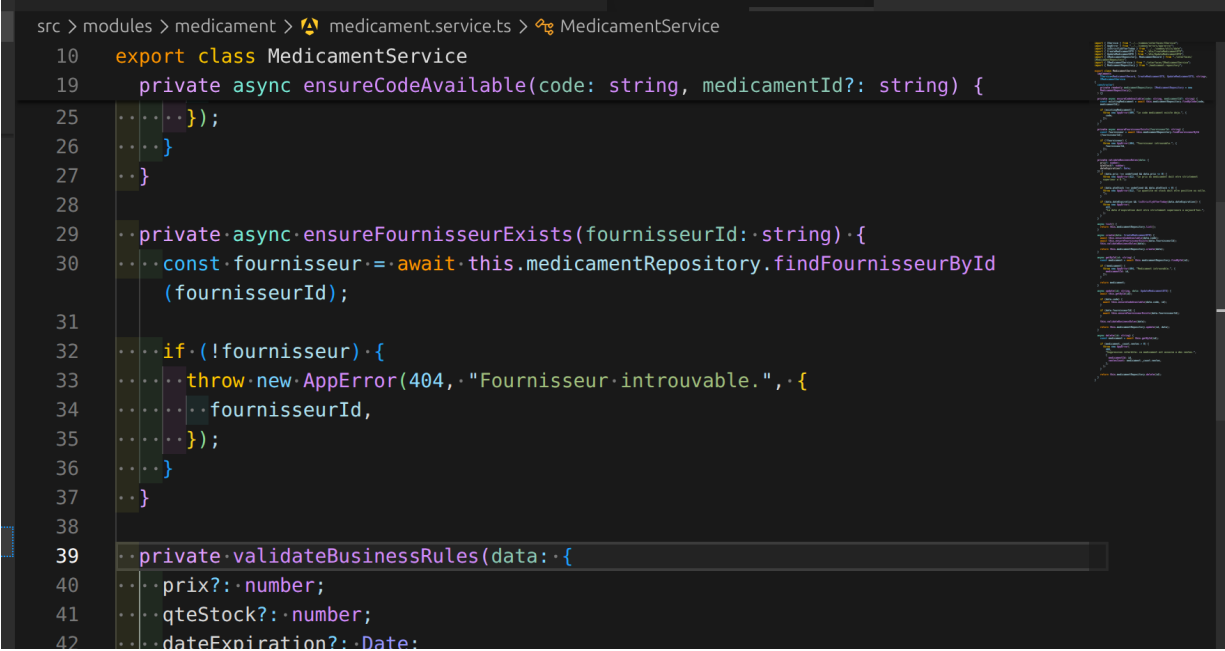
```

Figure 5.31: Capture réelle du controller du module médicament

Le contrôleur garde un rôle simple: renvoyer une réponse HTTP propre après exécution du service.

5.5.6. Étape 4. Écrire le service

Le service du module médicaments porte des contrôles plus riches que ceux des modules précédents.

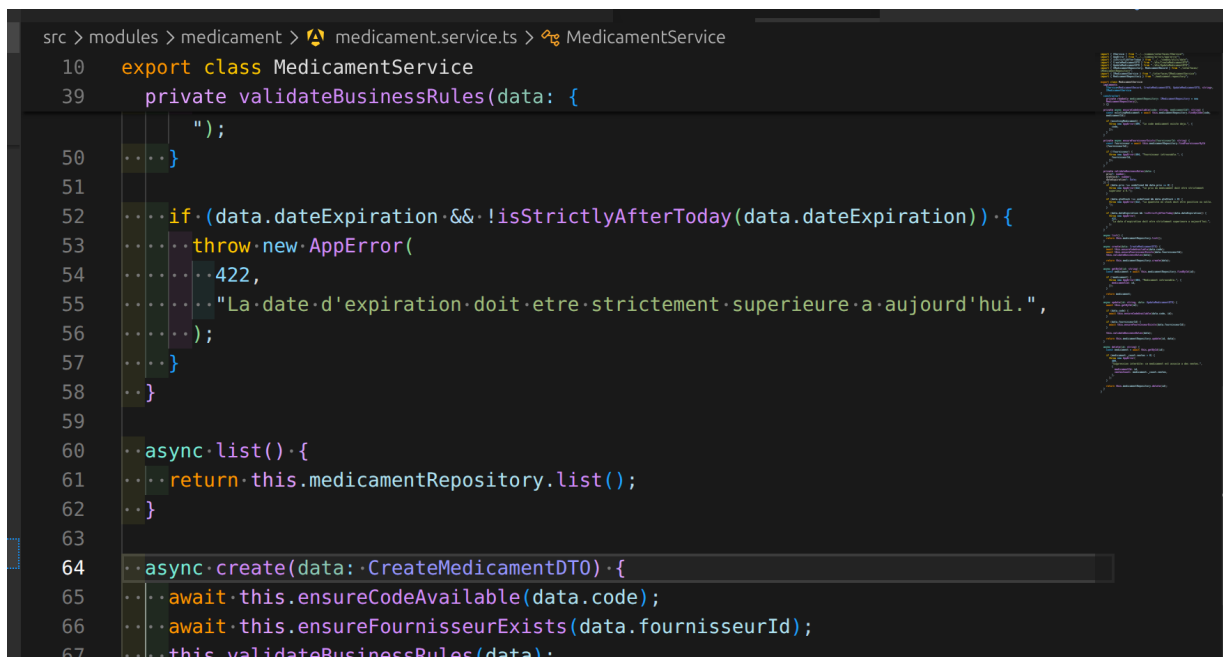


```

src > modules > medicament > medicament.service.ts > MedicamentService
10 export class MedicamentService
11     .. private async ensureCodeAvailable(code: string, medicamentId?: string) {
12         .. ..
13     }
14
15     .. private async ensureFournisseurExists(fournisseurId: string) {
16         .. const fournisseur = await this.medicamentRepository.findFournisseurById
17         .. (fournisseurId);
18
19         .. if (!fournisseur) {
20             .. throw new AppError(404, "Fournisseur introuvable.", {
21                 .. fournisseurId,
22             });
23         }
24     }
25
26     .. private validateBusinessRules(data: {
27         .. prix?: number;
28         .. qteStock?: number;
29         .. dateExpiration?: Date;
30     }) {
31         ..
32     }
33 }

```

Figure 5.32: Capture réelle des règles métier du module médicaments



```

src > modules > medicament > medication.service.ts > MedicamentService
10  export class MedicamentService
39  private validateBusinessRules(data: {
    ...
    });
50  ...
51  ...
52  ...if (data.dateExpiration && !isStrictlyAfterToday(data.dateExpiration)) {
53  ...  throw new AppError(
54  ...    422,
55  ...    "La date d'expiration doit etre strictement superieure a aujourd'hui.",
56  ...  );
57  ...}
58  ...}
59  ...
60  async list() {
61  ...  return this.medicamentRepository.list();
62  ...}
63  ...
64  async create(data: CreateMedicamentDTO) {
65  ...  await this.ensureCodeAvailable(data.code);
66  ...  await this.ensureFournisseurExists(data.fournisseurId);
67  ...  this.validateBusinessRules(data);

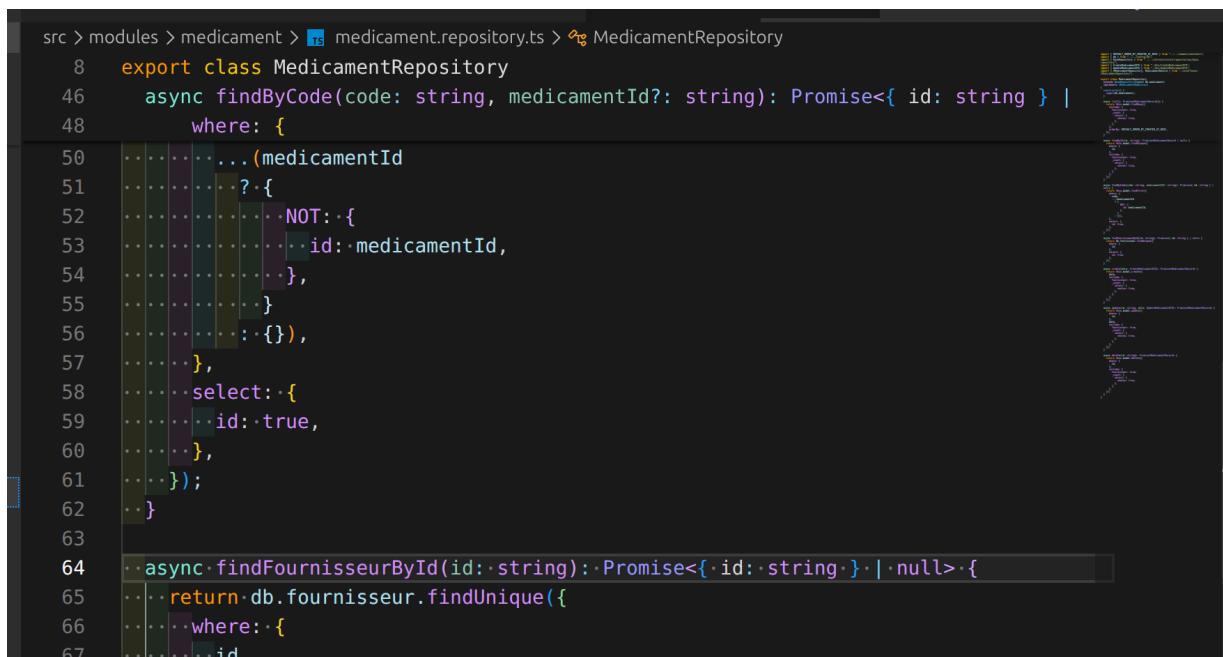
```

Figure 5.33: Capture réelle de la création d'un médicament

Les vérifications essentielles sont:

- unicité du code médicament;
- existence réelle du fournisseur;
- validité du prix;
- validité de la quantité en stock;
- cohérence de la date d'expiration.

5.5.7. Étape 5. Écrire le repository



```

src > modules > medicament > medication.repository.ts > MedicamentRepository
8  export class MedicamentRepository
46  async findByCode(code: string, medicationId?: string): Promise<{ id: string } |
48  where: {
50  ...  ... (medicationId
51  ...  ? {
52  ...  NOT: {
53  ...    id: medicationId,
54  ...  },
55  ...  },
56  ...  : {}),
57  ...  },
58  select: {
59  ...  id: true,
60  ...  },
61  ...  });
62  ...}
63  ...
64  async findFournisseurById(id: string): Promise<{ id: string } | null> {
65  ...  return db.fournisseur.findUnique({
66  ...    where: {
67  ...      id:

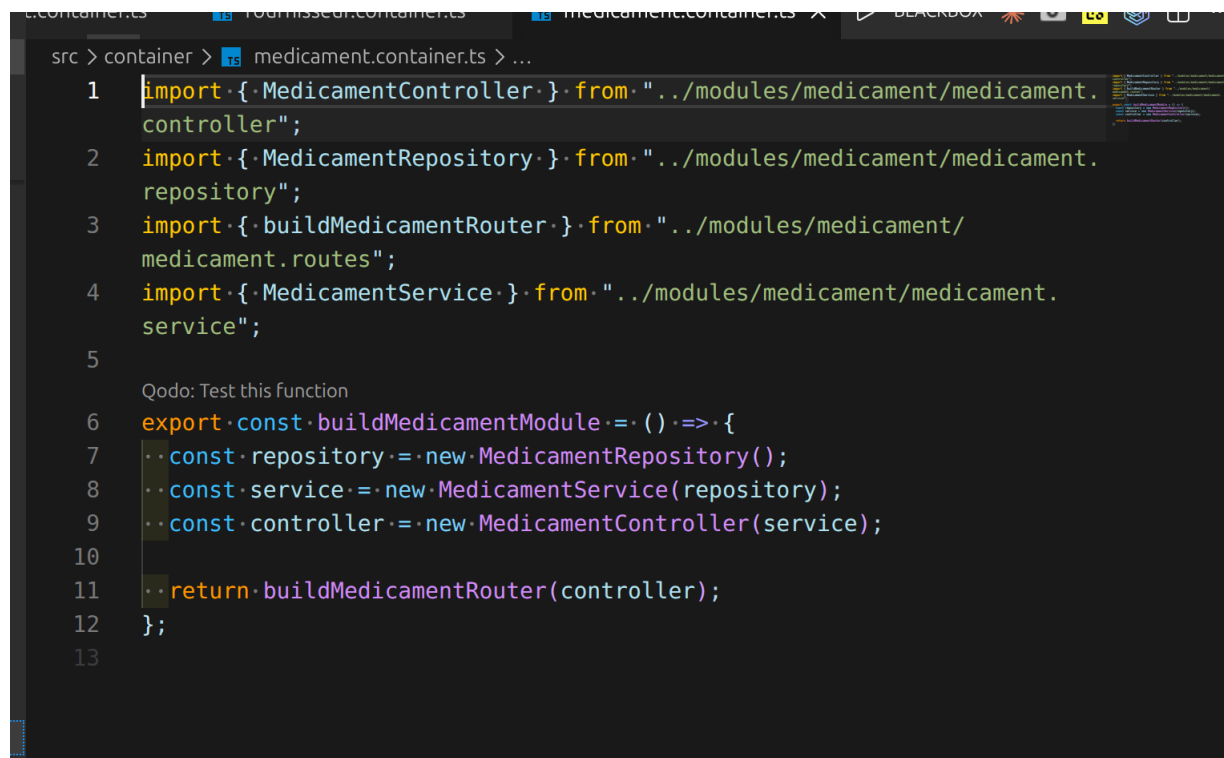
```

Figure 5.34: Capture réelle de la recherche du fournisseur associé

Le repository du module médicaments communique avec Prisma pour lire et écrire les produits, mais les décisions métier continuent à vivre dans le service.

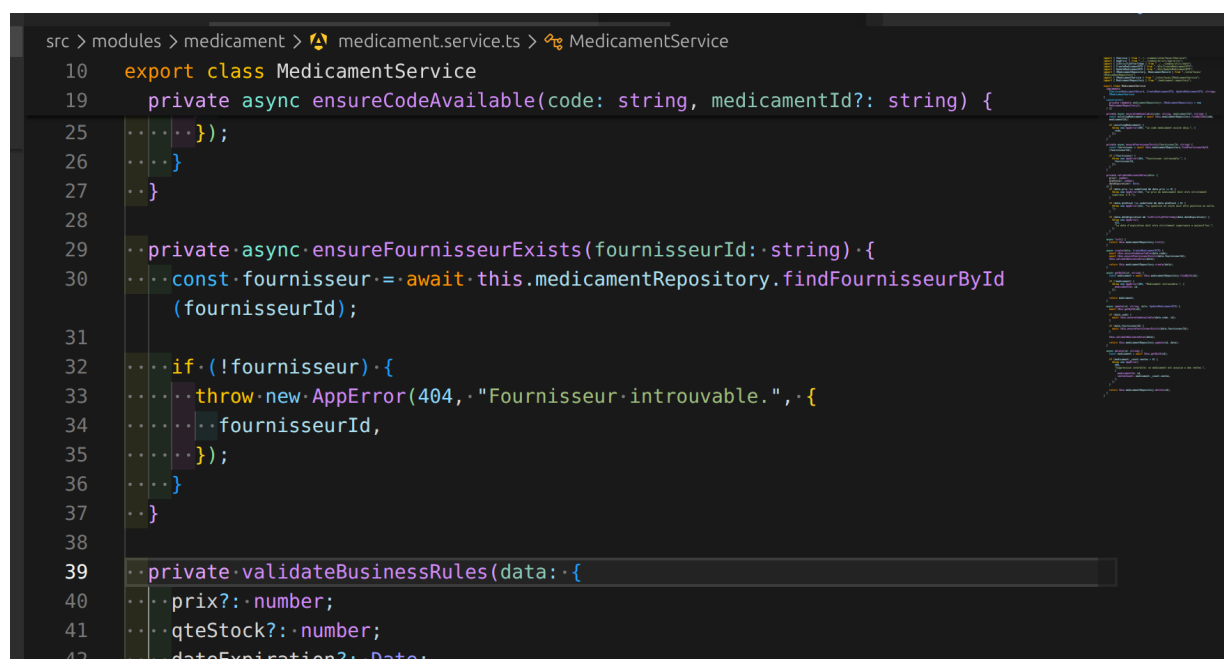
5.5.8. Étape 6. Assembler le module puis le raccorder au backend

Le container du module médicaments assemble les classes déjà écrites et retourne le routeur final du module.



```
src > container > medicament.container.ts > ...
1  import { MedicamentController } from "../modules/medicament/medicament.controller";
2  import { MedicamentRepository } from "../modules/medicament/medicament.repository";
3  import { buildMedicamentRouter } from "../modules/medicament/medicament.routes";
4  import { MedicamentService } from "../modules/medicament/medicament.service";
5
6  Qodo: Test this function
7  export const buildMedicamentModule = () => {
8    const repository = new MedicamentRepository();
9    const service = new MedicamentService(repository);
10   const controller = new MedicamentController(service);
11   return buildMedicamentRouter(controller);
12 };
13
```

Figure 5.35: Capture réelle de l'assemblage du module médicament



```
src > modules > medicament > medicament.service.ts > MedicamentService
10  export class MedicamentService
11  {
12    private async ensureCodeAvailable(code: string, medicamentId?: string) {
13      ...
14    }
15    ...
16  }
17
18  private async ensureFournisseurExists(fournisseurId: string) {
19    const fournisseur = await this.medicamentRepository.findFournisseurById(fournisseurId);
20
21    if (!fournisseur) {
22      throw new AppError(404, "Fournisseur introuvable.", {
23        fournisseurId,
24      });
25    }
26  }
27
28  private validateBusinessRules(data: {
29    prix?: number;
30    qteStock?: number;
31    dateExpiration?: Date;
32  }) {
33    ...
34  }
35}
```

Figure 5.36: Capture réelle du module médicaments

Tant que cette étape n'est pas faite, le backend ne peut pas encore exposer proprement les routes du stock.

5.5.9. Étape 7. Vérifier le module médicaments dans Swagger

Une fois branché, le module doit être testé avec une requête réelle de création de médicament.

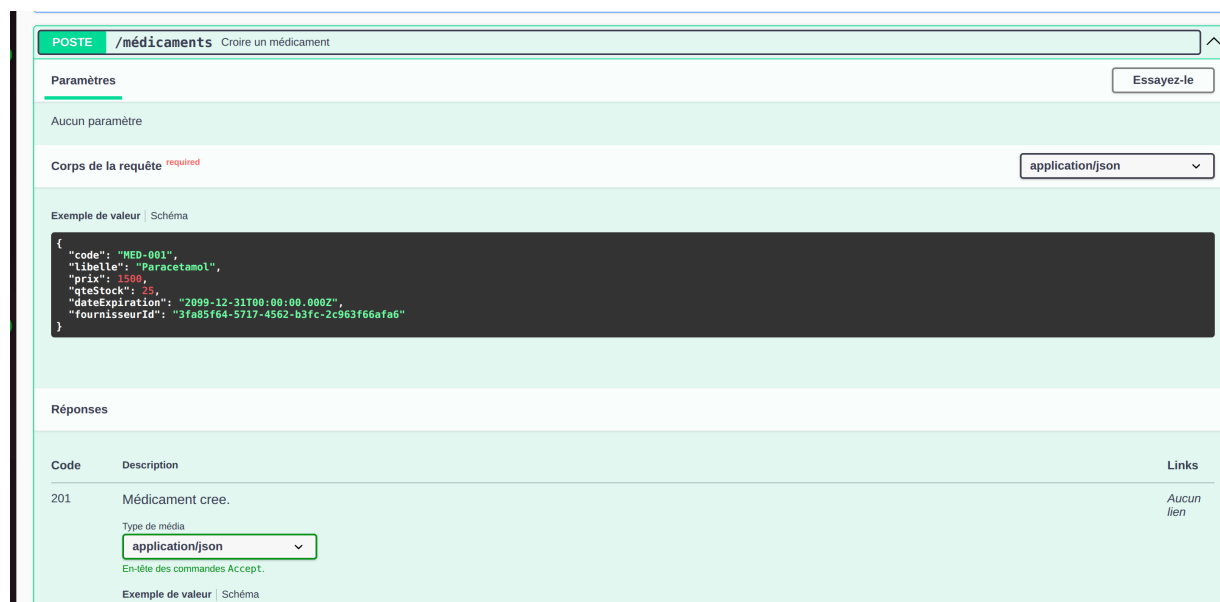


Figure 5.37: Capture Swagger de création d'un médicament

Le lecteur doit vérifier ici:

- qu'un médicament valide peut être créé;
- qu'un fournisseur inexistant est refusé;
- qu'un prix, une quantité ou une date incohérente déclenche une erreur compréhensible.

Ce module est important, car il prépare directement le module **ventes**. Si les médicaments sont mal définis, les ventes ne peuvent plus être sûres.

5.6. Module ventes

5.6.1. Ce que ce module doit faire

Le module **ventes** est le module le plus critique. Il doit:

- recevoir un client et un médicament;
- contrôler la quantité demandée;
- vérifier le stock;
- vérifier la date d'expiration;
- calculer le montant;
- créer la vente et mettre à jour le stock dans une même opération cohérente.

5.6.2. Dépendances principalement utilisées

Les dépendances principales sont:

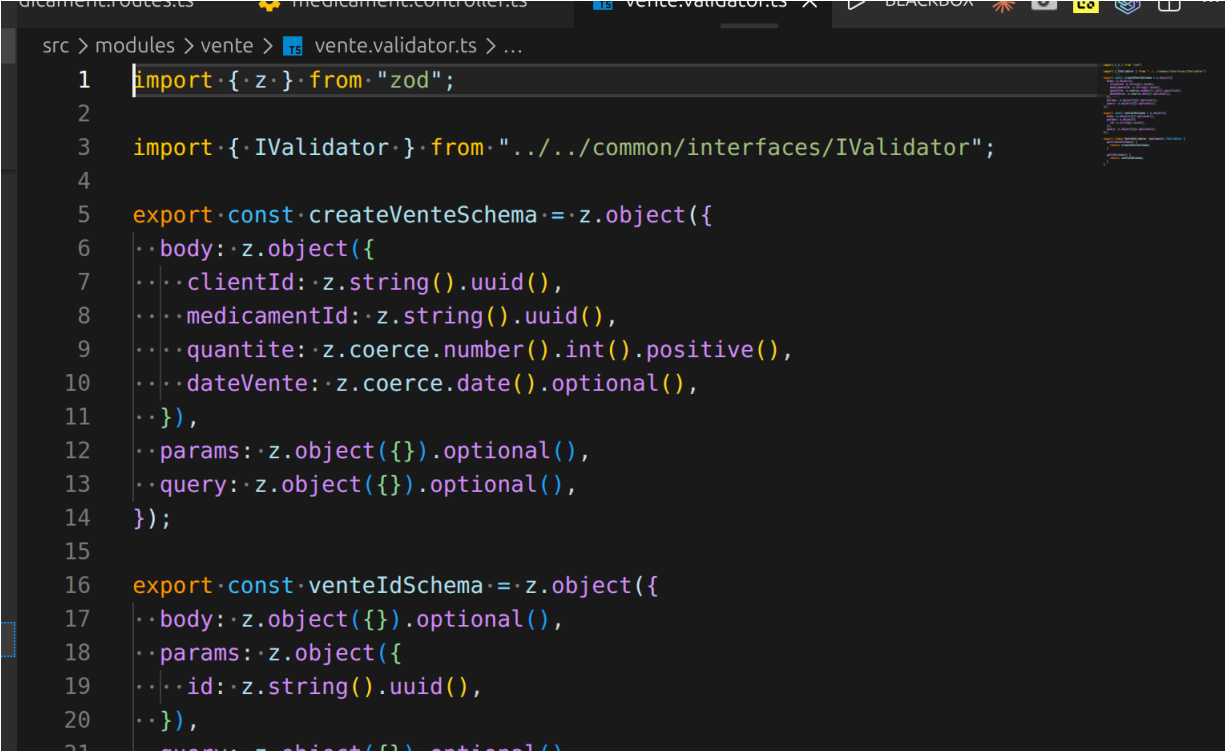
- express;
- zod;
- @prisma/client.

Commande d'installation minimale: `npm install express zod @prisma/client`.

Comme la vente dépend du modèle de données et des transactions Prisma, `npx prisma generate` reste une étape utile après modification du schéma.

5.6.3. Étape 1. Définir les données d'entrée

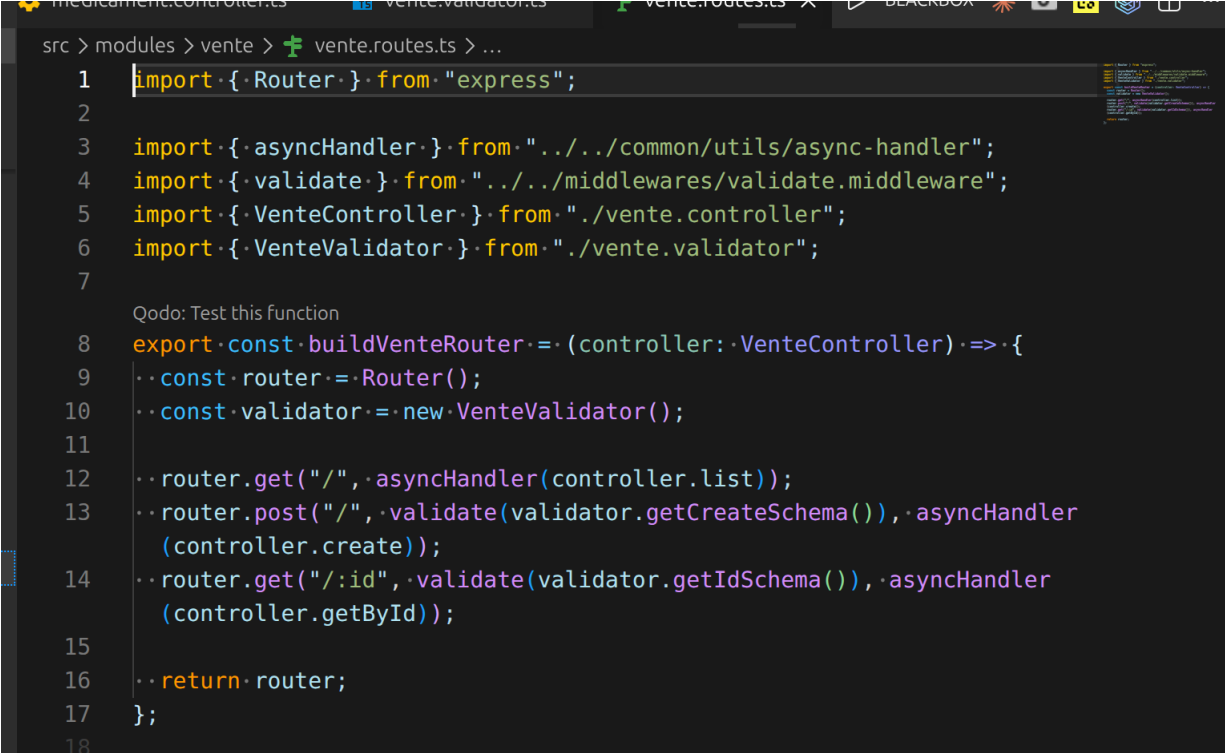
Le validateur du module ventes impose tout de suite les identifiants et la quantité.



```
src > modules > vente > vente.validator.ts > ...
1  import { z } from "zod";
2
3  import { IValidator } from "../../common/interfaces/IValidator";
4
5  export const createVenteSchema = z.object({
6    body: z.object({
7      clientId: z.string().uuid(),
8      medicamentId: z.string().uuid(),
9      quantite: z.coerce.number().int().positive(),
10     dateVente: z.coerce.date().optional(),
11   }),
12   params: z.object({}).optional(),
13   query: z.object({}).optional(),
14 });
15
16 export const venteIdSchema = z.object({
17   body: z.object({}).optional(),
18   params: z.object({
19     id: z.string().uuid(),
20   }),
21   query: z.object({}).optional()
```

Figure 5.38: Capture réelle du validateur du module vente

5.6.4. Étape 2. Déclarer les routes

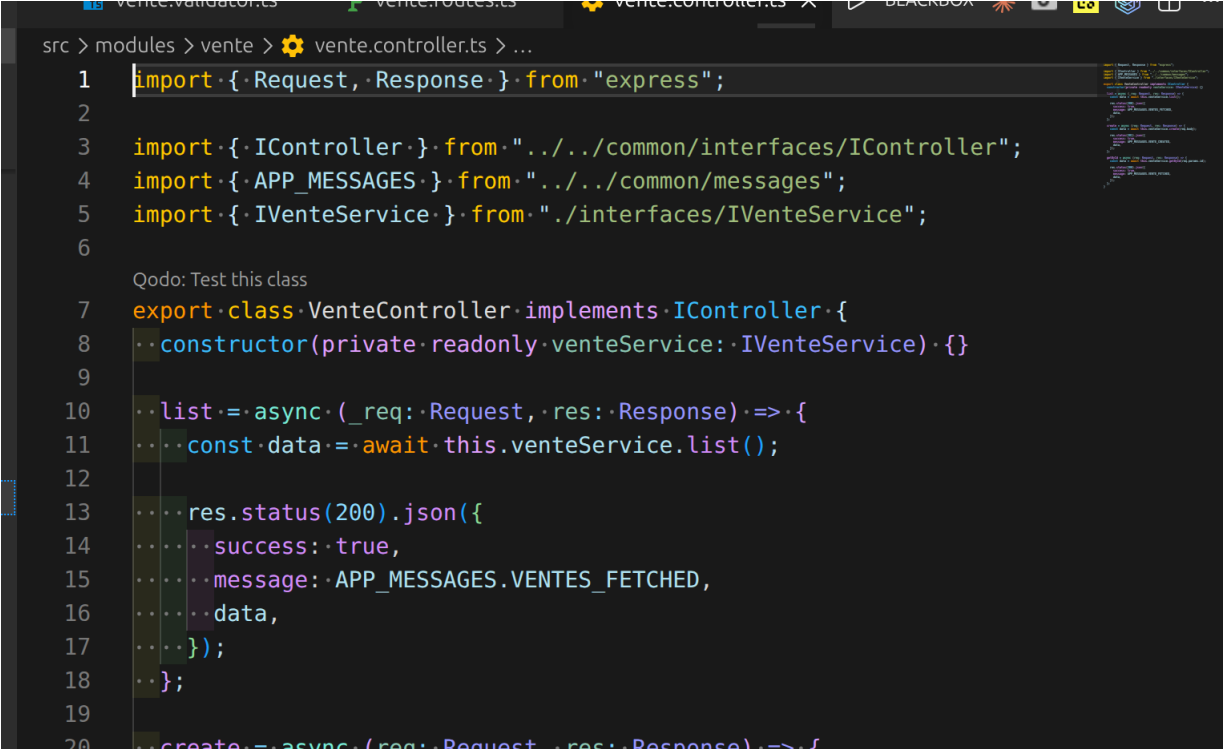


```
src > modules > vente > vente.routes.ts > ...
1  import { Router } from "express";
2
3  import { AsyncHandler } from "../../common/utils/async-handler";
4  import { validate } from "../../middlewares/validate.middleware";
5  import { VenteController } from "../vente.controller";
6  import { VenteValidator } from "../vente.validator";
7
8  Qodo: Test this function
9  export const buildVenteRouter = (controller: VenteController) => {
10     const router = Router();
11     const validator = new VenteValidator();
12     router.get("/", AsyncHandler(controller.list));
13     router.post("/", validate(validator.getCreateSchema()), AsyncHandler(
14         controller.create));
15     router.get("/:id", validate(validator.getIdSchema()), AsyncHandler(
16         controller.getById));
17     return router;
18 }
```

Figure 5.39: Capture réelle des routes du module vente

Le routeur du module ventes est plus petit, mais cela ne signifie pas qu'il est plus simple. La vraie complexité se trouve dans le service.

5.6.5. Étape 3. Construire le contrôleur



```

src > modules > vente > vente.controller.ts > ...
1  import { Request, Response } from "express";
2
3  import { IController } from "../../common/interfaces/IController";
4  import { APP_MESSAGES } from "../../common/messages";
5  import { IVenteService } from "../interfaces/IVenteService";
6
7  Qodo: Test this class
8  export class VenteController implements IController {
9      constructor(private readonly venteService: IVenteService) {}
10
11     list = async (_req: Request, _res: Response) => {
12         const data = await this.venteService.list();
13         res.status(200).json({
14             success: true,
15             message: APP_MESSAGES.VENTES_FETCHED,
16             data,
17         });
18     };
19
20     create = async (req: Request, res: Response) => {

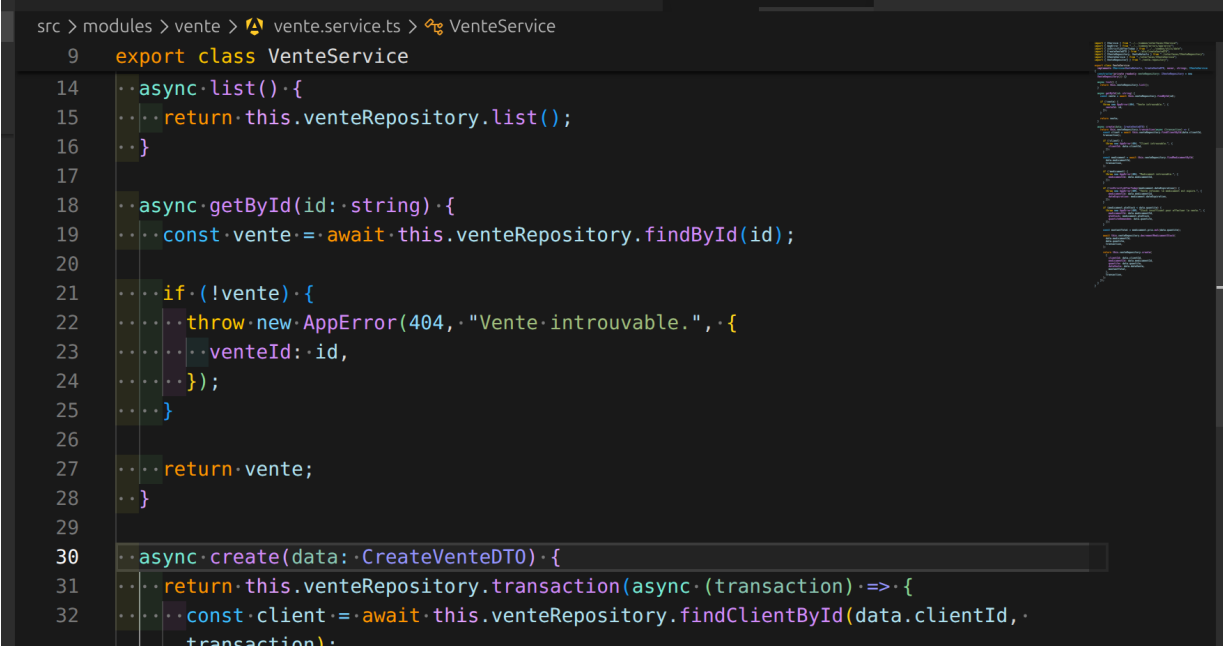
```

Figure 5.40: Capture réelle du controller du module vente

Le contrôleur reçoit le résultat du service et le traduit en réponse exploitable.

5.6.6. Étape 4. Écrire le service

Le service est ici le cœur de la décision métier.



```

src > modules > vente > vente.service.ts > VenteService
9  export class VenteService
14  .. async list() {
15  .. .. return this.venteRepository.list();
16  .. }
17
18  .. async getById(id: string) {
19  .. .. const vente = await this.venteRepository.findById(id);
20  ..
21  .. .. if (!vente) {
22  .. .. .. throw new AppError(404, "Vente introuvable.", {
23  .. .. .. .. venteId: id,
24  .. .. .. });
25  .. .. }
26
27  .. .. return vente;
28  .. }
29
30  .. async create(data: CreateVenteDTO) {
31  .. .. return this.venteRepository.transaction(async (transaction) => {
32  .. .. .. const client = await this.venteRepository.findClientById(data.clientId,
33  .. .. .. .. transaction);

```

Figure 5.41: Capture réelle de l'entrée de création d'une vente

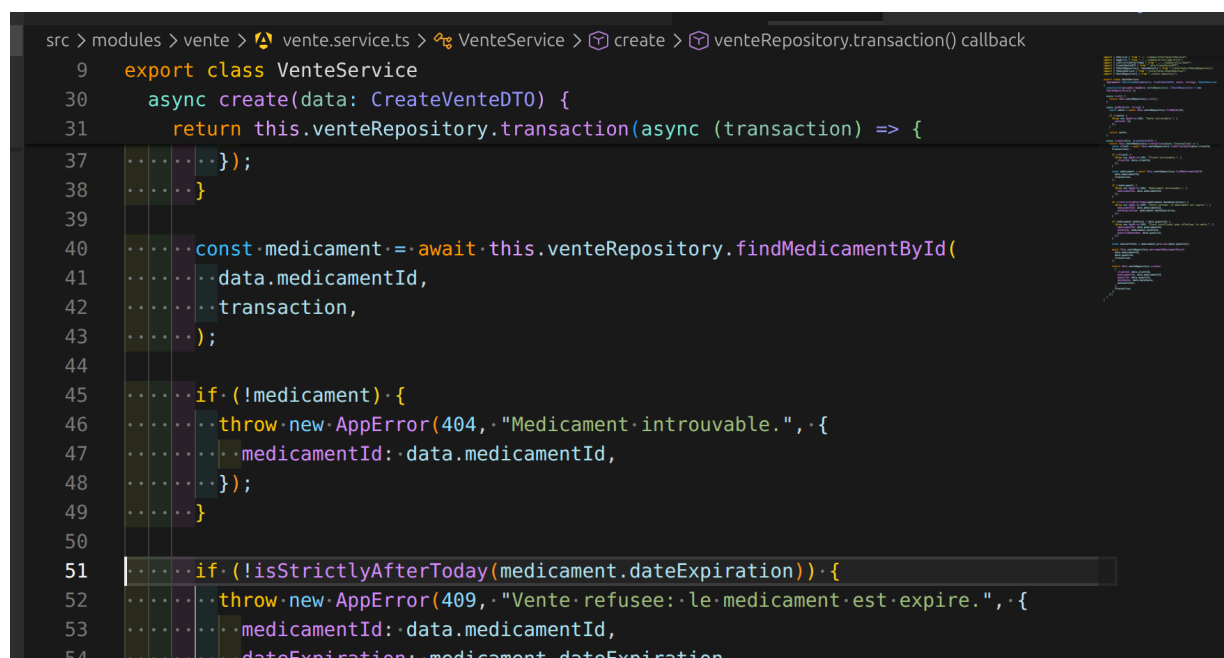


Figure 5.42: Capture réelle des contrôles de stock et d'expiration

Le service vérifie, dans cet ordre:

1. que le client existe;
2. que le médicament existe;
3. qu'il n'est pas expiré;
4. que le stock est suffisant;
5. que le montant total peut être calculé correctement.

5.6.7. Étape 5. Écrire le repository

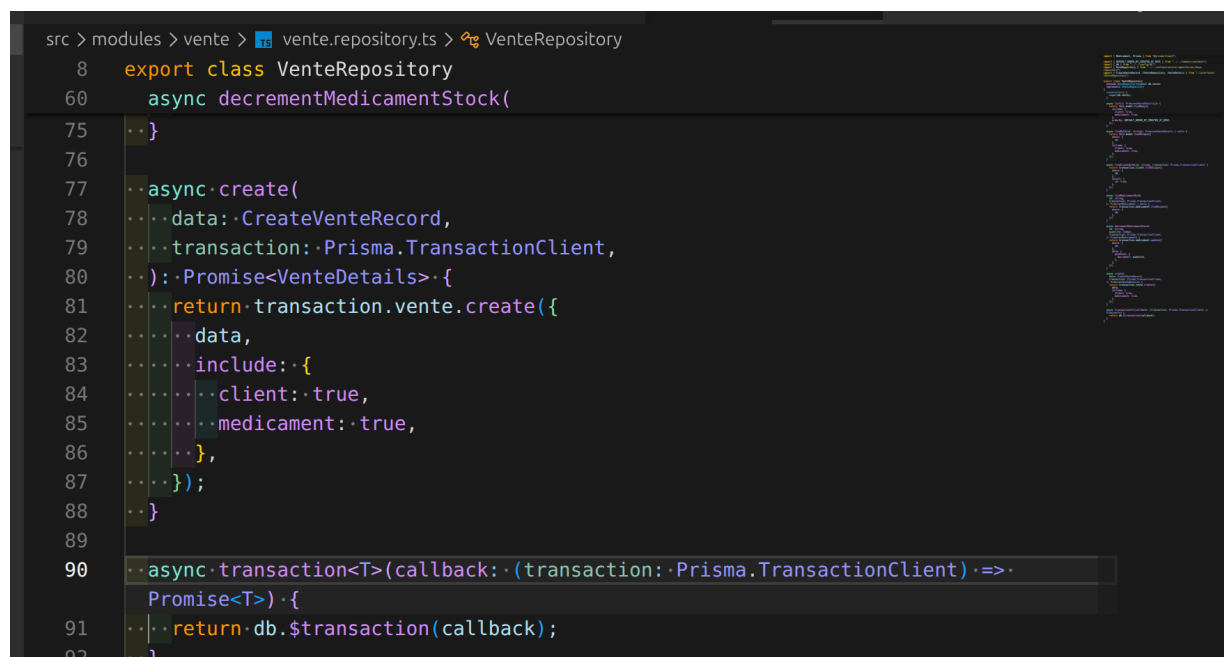


Figure 5.43: Capture réelle de la transaction Prisma du module ventes

La partie la plus importante est l'usage de `$transaction`. Cette instruction signifie que la création de la vente et la mise à jour du stock doivent réussir ensemble. Si l'une échoue, l'ensemble revient à l'état précédent.

5.6.8. Étape 6. Assembler le module puis le raccorder au backend

Le container ventes relie enfin les dernières briques et renvoie un routeur opérationnel.

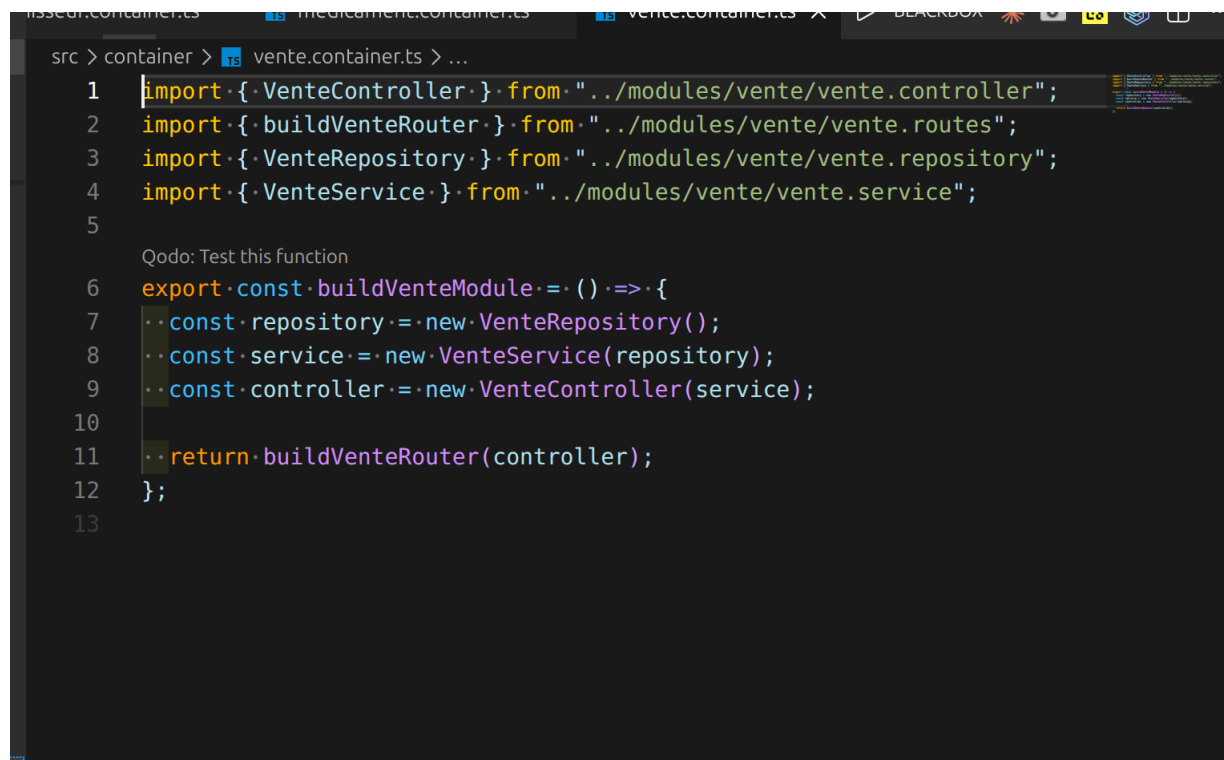


Figure 5.44: Capture réelle de l'assemblage du module ventes

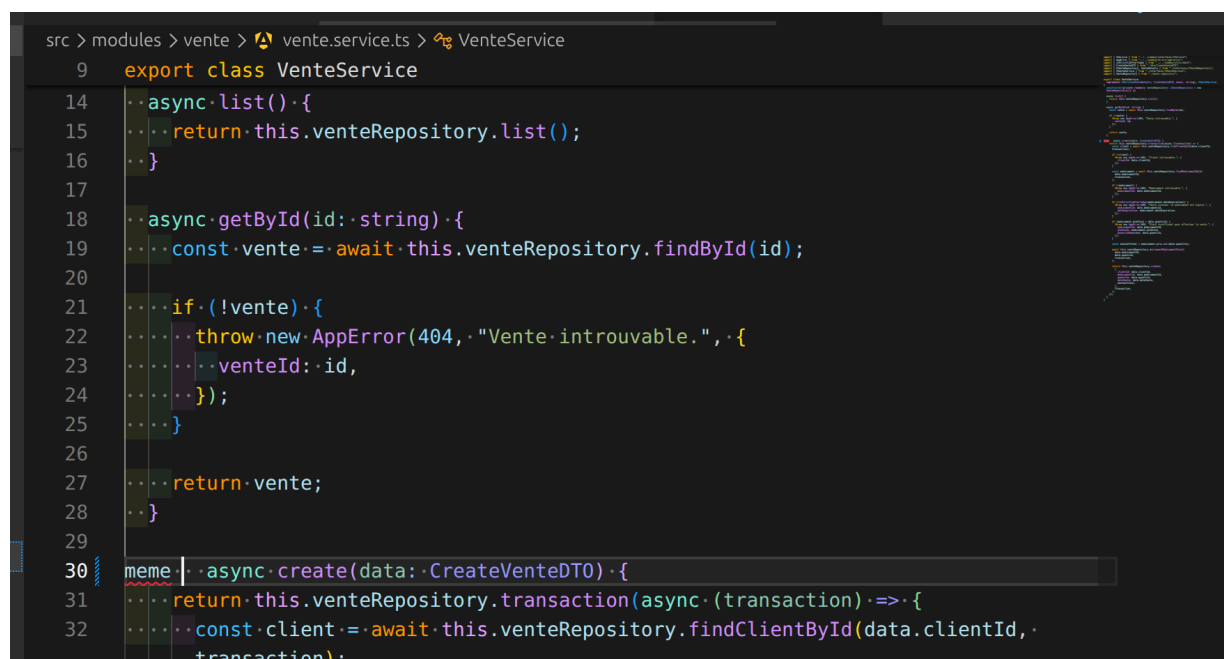


Figure 5.45: Capture réelle du module ventes

Quand cette étape est terminée, le backend est capable d'exposer le module le plus sensible de l'application.

5.6.9. Étape 7. Vérifier le module ventes dans Swagger

La vérification finale doit se faire avec une vraie entrée Swagger, car c'est elle qui prouve que la chaîne complète fonctionne jusqu'au contrat HTTP publié.

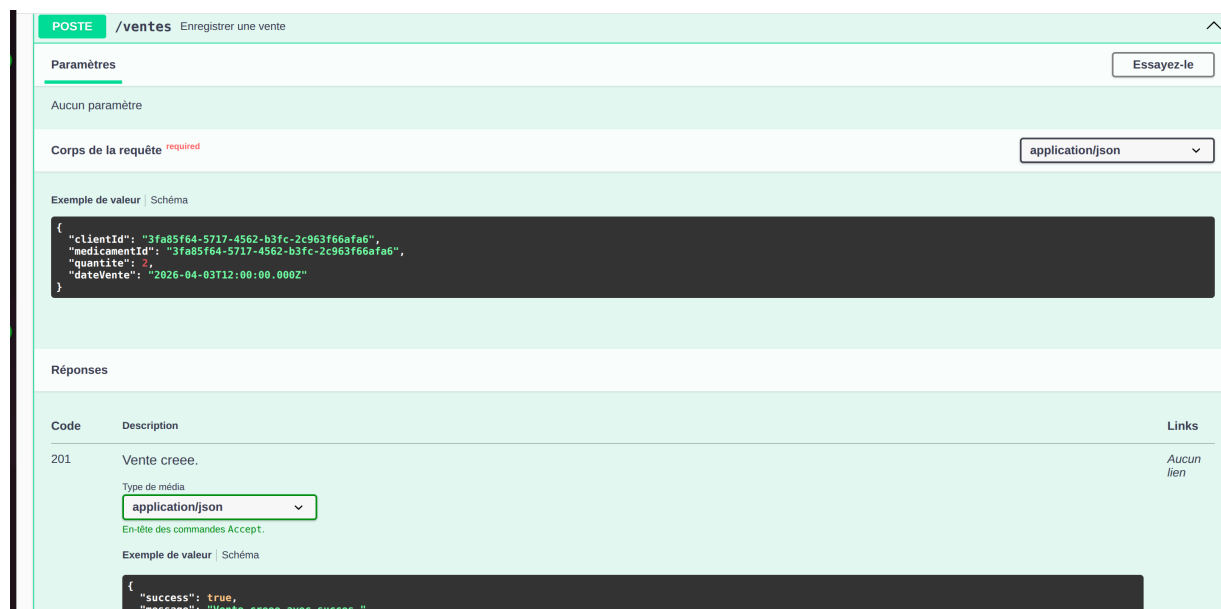


Figure 5.46: Capture Swagger de création d'une vente

Le test attendu suit cet ordre:

1. choisir un client existant;
2. choisir un médicament existant;
3. saisir une quantité réaliste;
4. envoyer la requête;
5. vérifier que la vente est créée et que le stock est décrémenté.

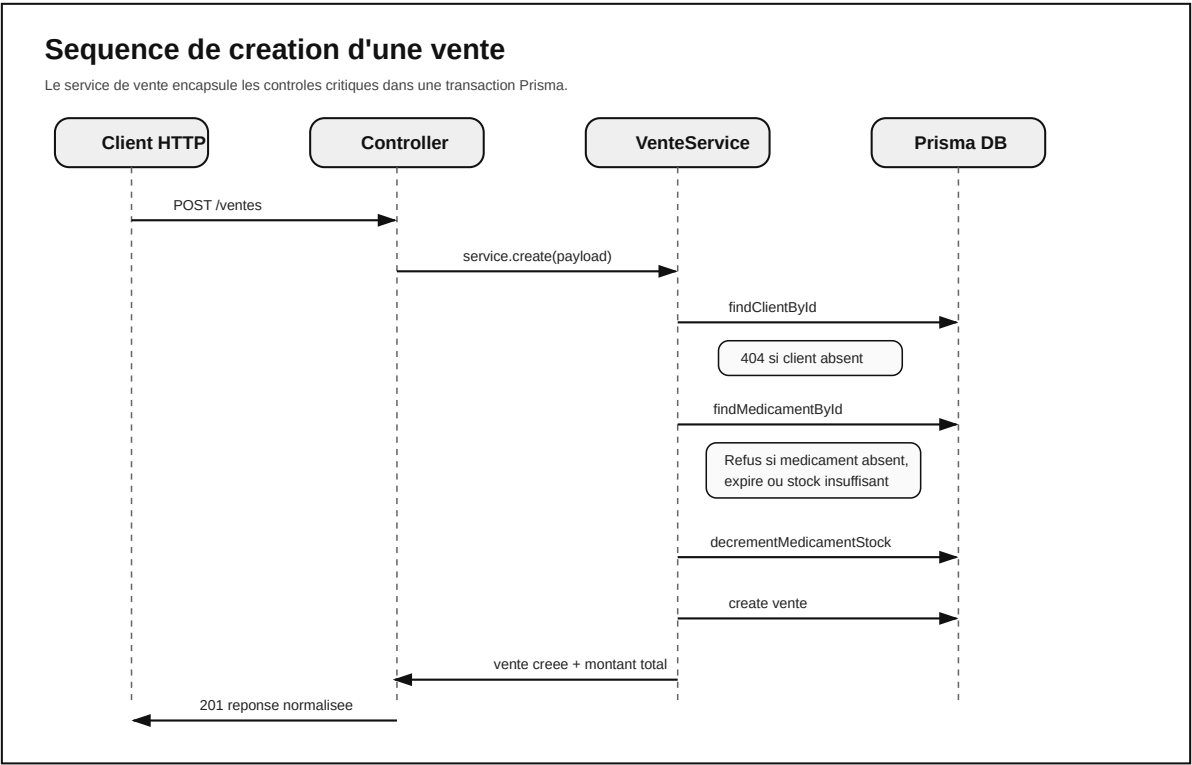


Figure 5.47: Séquence de création d'une vente

Ce module montre la maturité réelle du backend. Il ne fait pas une simple insertion de données. Il exécute une opération métier complète et contrôlée.

5.7. Synthèse des responsabilités métier

Module	Ce que le module veut accomplir	Ce que l'implémentation contrôle réellement
Auth	ouvrir une session technique	format de connexion, JWT, lecture de /me
Clients	gérer la base client	email unique, suppression protégée
Fournisseurs	gérer l'approvisionnement	code unique, suppression protégée
Médicaments	gérer le stock vendable	fournisseur valide, prix, stock, expiration
Ventes	enregistrer une sortie de stock	client, médicament, expiration, stock, transaction

6. Partie 4. Construction du modèle de données

6.1. Ce que nous voulons construire

Avant de coder les routes et les services, il faut construire la base logique du projet. Dans **GesPharmacie**, cette base est le modèle de données. Il répond à une question simple: quelles informations la pharmacie doit-elle mémoriser pour fonctionner correctement?

La réponse retenue dans ce projet est la suivante:

- un **Client** représente la personne à qui la pharmacie vend un produit;
- un **Fournisseur** représente l'entreprise qui livre les médicaments;
- un **Medicament** représente un produit stocké et vendu;
- une **Vente** représente une sortie de stock liée à un client et à un médicament.

Si cette base est mal pensée, tout le reste devient fragile. C'est pour cette raison que la construction du modèle de données vient ici juste après l'étude des modules.

6.2. Dépendances et commandes à connaître

Pour reconstruire cette partie depuis zéro, les dépendances principales sont:

- `@prisma/client` pour utiliser la base de données dans le code TypeScript;
- `prisma` pour décrire le schéma, générer le client et gérer les migrations;
- `postgresql` comme moteur de base de données.

Les commandes utiles sont:

- installation de la couche de données: `npm install @prisma/client`
- installation de l'outil Prisma: `npm install -D prisma`
- initialisation Prisma si on repart d'un projet vide: `npx prisma init`
- génération du client après modification du schéma: `npm run prisma:generate`
- création d'une migration en local: `npm run prisma:migrate`
- injection des données de démonstration: `npm run prisma:seed`

6.3. Ordre conseillé pour reconstruire la base

1. lister les entités métier dont l'application a besoin;
2. définir les relations entre ces entités;
3. écrire le fichier `prisma/schema.prisma`;
4. générer le client Prisma;

5. créer puis appliquer les migrations;
6. écrire un **seed** pour obtenir des données de démonstration;
7. vérifier ensuite que les modules du backend lisent bien ces données.

Cet ordre est important. On ne commence pas par les services ou par Swagger. On commence par la structure qui va supporter toutes les opérations métier.

6.4. Étape 1. Définir les tables, les relations et les contraintes

Le fichier principal de cette partie est `prisma/schema.prisma`. C'est lui qui dit à la fois:

- quelles tables existent;
- quels champs elles contiennent;
- quelles colonnes doivent rester uniques;
- quelles relations doivent être protégées.

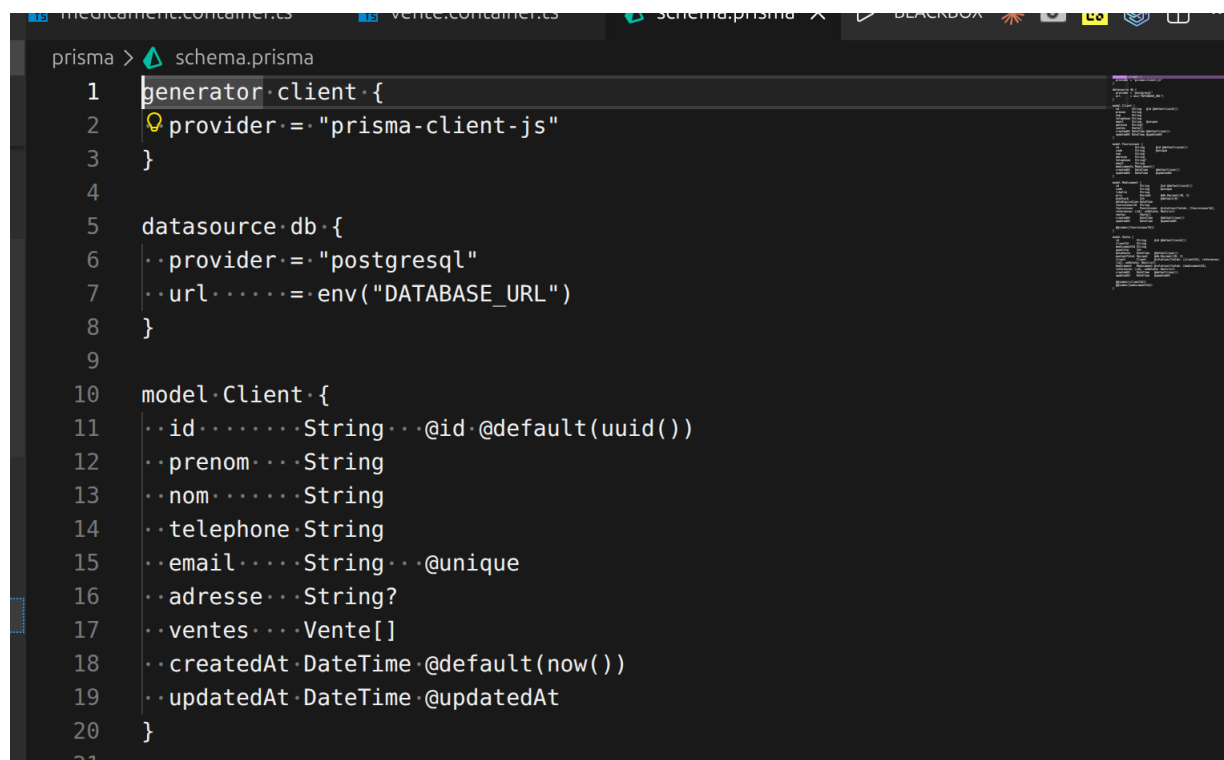


Figure 6.1: Capture réelle du schéma Prisma

Dans cette capture, on voit que le projet ne se contente pas de stocker des informations simples. Il impose aussi des règles:

- `Client.email` doit rester unique;
- `Fournisseur.code` doit rester unique;
- `Medicament.code` doit rester unique;
- les relations sensibles utilisent `onDelete: Restrict`.

Le choix `onDelete: Restrict` est particulièrement important. Il veut dire qu'on ne peut pas supprimer librement un élément encore utilisé ailleurs. Par exemple, on ne doit pas effacer un client si ce client apparaît déjà dans l'historique des ventes.

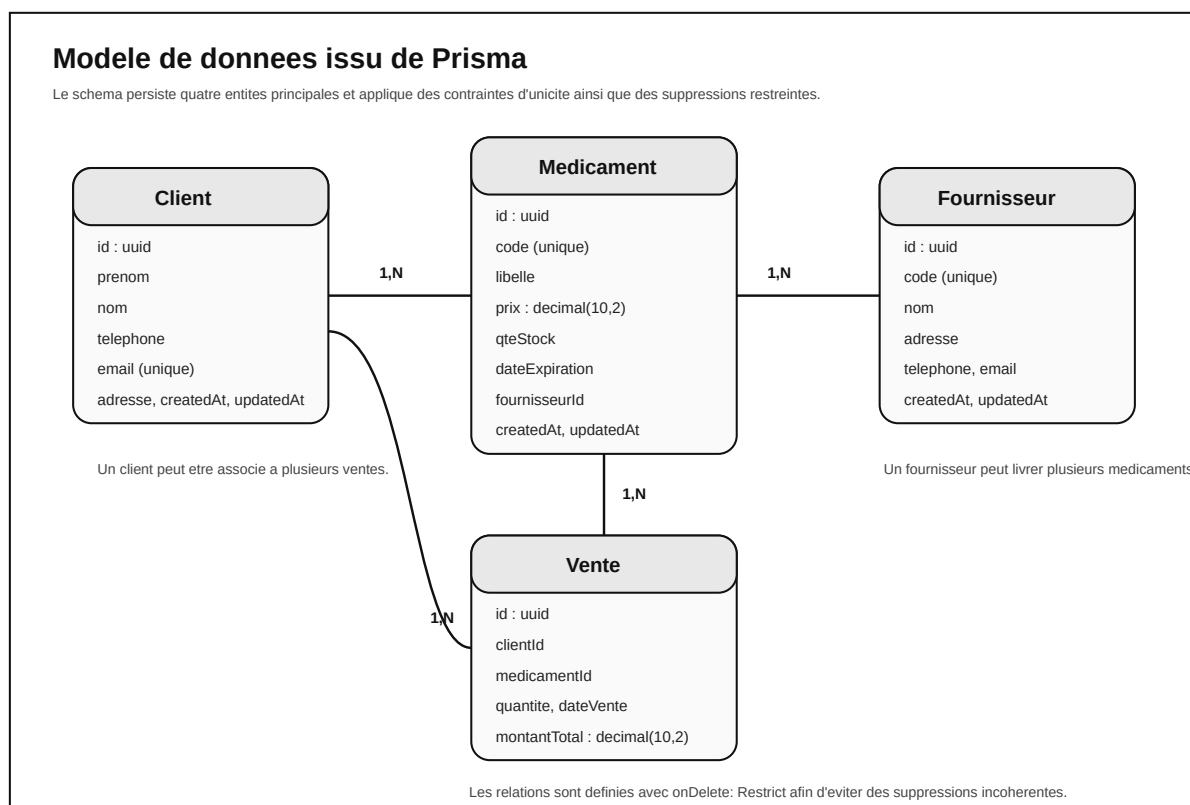


Figure 6.2: Schéma synthétique du modèle de données

6.5. Étape 2. Générer le client Prisma et les migrations

Une fois le schéma écrit, il ne suffit pas de le “regarder”. Il faut produire les outils qui permettront au backend de l'utiliser.

L'ordre correct est:

1. lancer `npm run prisma:generate` pour produire le client Prisma;
2. lancer `npm run prisma:migrate` pour créer et appliquer les migrations en local;
3. vérifier que la base contient bien les tables attendues.

Le rôle de cette étape est simple: transformer une description théorique en structure réellement exécutable par PostgreSQL et TypeScript.

6.6. Étape 3. Préparer un jeu de données de démonstration

Une base vide est difficile à tester. C'est pour cela que le projet contient `prisma/seed.ts`.

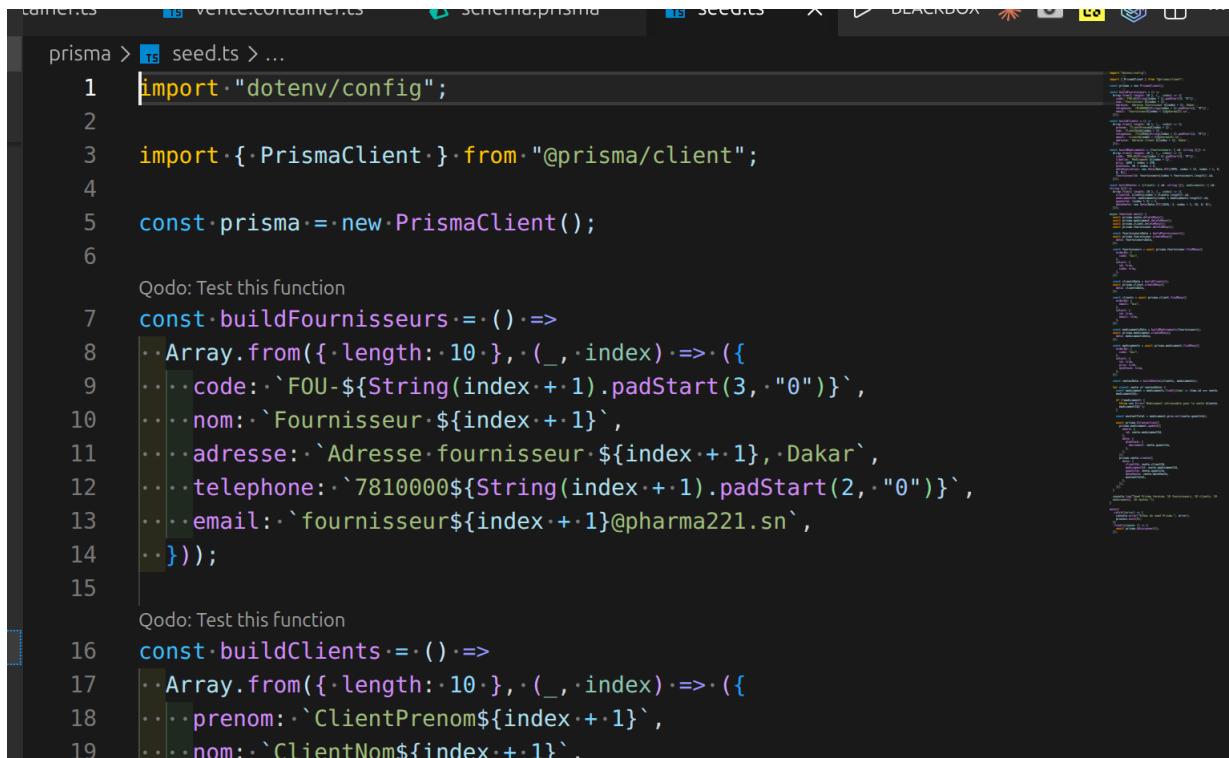


Figure 6.3: Capture réelle du seed Prisma

Ce fichier sert à injecter un jeu de données cohérent pour la démonstration:

- des fournisseurs;
- des clients;
- des médicaments;
- des ventes.

Le point important est que ce **seed** ne dépose pas seulement des lignes “au hasard”. Il respecte aussi la logique métier, notamment la diminution du stock au moment où des ventes sont créées. Cela rend les démonstrations et les tests plus crédibles.

6.7. Ce que fait chaque fichier de cette partie

Fichier ou dossier	Rôle simple
prisma/schema.prisma	décrire les tables, les relations et les contraintes
prisma/migrations/	conserver l'historique de transformation de la base
prisma/seed.ts	remplir la base avec des données de départ cohérentes
src/config/db.ts	exposer l'accès à la connexion base de données

Cette séparation est pédagogique. Un débutant comprend plus vite s'il sait exactement où regarder pour chaque responsabilité.

6.8. Ce qu'il faut vérifier à la fin

Quand cette partie est terminée, il faut pouvoir répondre “oui” à ces questions:

- les quatre entités principales existent-elles bien?
- les relations entre clients, fournisseurs, médicaments et ventes sont-elles claires?
- les contraintes d'unicité empêchent-elles les doublons importants?
- le **seed** permet-il de tester l'application sans saisir toutes les données à la main?

Si la réponse est non, il faut corriger cette partie avant de revenir aux modules.

7. Partie 5. Sécurité, validation et robustesse

7.1. Ce que nous voulons protéger

Une application backend ne doit pas seulement fonctionner. Elle doit aussi refuser ce qui est faux, dangereux ou incohérent.

Dans **GesPharmacie**, cette protection repose sur cinq idées:

- vérifier les variables d'environnement dès le démarrage;
- valider les entrées HTTP avant la logique métier;
- vérifier les jetons JWT sur les routes protégées;
- centraliser la gestion des erreurs;
- journaliser ce qui se passe pour comprendre les incidents.

7.2. Dépendances et commandes à connaître

Les dépendances les plus importantes de cette partie sont:

- **zod** pour la validation;
- **jsonwebtoken** pour les jetons JWT;
- **helmet** pour sécuriser les en-têtes HTTP;
- **cors** pour contrôler les échanges entre origines;
- **dotenv** pour charger les variables d'environnement;
- **pino** et **pino-pretty** pour la journalisation.

Commande minimale si on reconstruit cette partie dans un autre projet: `npm install zod jsonwebtoken helmet cors dotenv pino pino-pretty`.

7.3. Ordre conseillé pour reconstruire la sécurité

1. définir les variables d'environnement obligatoires;
2. sécuriser la base Express avec **helmet**, **cors** et `express.json()`;
3. écrire le middleware de validation des requêtes;
4. écrire le middleware d'authentification JWT;
5. écrire le middleware de gestion d'erreurs;
6. brancher un logger;
7. tester les cas d'erreur avant même de penser à la production.

7.4. Étape 1. Vérifier les variables d'environnement au démarrage

Avant de lancer le serveur, le projet vérifie que ses variables critiques existent et ont un format correct.

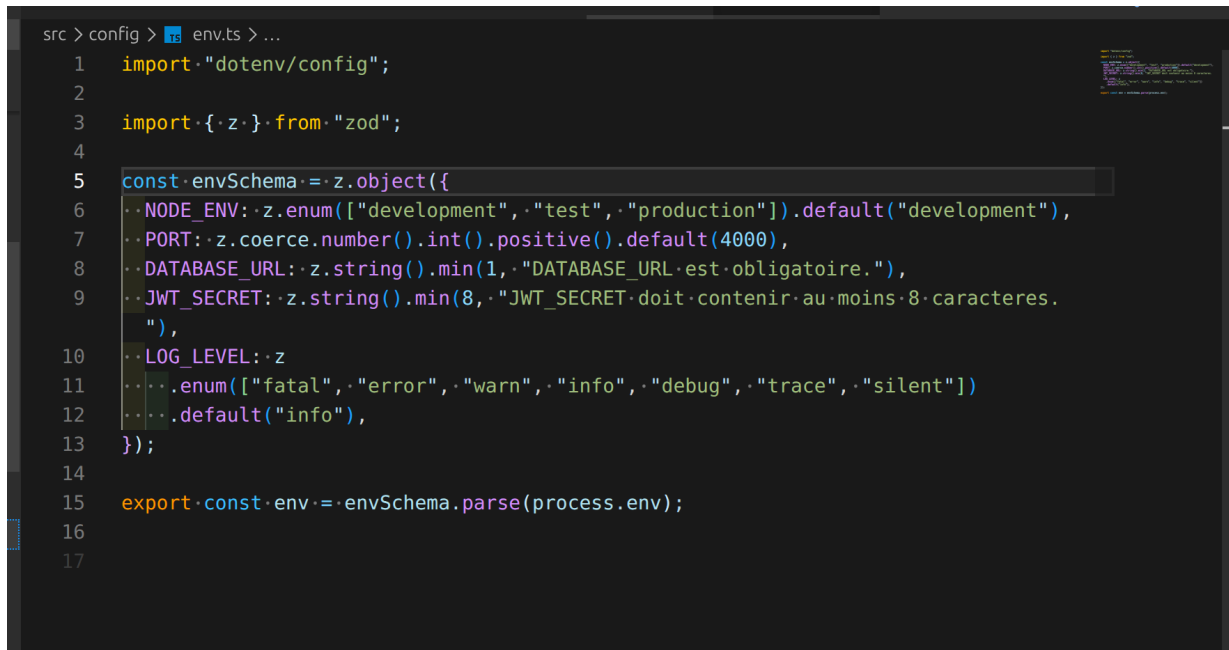


Figure 7.1: Capture réelle de la validation des variables d'environnement

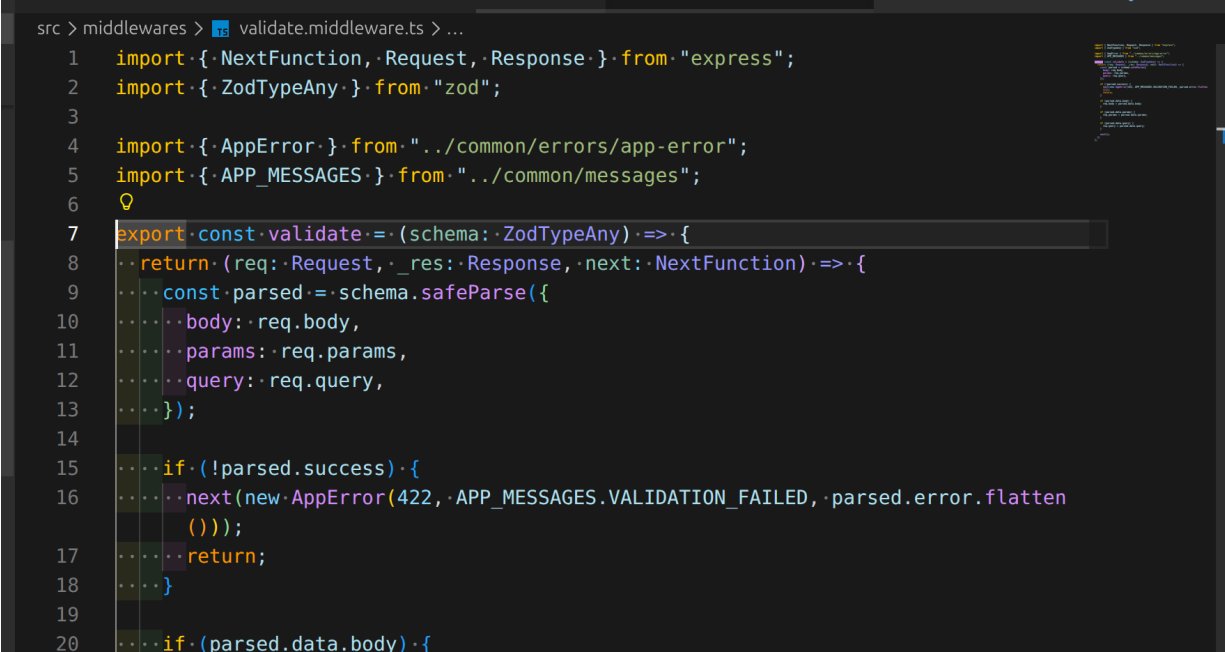
Cette étape évite un problème fréquent chez les débutants: démarrer une API alors qu'il manque DATABASE_URL ou JWT_SECRET. Ici, si la configuration est invalide, l'application s'arrête immédiatement. C'est beaucoup plus sûr.

Les variables principales sont:

Variable	Rôle
NODE_ENV	dire si l'application tourne en développement, test ou production
PORT	choisir le port d'écoute HTTP
DATABASE_URL	donner l'adresse de la base PostgreSQL
JWT_SECRET	signer et vérifier les jetons JWT
LOG_LEVEL	choisir le niveau de détail des logs

7.5. Étape 2. Contrôler les entrées HTTP

Quand une requête arrive, le backend ne doit pas la croire immédiatement. Il faut d'abord vérifier qu'elle contient les bons champs.



```
src > middlewares > validate.middleware.ts > ...
1  import { NextFunction, Request, Response } from "express";
2  import { ZodTypeAny } from "zod";
3
4  import { AppError } from "../common/errors/app-error";
5  import { APP_MESSAGES } from "../common/messages";
6  Q
7  export const validate = (schema: ZodTypeAny) => {
8    return (req: Request, _res: Response, next: NextFunction) => {
9      const parsed = schema.safeParse({
10        body: req.body,
11        params: req.params,
12        query: req.query,
13      });
14
15      if (!parsed.success) {
16        next(new AppError(422, APP_MESSAGES.VALIDATION_FAILED, parsed.error.flatten()));
17        return;
18      }
19
20      if (parsed.data.body) {
```

Figure 7.2: Capture réelle du middleware de validation

Le rôle de `validate.middleware.ts` est très simple à comprendre: il lit `body`, `params` et `query`, puis il demande à `Zod` si la forme de la requête est correcte. Si ce n'est pas le cas, il bloque la route et transmet une erreur claire.

Cette étape protège les contrôleurs et les services. Sans elle, on transmettrait trop vite des données incomplètes ou mal typées au cœur métier.

7.6. Étape 3. Vérifier les jetons JWT

Une fois la validation des données mise en place, il faut vérifier l'identité de l'appelant sur les routes qui exigent une authentification.

```

src > middlewares > auth.middleware.ts > authMiddleware
1  import { NextFunction, Request, Response } from "express";
2  import jwt from "jsonwebtoken";
3
4  import { AppError } from "../common/errors/app-error";
5  import { APP_MESSAGES } from "../common/messages";
6  import { env } from "../config/env";
7
8  type JwtPayload = {
9    role?: string;
10 };
11
12 export const authMiddleware = (req: Request, _res: Response, next: NextFunction) => {
13   const authorization = req.headers.authorization;
14
15   if (!authorization?.startsWith("Bearer ")) {
16     next(new AppError(401, APP_MESSAGES.UNAUTHORIZED));
17     return;
18   }
19
20   const token = authorization.replace("Bearer ", "").trim();

```

Figure 7.3: Capture réelle du middleware JWT

Le middleware lit l'en-tête **Authorization**, vérifie qu'il commence bien par **Bearer**, puis tente de décoder le jeton avec **JWT_SECRET**. Si le jeton est correct, il ajoute **req.user** à la requête.

Il faut toutefois bien noter une limite actuelle du projet: à ce stade du code, la protection JWT est surtout utilisée pour **GET /auth/me**. Les autres modules métiers ne sont pas encore entièrement placés derrière ce filtre.

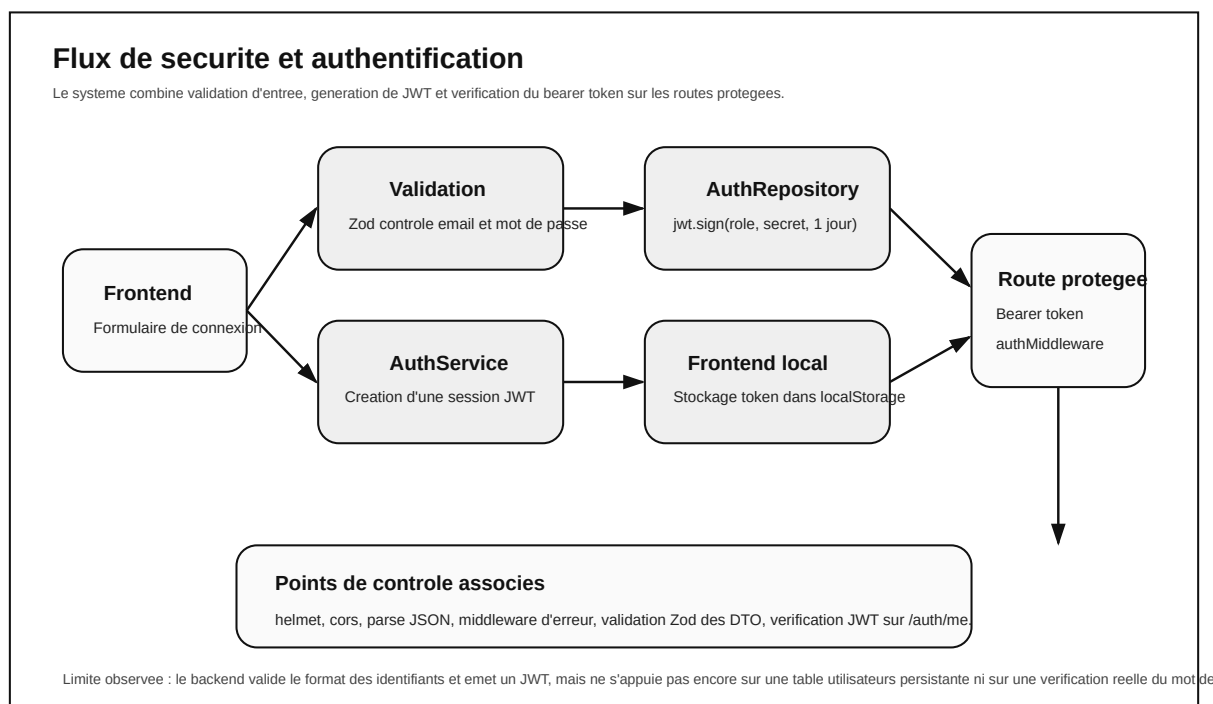


Figure 7.4: Schéma du flux de sécurité

7.7. Étape 4. Gérer les erreurs proprement

Quand un projet commence à grandir, il devient dangereux de laisser chaque route traiter ses erreurs à sa manière. Il faut une règle commune.

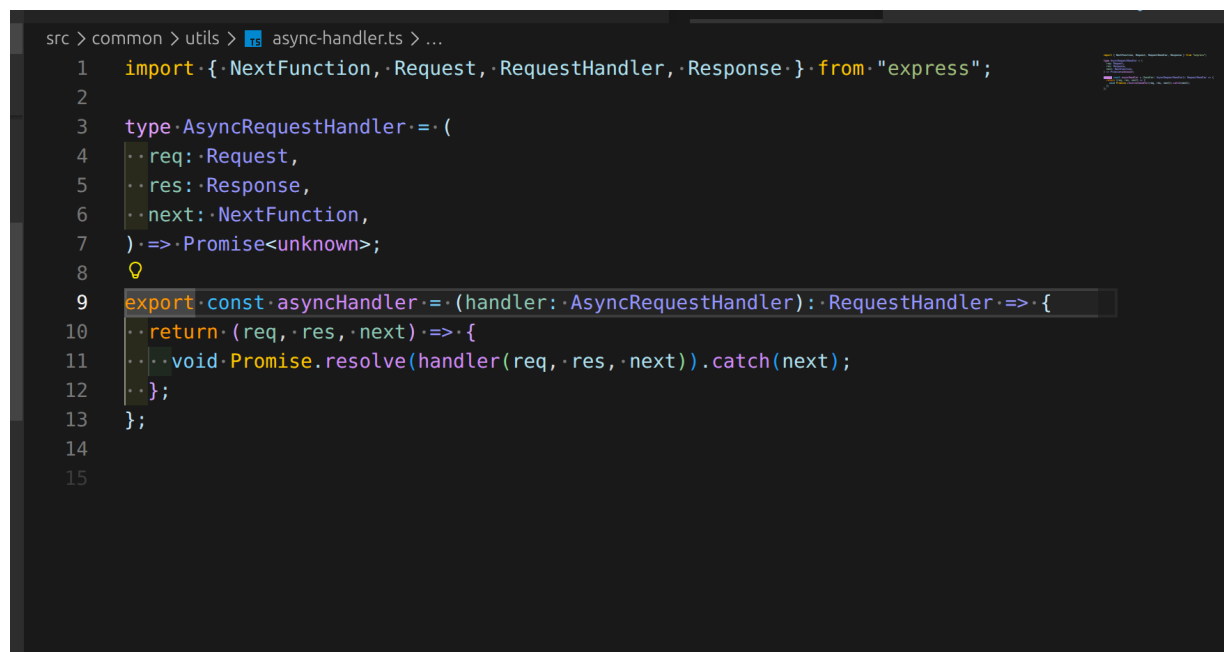


Figure 7.5: Capture réelle du wrapper asynchrone

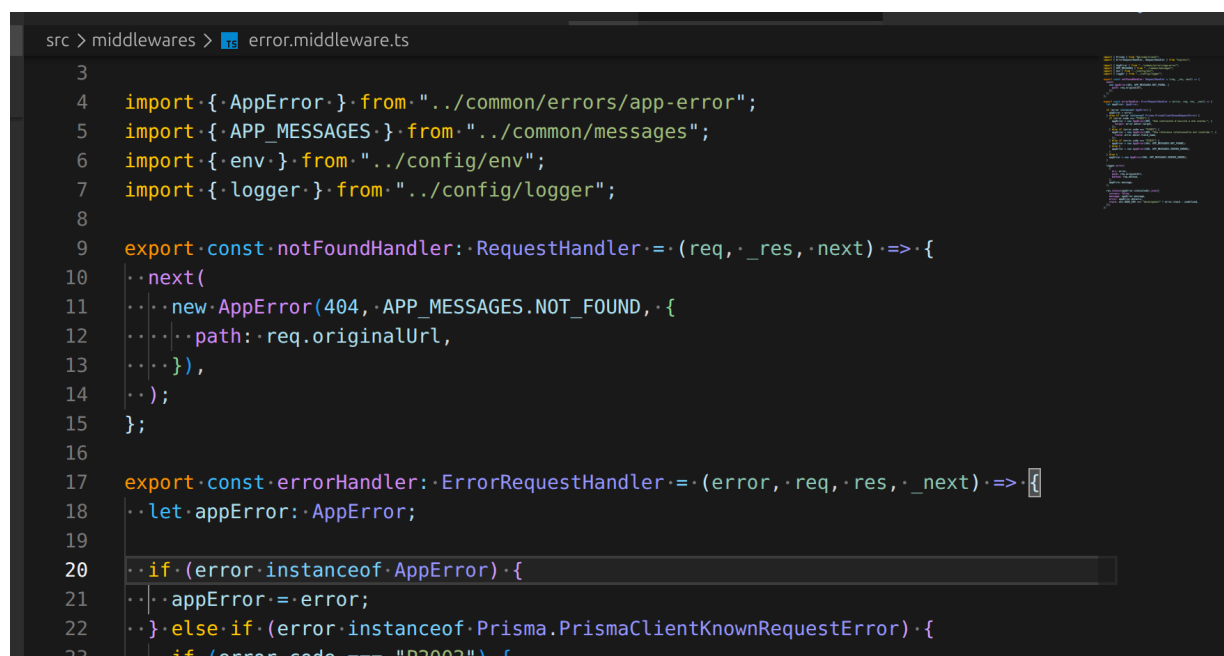


Figure 7.6: Capture réelle du middleware global d'erreur

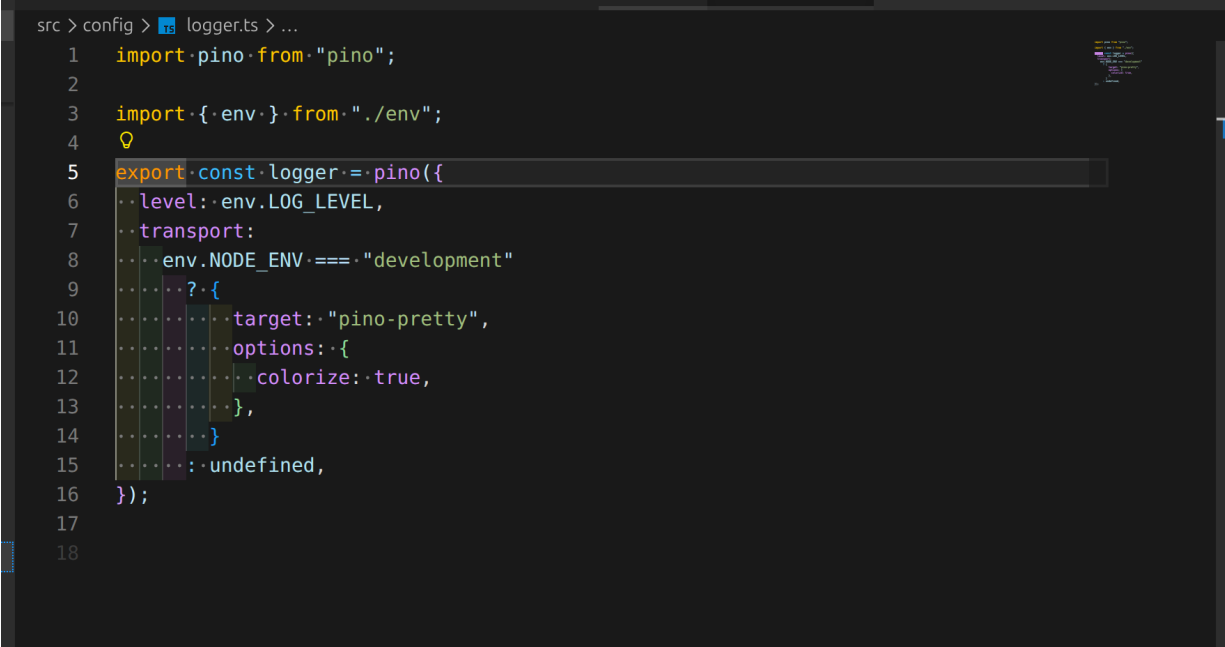
Le fonctionnement devient alors plus simple:

1. une route déclenche une erreur;
2. l'erreur est transmise au pipeline Express;
3. le middleware `errorHandler` transforme cette erreur en réponse JSON homogène.

Ce système rend les retours beaucoup plus lisibles pour le frontend ou pour Swagger.

7.8. Étape 5. Journaliser ce qui se passe

Une application sérieuse doit raconter ce qu'elle fait. C'est le rôle du logger.



```
src > config > logger.ts > ...
1  import pino from "pino";
2
3  import { env } from "../env";
4
5  export const logger = pino({
6    level: env.LOG_LEVEL,
7    transport:
8      env.NODE_ENV === "development"
9        ? {
10          target: "pino-pretty",
11          options: {
12            colorize: true,
13          },
14        }
15        : undefined,
16  });
17
18
```

Figure 7.7: Capture réelle de la configuration Pino

En développement, `pino-pretty` rend les messages plus lisibles. En production, les logs restent structurés et donc plus faciles à exploiter dans un système d'observabilité.

7.9. Ce qu'il faut retenir

Cette partie montre une idée importante: la sécurité n'est pas un seul fichier magique. C'est une chaîne.

- l'environnement doit être valide;
- la requête doit être propre;
- le jeton doit être fiable;
- l'erreur doit être explicable;
- le serveur doit laisser une trace compréhensible.

Si l'une de ces étapes manque, le backend devient beaucoup plus difficile à maintenir.

8. Partie 6. Documentation d'API et observabilité

8.1. Ce que nous voulons rendre visible

Quand le backend est construit, il faut encore le rendre compréhensible pour d'autres personnes. C'est le rôle de la documentation d'API.

Dans ce projet, cette partie doit permettre:

- de voir toutes les routes disponibles;
- de comprendre le format des entrées et des sorties;
- de tester les endpoints sans écrire de frontend;
- d'exposer en ligne le PDF du mémoire avec `/pdf`;
- de vérifier rapidement que le service répond avec `/api/v1/health`.

8.2. Dépendances et commandes à connaître

Les éléments principaux sont:

- `swagger-ui-express` pour afficher Swagger UI;
- `@types/swagger-ui-express` pour le typage TypeScript;
- la route `/docs/openapi.json` pour exposer le contrat brut;
- la route `/pdf` pour afficher le mémoire directement dans le navigateur.

Commande minimale pour mettre en place Swagger dans un autre projet: `npm install swagger-ui-express @types/swagger-ui-express`.

8.3. Ordre conseillé pour reconstruire cette partie

1. écrire le document OpenAPI;
2. exposer ce document en JSON;
3. brancher Swagger UI sur `/docs`;
4. brancher la route `/pdf`;
5. garder un endpoint de santé très simple;
6. vérifier ensuite toutes ces routes dans le navigateur.

8.4. Étape 1. Décrire l'API dans OpenAPI

Le fichier `src/docs/openapi.ts` joue le rôle de contrat global.

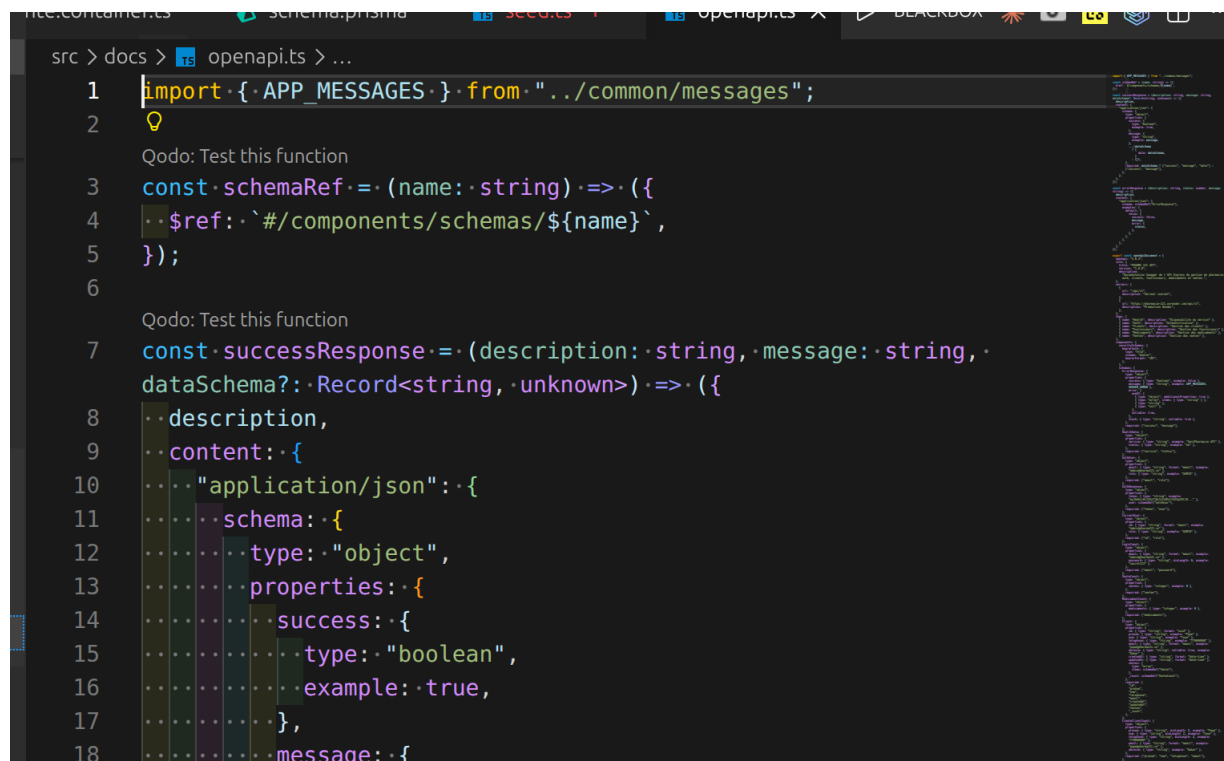


Figure 8.1: Capture réelle du document OpenAPI

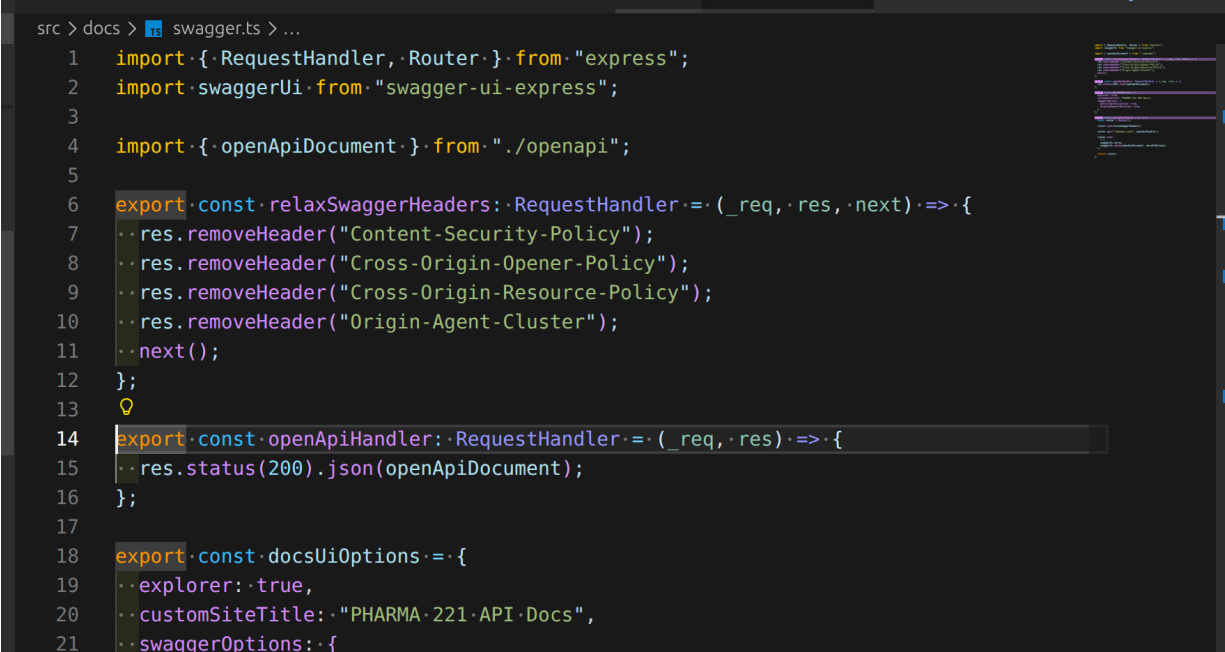
Ce fichier dit:

- quel est le nom de l'API;
- quelles routes existent;
- quels objets sont attendus;
- quels schémas de sécurité sont disponibles;
- quels serveurs sont déclarés.

Une personne qui ne connaît pas le projet peut donc comprendre l'API sans lire tous les modules un par un.

8.5. Étape 2. Publier Swagger UI

Une fois le contrat OpenAPI écrit, il faut l'exposer dans une interface que l'on peut utiliser directement.



```
src > docs > swagger.ts > ...
1 import {·RequestHandler,·Router·}·from·"express";
2 import swaggerUi·from·"swagger-ui-express";
3
4 import {·openApiDocument·}·from·"./openapi";
5
6 export·const·relaxSwaggerHeaders:·RequestHandler·=·(_req,·res,·next)·=>·{
7   ··res.removeHeader("Content-Security-Policy");
8   ··res.removeHeader("Cross-Origin-Opener-Policy");
9   ··res.removeHeader("Cross-Origin-Resource-Policy");
10  ··res.removeHeader("Origin-Agent-Cluster");
11  ··next();
12 };
13
14 export·const·openApiHandler:·RequestHandler·=·(_req,·res)·=>·{
15   ··res.status(200).json(openApiDocument);
16 };
17
18 export·const·docsUiOptions·=·{
19   ··explorer:·true,
20   ··customSiteTitle:·"PHARMA·221·API·Docs",
21   ··swaggerOptions:·{
```

Figure 8.2: Capture réelle du routeur Swagger

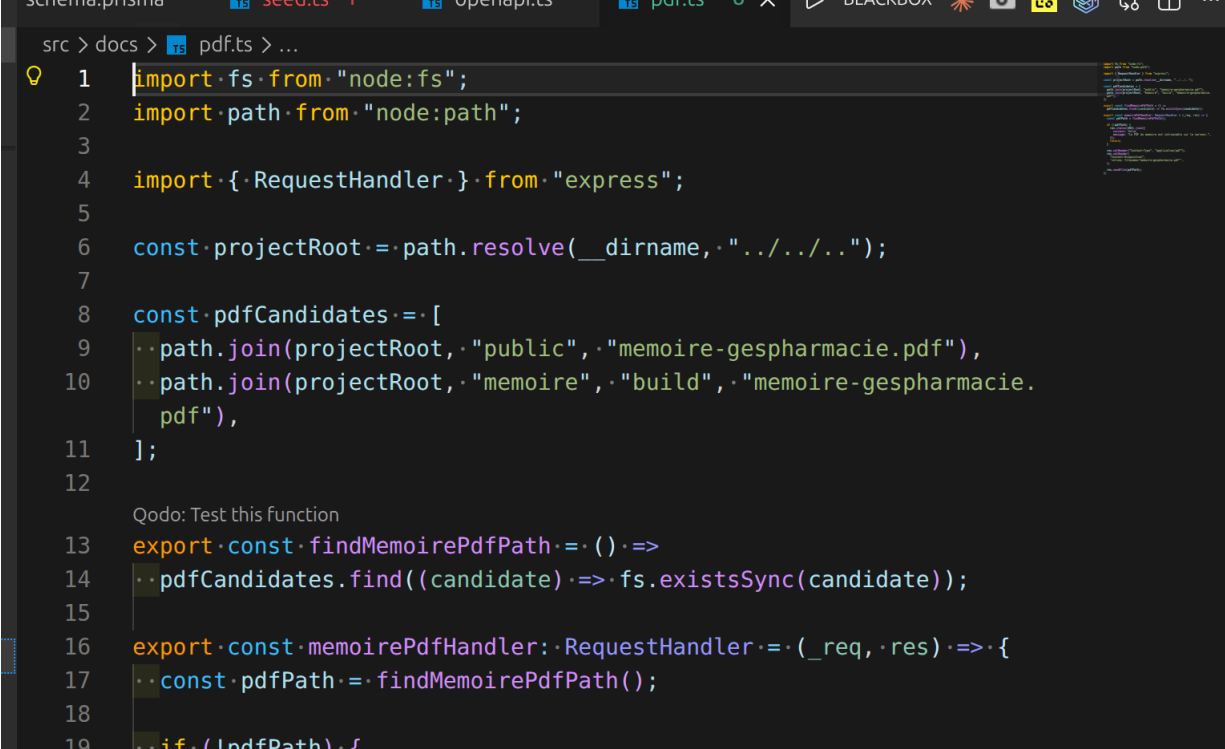
Le rôle de `swagger.ts` est simple:

- servir `openapi.json`;
- préparer les options de Swagger UI;
- monter l'interface interactive sur `/docs`.

Cette étape est essentielle pour la soutenance. Elle transforme le backend en système montrable, testable et vérifiable.

8.6. Étape 3. Publier le mémoire en ligne avec `/pdf`

Le projet ne s'arrête pas aux endpoints métier. Il publie aussi le mémoire directement sur le backend.



```

src > docs > pdf.ts > ...
1  import fs from "node:fs";
2  import path from "node:path";
3
4  import { RequestHandler } from "express";
5
6  const projectRoot = path.resolve(__dirname, "../..");
7
8  const pdfCandidates = [
9    path.join(projectRoot, "public", "memoire-gespharmacie.pdf"),
10   path.join(projectRoot, "memoire", "build", "memoire-gespharmacie.pdf"),
11 ];
12
13  Qodo: Test this function
14  export const findMemoirePdfPath = () =>
15     pdfCandidates.find((candidate) => fs.existsSync(candidate));
16
17  export const memoirePdfHandler: RequestHandler = (_req, _res) => {
18     const pdfPath = findMemoirePdfPath();
19     if (!pdfPath) {

```

Figure 8.3: Capture réelle du handler `/pdf`

Le principe est facile à comprendre:

1. le backend cherche le fichier PDF du mémoire;
2. s'il le trouve, il renvoie ce PDF avec le bon `Content-Type`;
3. sinon, il renvoie une erreur 404 claire.

Cette idée est utile dans un contexte académique, car elle permet d'avoir le service et la documentation finale sur la même application.

8.7. Étape 4. Vérifier les points d'accès publics

L'URL de publication communiquée pour ce projet est <https://pharmacie-221.onrender.com/>. À partir de cette base, les points d'accès importants sont:

Point d'accès	Usage
https://pharmacie-221.onrender.com/docs	interface Swagger
https://pharmacie-221.onrender.com/docs/openapi.json	contrat OpenAPI brut
https://pharmacie-221.onrender.com/api/v1/health	route de santé
https://pharmacie-221.onrender.com/pdf	mémoire en ligne

Les captures conservées dans ce mémoire montrent la documentation Swagger du projet avec les principaux groupes d'endpoints.

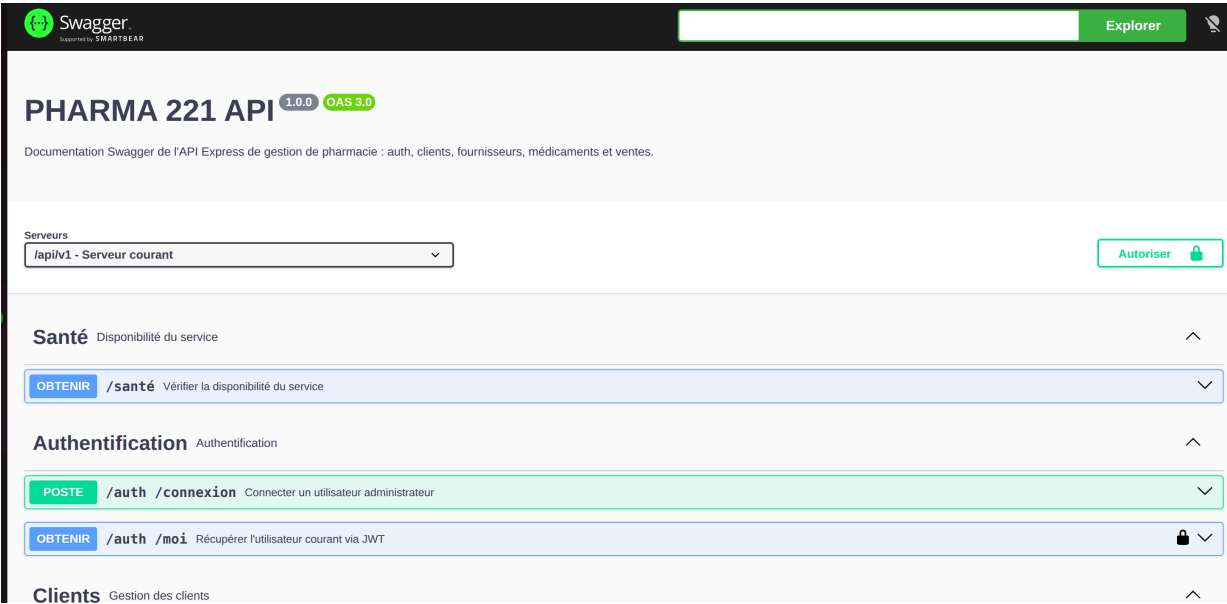


Figure 8.4: Vue générale de la documentation Swagger



Figure 8.5: Vue des routes clients et fournisseurs dans Swagger

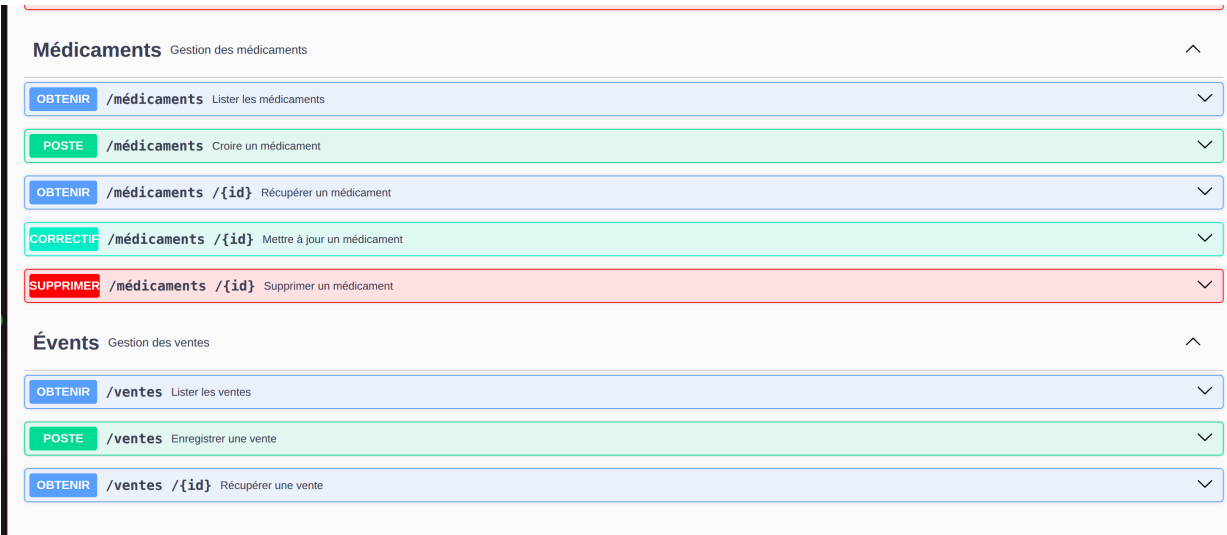


Figure 8.6: Vue des routes médicaments et ventes dans Swagger



Figure 8.7: Première vue des schémas publiés par Swagger



Figure 8.8: *Deuxième vue des schémas publiés par Swagger*

8.8. Ce qu'il faut retenir

Dans ce projet, l'observabilité reste volontairement simple, mais elle existe réellement:

- les logs aident à comprendre ce qui se passe;
- `/api/v1/health` confirme que le service répond;
- `/docs` rend le contrat visible;
- `/pdf` rend la documentation finale consultable en ligne.

Pour un projet académique, cette base est déjà très solide.

9. Partie 7. Qualité logicielle, tests et intégration continue

9.1. Ce que nous voulons sécuriser

Quand le backend commence à fonctionner, une nouvelle question apparaît: comment éviter de casser l'existant en ajoutant une nouvelle fonctionnalité?

La réponse du projet est double:

- écrire des tests unitaires autour des règles métier;
- automatiser la vérification dans GitHub Actions.

9.2. Dépendances et commandes à connaître

Les outils principaux sont:

- `vitest` pour les tests;
- `supertest` pour tester le comportement HTTP;
- `@types/supertest` pour le typage.

Commandes importantes:

- installation: `npm install -D vitest supertest @types/supertest`
- exécution des tests: `npm run test`
- exécution en mode watch: `npm run test:watch`

9.3. Ordre conseillé pour construire la qualité logicielle

1. choisir les scénarios les plus risqués;
2. écrire les tests unitaires sur les services;
3. vérifier la route de santé et la documentation;
4. lancer la suite localement;
5. automatiser cette vérification dans GitHub Actions;
6. ne déployer qu'après succès des tests.

9.4. Étape 1. Écrire les tests autour des règles métier

Le dépôt contient un dossier `tests/unit` qui couvre:

- la santé générale de l'application;

- la documentation d'API;
- le service d'authentification;
- les services `client`, `fournisseur`, `medicament` et `vente`.

Le dossier `tests/integration` existe déjà, mais il est encore vide. Cela signifie que le projet a préparé l'emplacement pour la suite, même si l'effort principal est encore placé sur les tests unitaires.

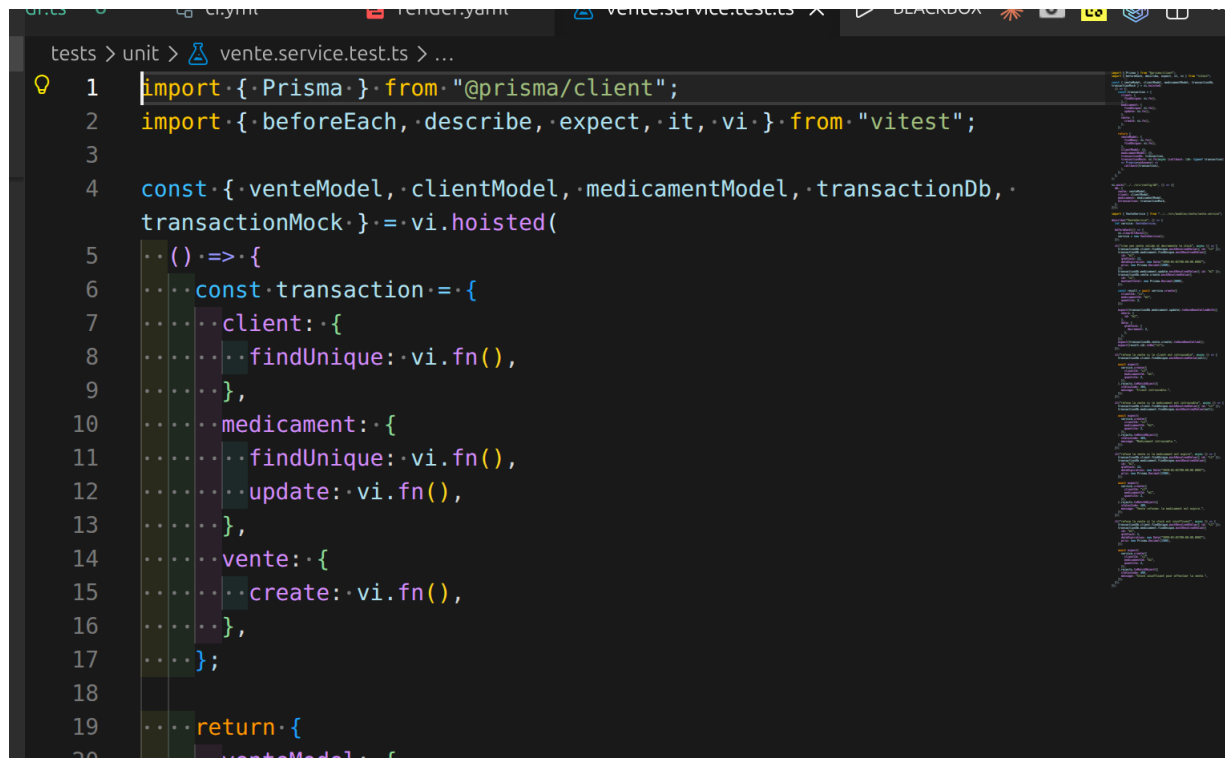


Figure 9.1: Capture réelle d'un test métier sur les ventes

Le module `vente` est un bon exemple, car c'est là que les règles métier sont les plus sensibles. Les tests y vérifient notamment:

- le cas nominal;
- le client introuvable;
- le médicament introuvable;
- le médicament expiré;
- le stock insuffisant.

9.5. Étape 2. Exécuter la suite localement

Une fois les tests écrits, il faut lancer la suite complète avec `npm run test`.

● ● ● Capture technique 2. Sortie des tests unitaires

```
> gestpharmacieapi-express-221@0.1.0 test
> vitest run

RUN v2.1.9 /home/.../GestPharmacie

✓ tests/unit/vente.service.test.ts (5 tests)
✓ tests/unit/medicament.service.test.ts (6 tests)
✓ tests/unit/client.service.test.ts (5 tests)
✓ tests/unit/fournisseur.service.test.ts (7 tests)
✓ tests/unit/auth.service.test.ts (1 test)
✓ tests/unit/docs.test.ts (3 tests)
✓ tests/unit/app.test.ts (1 test)

Test Files 7 passed (7)
Tests 28 passed (28)
Duration 2.72s
```

Source: execution locale réelle de npm test durant la preparation du memoire.

Figure 9.2: *Capture de l'exécution locale des tests*

Cette étape est indispensable. Un test non exécuté n'est pas une preuve. Il faut voir le résultat passer avant d'aller vers la CI.

9.6. Étape 3. Automatiser la vérification dans GitHub Actions

Le workflow du dépôt reprend exactement la logique attendue dans une petite API professionnelle.

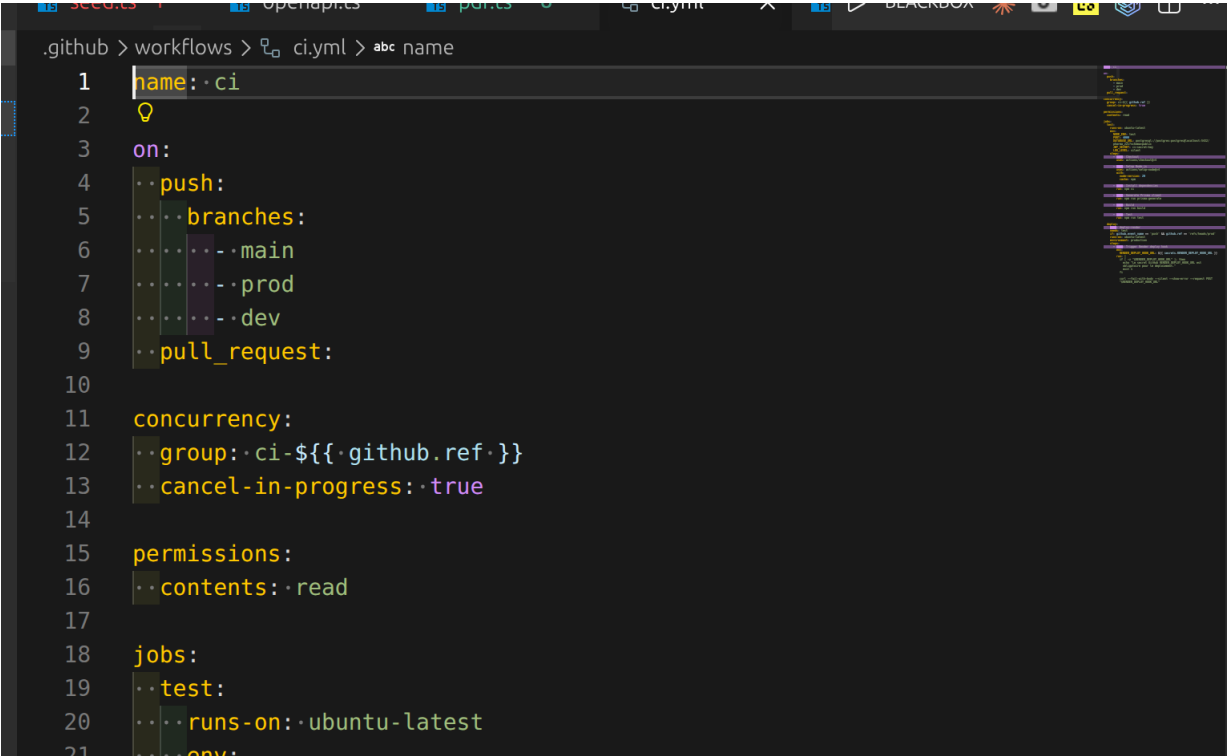


Figure 9.3: Capture réelle du workflow GitHub Actions

Le job `test` installe les dépendances, génère le client Prisma, compile TypeScript puis lance les tests. Le job `deploy` n'intervient qu'après ce contrôle et seulement dans une condition bien précise.

9.7. Étape 4. Comprendre la logique du pipeline

Élément du pipeline	Ce qu'il fait
push sur main, dev, prod	déclenche la vérification
pull_request	contrôle les changements avant fusion
job test	installe, génère Prisma, build et teste
job deploy	déclenche Render uniquement après succès du job test

Ce point est important pour un débutant: la CI n'est pas "magique". Elle répète simplement, sur un serveur GitHub, les mêmes étapes qu'on devrait savoir exécuter localement.

9.8. Ce qu'il faut retenir

La qualité logicielle du projet repose sur une idée très saine: avant de parler de production, on prouve d'abord que les règles métier les plus critiques se comportent comme prévu.

10. Partie 8. Déploiement et exploitation

10.1. Ce que nous voulons obtenir

Après avoir construit les modules, la base de données, la sécurité, la documentation et les tests, il reste une dernière étape technique: rendre le backend réellement exécutable.

Cette partie a deux objectifs:

- permettre un lancement local reproductible;
- permettre un déploiement public sur Render.

10.2. Dépendances et commandes à connaître

Les commandes principales de cette étape sont:

- installation des dépendances: `npm ci`
- génération Prisma: `npm run prisma:generate`
- migration locale: `npm run prisma:migrate`
- seed local: `npm run prisma:seed`
- lancement local: `npm run dev`
- build de production: `npm run build`
- exécution Docker locale: `docker compose -f docker/docker-compose.yml up --build`

10.3. Ordre conseillé pour passer du local à la production

1. préparer `.env`;
2. démarrer PostgreSQL en local ou avec Docker;
3. générer le client Prisma;
4. appliquer les migrations et injecter le seed;
5. démarrer le serveur local;
6. construire le build TypeScript;
7. décrire le service Render;
8. vérifier la santé de l'API et la documentation après publication.

10.4. Étape 1. Vérifier les scripts et le démarrage local

Le fichier `package.json` contient les scripts qui structurent tout le cycle de vie du backend.

=20"
9 },
10 "scripts": {
11 "dev": "tsx watch src/server.ts",
12 "build": "tsc -p tsconfig.json",
13 "start": "node dist/src/server.js",
14 "start:render": "npm run prisma:migrate:deploy && npm run start",
15 "test": "vitest run",
16 "test:watch": "vitest",
17 "agent:codex-guard": "bash scripts/codex-guard.sh",
18 "agent:codex-guard:local": "CODEX_GUARD_FORCE_LOCAL=1 bash scripts/codex-guard.sh",
19 "prisma:generate": "prisma generate",
20 "prisma:migrate": "prisma migrate dev"
21 }
22 }

Figure 10.1: Capture des scripts principaux du projet

Ensuite, `src/server.ts` orchestre le vrai démarrage:

Figure 10.2: Capture du démarrage du serveur

Cette séparation est bonne à retenir:

- `package.json` dit quelles commandes lancer;
- `server.ts` dit comment démarrer réellement l'API;
- `app.ts` assemble les middlewares et les routes.

10.5. Étape 2. Préparer l'exécution locale avec Docker

Le projet propose aussi une exécution locale plus structurée grâce à Docker.

Le fichier `docker/docker-compose.yml` sert à lancer:

- l'API;
- la base PostgreSQL locale.

Le fichier `docker/Dockerfile` sert à construire une image Node.js capable de builder puis d'exécuter l'application. Cette option est utile pour reproduire un environnement proche de la production.

10.6. Étape 3. Décrire la production dans `render.yaml`

Une fois le projet stable en local, il faut décrire à la plateforme comment le construire et comment le lancer.

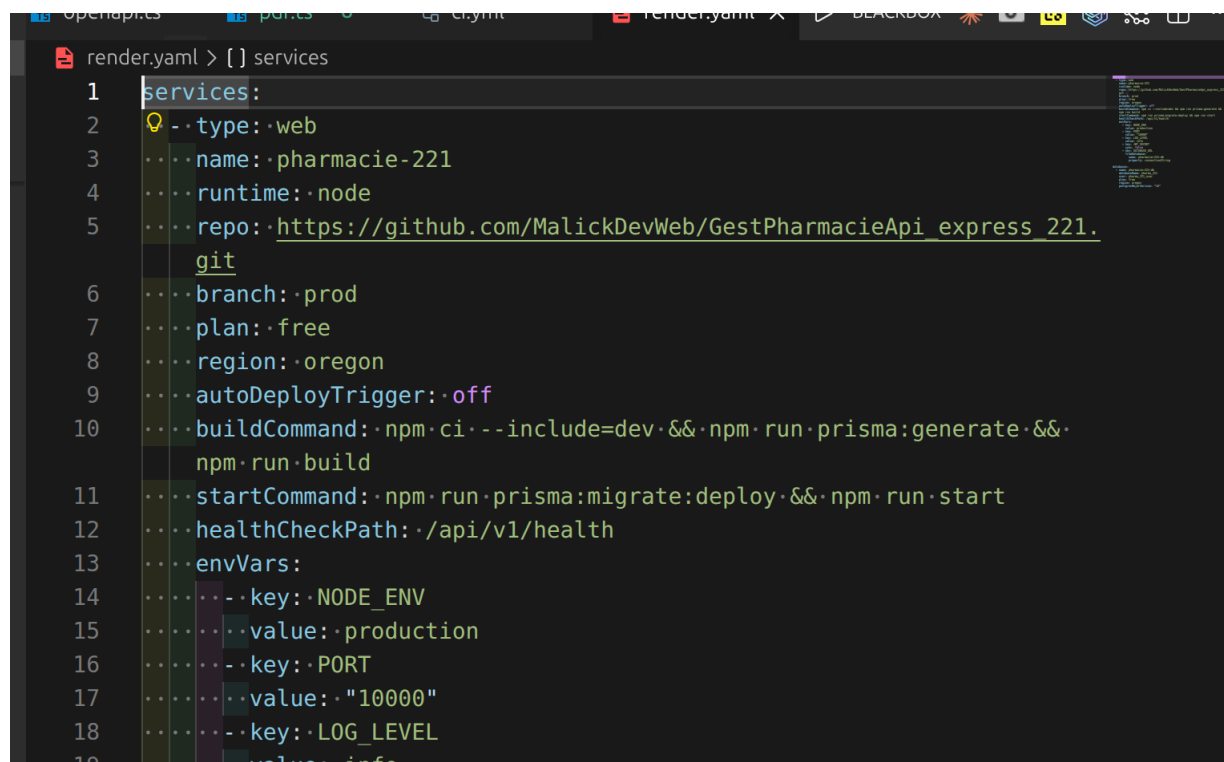


Figure 10.3: Capture réelle du blueprint Render

Ce fichier indique notamment:

- le nom du service;
- la branche de déploiement `prod`;
- la commande de build;
- la commande de démarrage;
- la route de santé `/api/v1/health`;
- la base de données PostgreSQL associée.

Le point pédagogique à retenir est le suivant: un déploiement sérieux n'est pas fait "à la main" à chaque fois. Il est décrit dans un fichier reproductible.

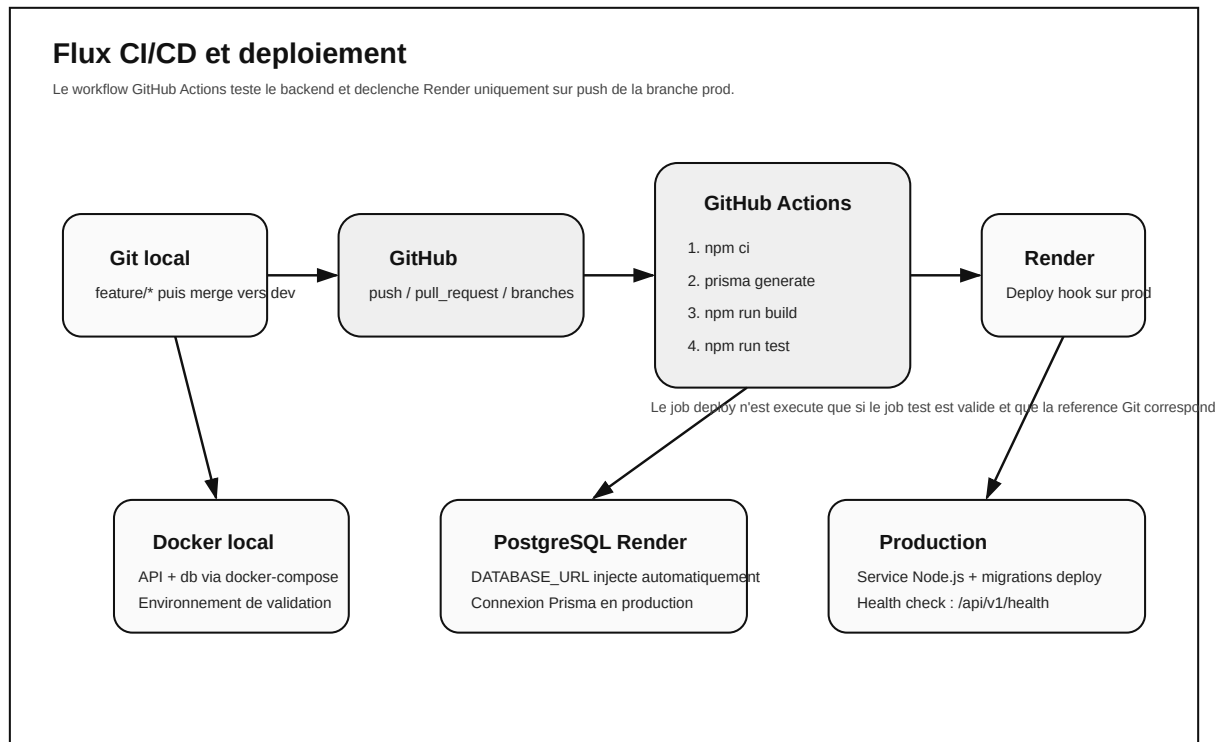


Figure 10.4: Schéma synthétique du flux CI/CD et déploiement

10.7. Ce que fait chaque fichier de déploiement

Fichier	Rôle simple
<code>package.json</code>	expose les commandes de build, test, seed et démarrage
<code>src/server.ts</code>	connecte la base puis démarre l'API
<code>docker/docker-compose.yml</code>	lance l'API et PostgreSQL en local
<code>docker/Dockerfile</code>	construit une image exécutable du backend
<code>render.yaml</code>	décrit le service publié sur Render

10.8. Ce qu'il faut vérifier à la fin

Quand le backend est déployé, il faut vérifier au minimum:

- que la route de santé répond;
- que Swagger est visible;
- que le PDF du mémoire peut s'ouvrir via `/pdf`;
- que les modules métier sont présents dans `/docs`;
- que la configuration Render correspond bien à la branche `prod`.

11. Partie 9. Démonstration technique backend

11.1. Comment démontrer le projet dans le bon ordre

Une démonstration technique réussie ne consiste pas à cliquer au hasard. Il faut suivre un ordre logique pour que même une personne non développeuse comprenne ce qui se passe.

L'ordre conseillé pour présenter ce backend est:

1. montrer d'abord l'objectif général du projet;
2. ouvrir Swagger pour faire voir les modules disponibles;
3. tester l'authentification;
4. tester un fournisseur;
5. tester un médicament;
6. tester un client;
7. tester une vente;
8. montrer ensuite la route `/health`;
9. terminer par `/pdf` pour présenter le mémoire en ligne.

11.2. Les points d'accès utiles pour la soutenance

Point d'accès public	Rôle dans la démonstration
https://pharmacie-221.onrender.com/	base du backend
https://pharmacie-221.onrender.com/docs	porte d'entrée principale de la démonstration
https://pharmacie-221.onrender.com/docs/openapi.json	preuve du contrat machine-readable
https://pharmacie-221.onrender.com/api/v1/health	preuve minimale que le service répond
https://pharmacie-221.onrender.com/pdf	mémoire consultable en ligne

11.3. Étape 1. Montrer la documentation interactive

La première chose à montrer à un jury ou à un correcteur est l'interface Swagger, car elle résume visuellement le périmètre du backend.

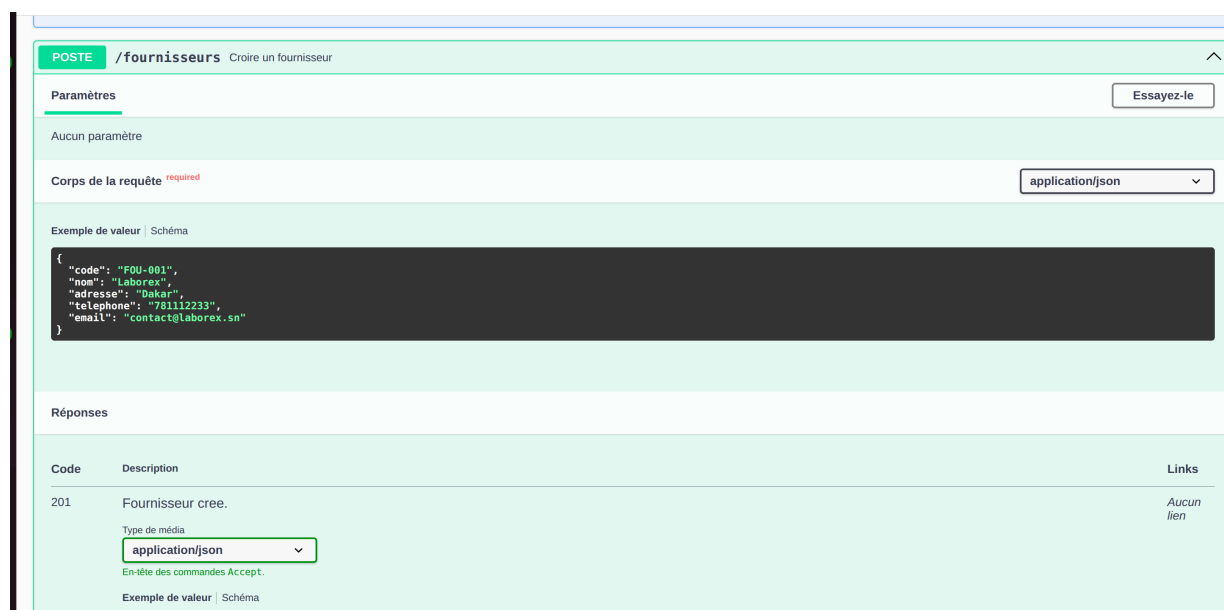


Figure 11.1: Capture Swagger de création d'un fournisseur

Cette vue prouve qu'on peut déjà interagir avec le module `fournisseurs`.

11.4. Étape 2. Enchaîner avec les médicaments

Une fois le fournisseur compris, on peut passer au module `medicaments`, car il dépend directement de cette entité.

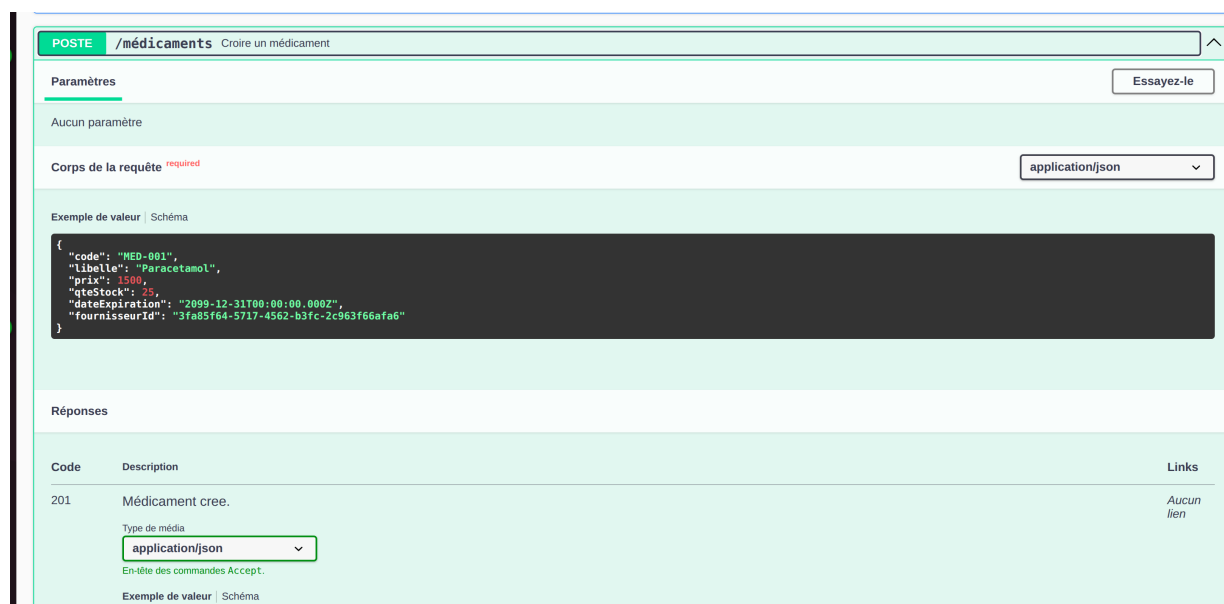


Figure 11.2: Capture Swagger de création d'un médicament

Le lecteur voit alors que la progression est logique:

- on crée d'abord la source d'approvisionnement;
- ensuite on crée le produit qui sera vendu.

11.5. Étape 3. Terminer par la vente

La vente est la meilleure preuve fonctionnelle du backend, car elle mobilise plusieurs modules à la fois.

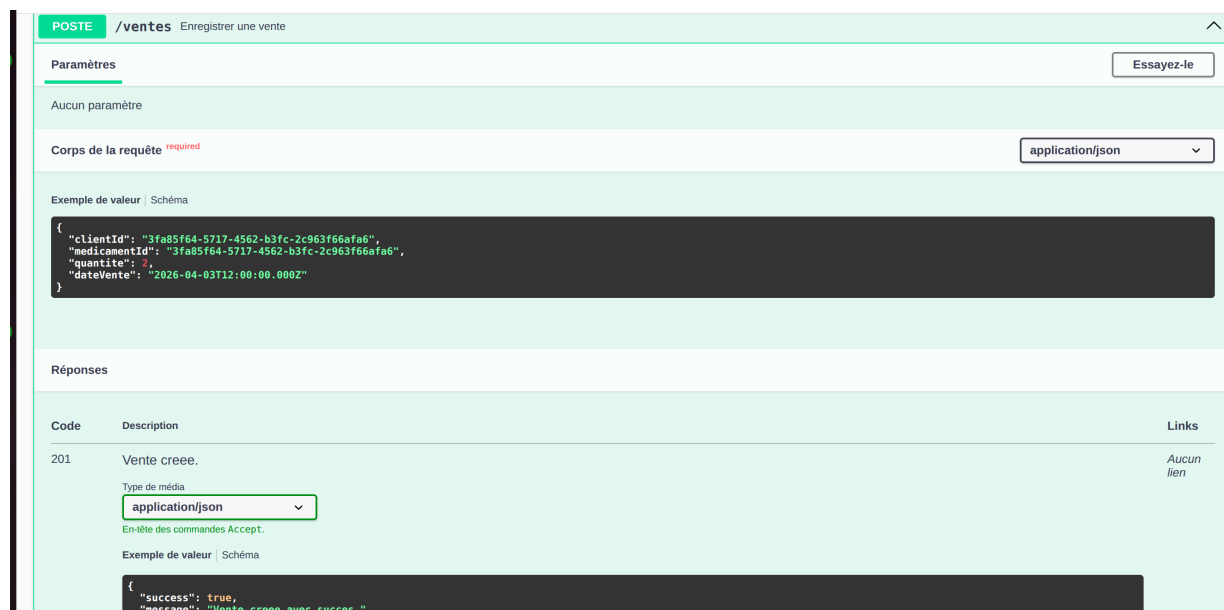


Figure 11.3: Capture Swagger de création d'une vente

Quand cette route fonctionne, cela signifie qu'un grand nombre d'éléments ont déjà été correctement mis en place:

- le client existe;
- le médicament existe;
- le stock est suffisant;
- le produit n'est pas expiré;
- la transaction métier est cohérente.

11.6. Ce que prouve réellement cette démonstration

Cette démonstration technique ne montre pas seulement "des routes qui répondent". Elle prouve en réalité quatre choses:

- le backend est structuré et documenté;
- la base de données supporte correctement le domaine;
- les règles métier importantes sont codées;
- le projet peut être présenté en ligne de manière professionnelle.

Pour une soutenance, cette logique est beaucoup plus forte qu'une simple liste de fichiers ou qu'une description trop abstraite.

12. Limites actuelles et perspectives

12.1. Limites identifiées

L'analyse du code met en évidence plusieurs limites importantes:

1. L'authentification ne repose pas encore sur une table d'utilisateurs persistante.
2. La vérification du mot de passe n'est pas implémentée contre une base réelle.
3. La protection JWT ne couvre actuellement que la route `/auth/me`.
4. La suite de tests est essentiellement unitaire; les tests d'intégration HTTP et base de données restent à développer.
5. Le périmètre métier est centré sur les entités principales, sans gestion avancée des rôles, de l'audit ou des statistiques.

12.2. Perspectives d'évolution

Les évolutions prioritaires les plus cohérentes seraient les suivantes:

- introduire une table `User` avec mot de passe haché;
- protéger toutes les routes métier sensibles par JWT;
- ajouter une autorisation par rôle (`ADMIN`, `PHARMACIEN`, `CAISSIER`);
- enrichir les ventes avec une historisation plus détaillée;
- introduire des tests d'intégration sur base PostgreSQL éphémère;
- compléter l'observabilité en production.

Ces perspectives prolongent naturellement l'architecture actuelle. Elles ne remettent pas en cause les fondations choisies; elles les renforcent.

13. Conclusion générale

Le backend GesPharmacie présente une base de travail sérieuse, cohérente et bien structurée. Son architecture modulaire permet de distinguer clairement le transport HTTP, la logique métier et la persistance. Les décisions les plus solides du projet résident dans:

- la structuration régulière des modules;
- la centralisation des validations;
- la protection de l'intégrité métier;
- l'usage de Prisma pour formaliser les relations;
- la présence d'une documentation OpenAPI et d'une CI exécutable;
- la mise en place d'une logique transactionnelle sur la vente.

Le projet n'est pas encore un backend pleinement industrialisé. L'authentification et la couverture sécuritaire doivent être renforcées. Néanmoins, du point de vue académique et logiciel, la solution démontre déjà une compréhension correcte des principes d'architecture, de qualité et de modélisation nécessaires à un système de gestion de pharmacie.

En ce sens, ce backend constitue une base crédible pour une soutenance technique: il montre non seulement ce qui a été codé, mais aussi la logique qui a guidé les choix d'implémentation.

14. Annexes techniques

14.1. Annexe A. Variables et configuration

Élément	Valeur ou rôle
Runtime cible	Node.js >=20
Langage	TypeScript
ORM	Prisma
Base de données	PostgreSQL
Documentation	Swagger UI + OpenAPI 3.0.3
Logger	Pino
Tests	Vitest
Déploiement	Render
URL backend publiée	Instance Render
Exécution locale conteneurisée	Docker Compose

14.2. Annexe B. Extraits de preuves d'exécution

- Build backend exécuté avec succès via `npm run build`.
- Tests backend exécutés avec succès via `npm test`.
- Workflow CI analysé dans `.github/workflows/ci.yml`.
- Schéma relationnel analysé dans `prisma/schema.prisma`.
- Documentation d'API analysée dans `src/docs/openapi.ts` et `src/docs/swagger.ts`.